ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Neural Network Architectures for LLMs

ONE LOVE. ONE FUTURE.

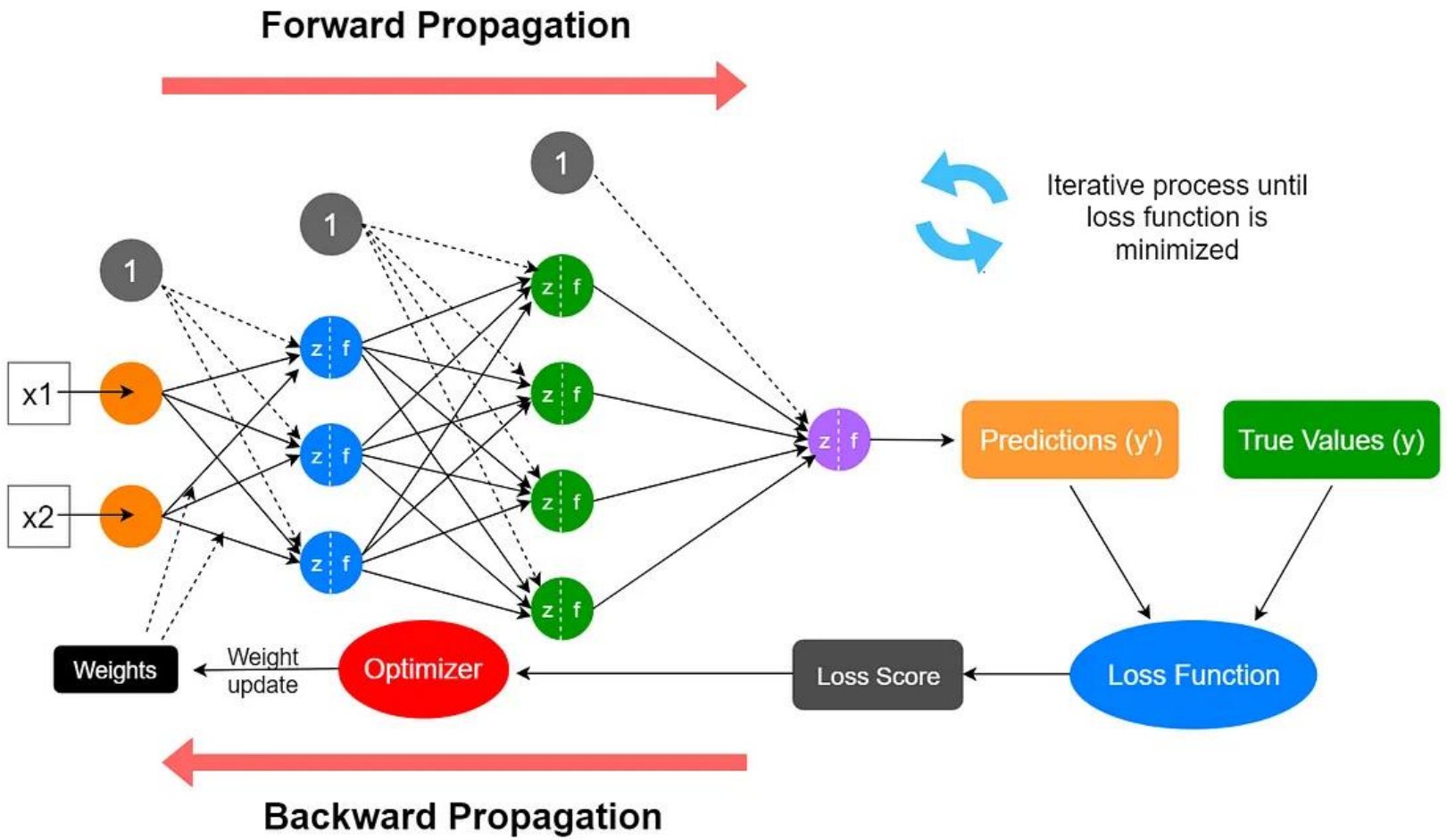
Contents

HUST

hust.edu.vn fb.com/dhbkhn

Neural Network and optimization

Neural Networks



Loss function

Really complex neural networks!!

input image

loss

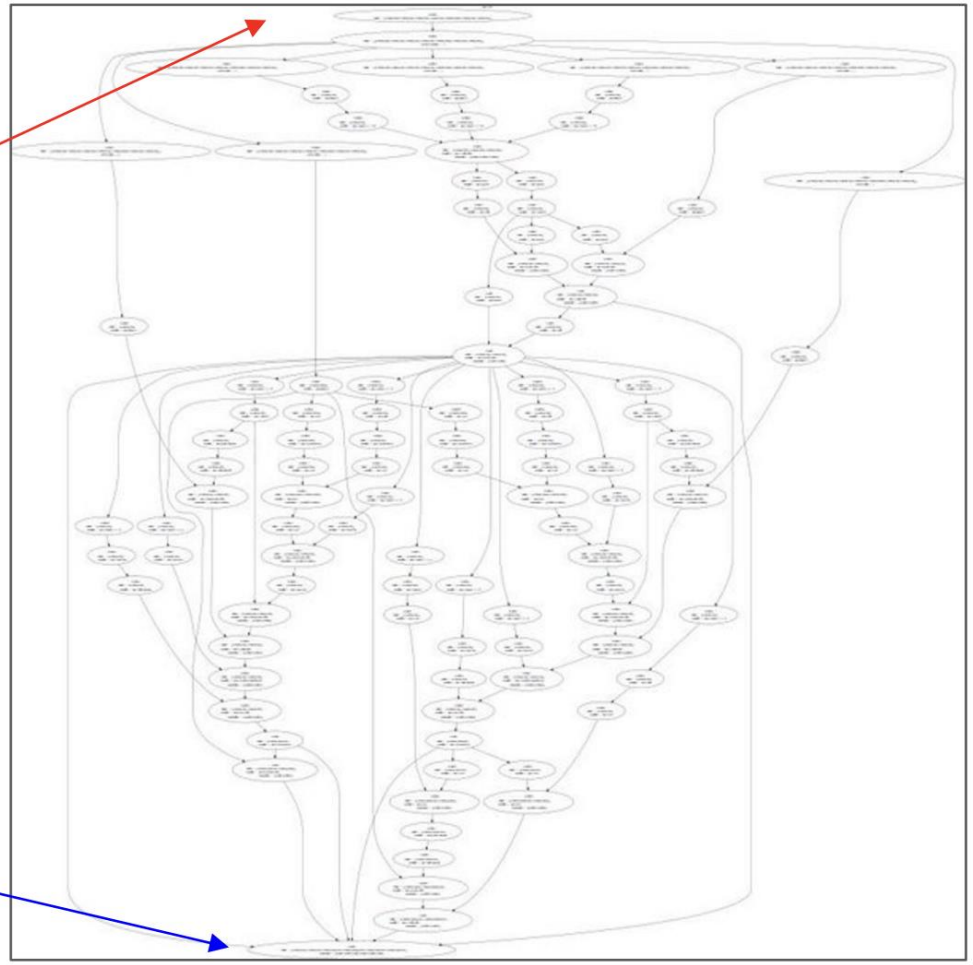


Figure reproduced with permission from a Twitter post by Andrej Karpathy.

Loss function

λ = regularization strength (hyperparameter)

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

More complex:

Dropout

Batch normalization

Stochastic depth, fractional pooling, etc

Optimization

https://www.youtube.com/watch?v=Q3pTEtSEvDI

Backpropagation and SGD

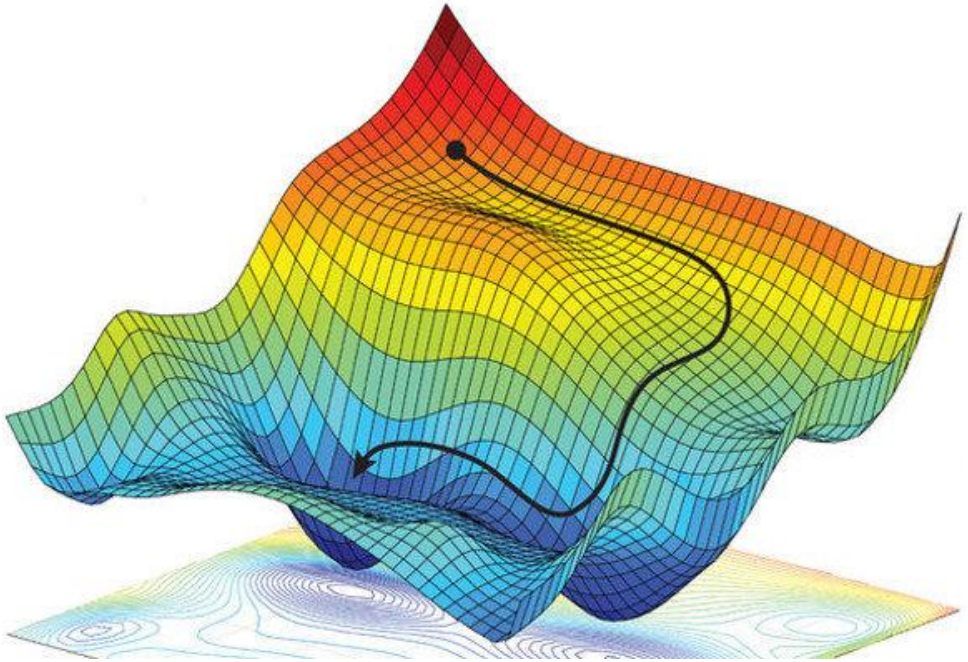
Algorithm

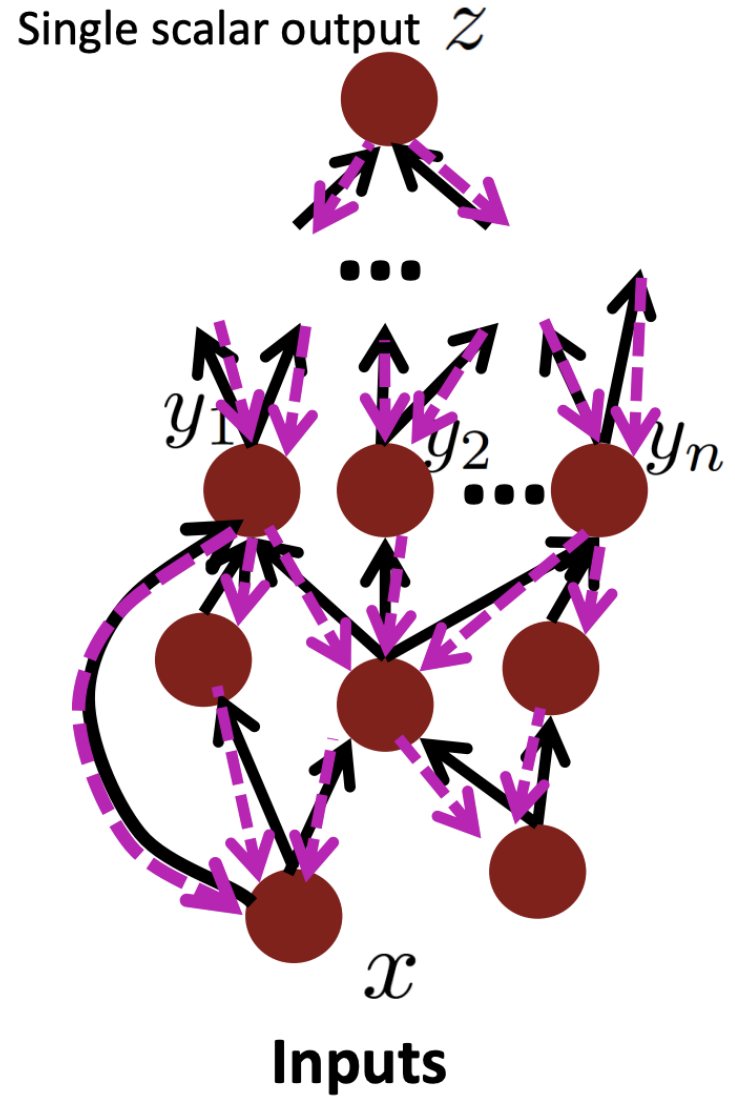
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(W)}{\partial W}$
4. Update weights, $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$
5. Return weights

```
weights = tf.random_normal(shape, stddev=sigma)
```

```
grads = tf.gradients(ys=loss, xs=weights)
```

```
weights_new = weights.assign(weights - lr * grads)
```





Backpropagation and SGD

HUST

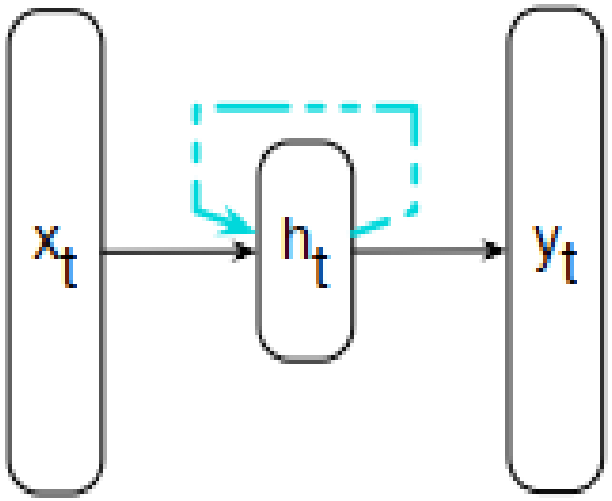
hust.edu.vn fb.com/dhbkhn

RNN and seq2seq

RNNs

RNN = Recurrent Neural Network

Recurrent: Some unit within the network is indirectly or directly takes its *own* earlier outputs as an input



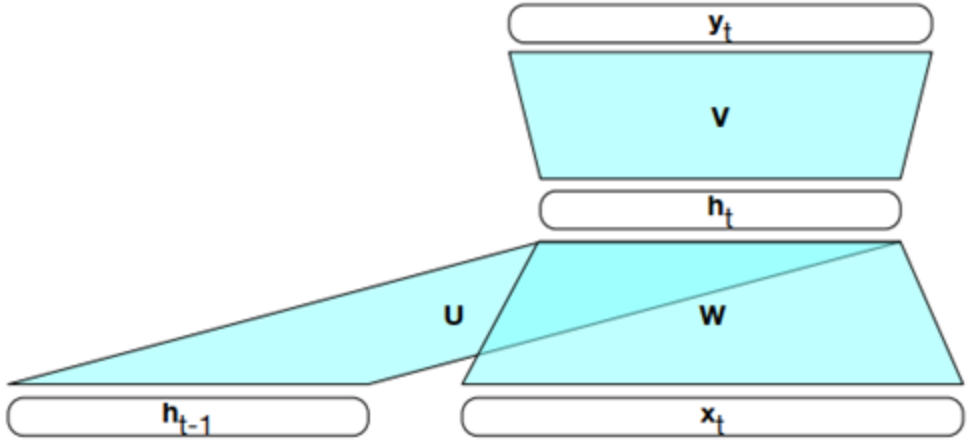
RNNs - Forward Inference

To map a sequence of inputs to outputs, we need to find the activation value for the hidden layer $\mathbf{h}_t$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

Then we can calculate the output vector normally

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$



Input and Output for RNN language models

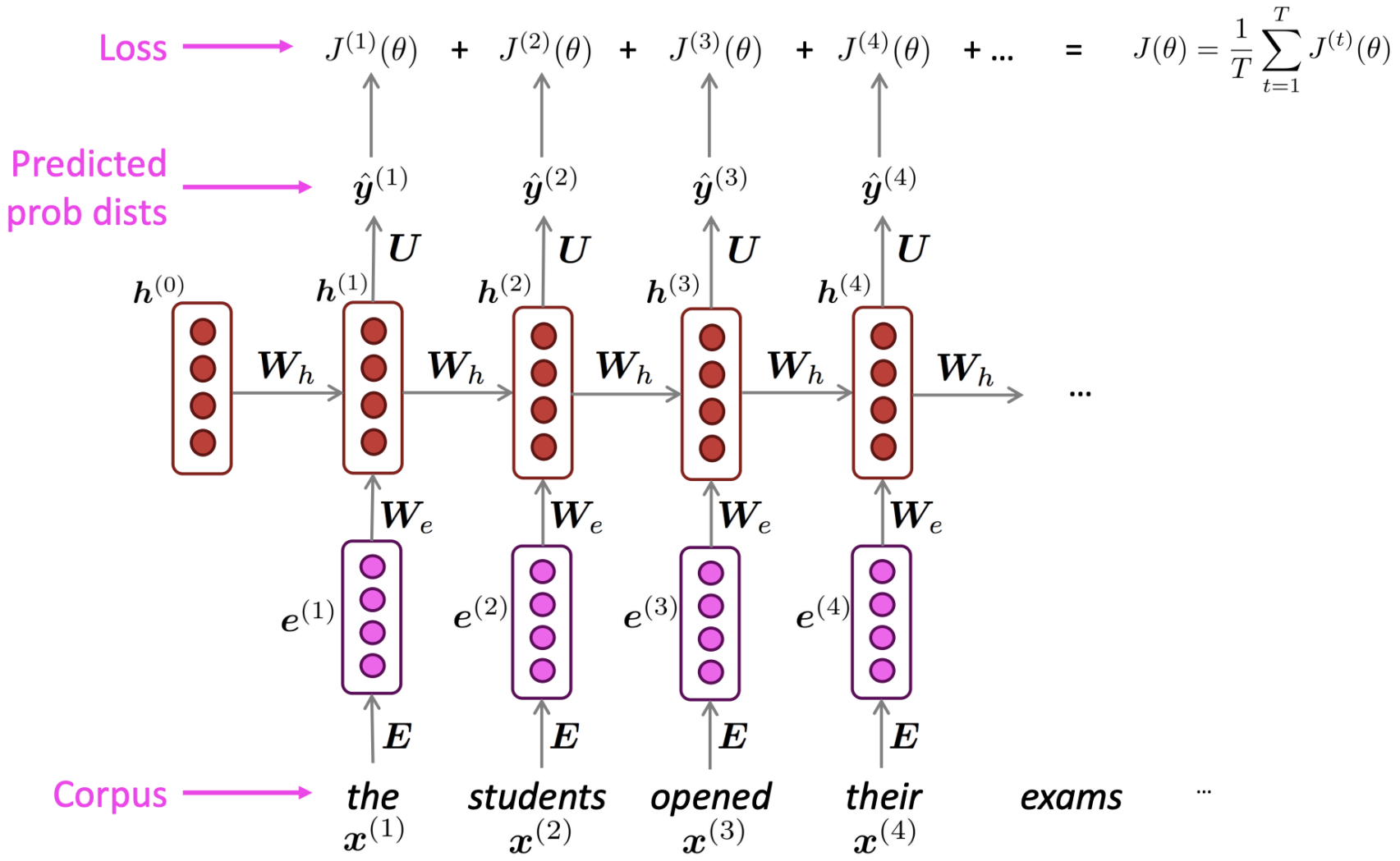
Input: A sequence of words

- RNN sees the sequence of words as a series of *word embeddings*, where each word is represented by a *one-hot* vector

Output: Prediction for the next word

- A probability distribution over the vocabulary (set of all possible words)

Training an RNN Language Model



Advantage of RNNs over N-gram

Recall: role of language model is to predict the next word in a sequence, given preceding words (i.e. assign a probability to every word in the vocabulary, using preceding words)

N-gram: compute probability of a word given occurrence information with

n−1 prior words

- Context has size n-1

RNN: predict the next word using the previous word + hidden state

- Theoretically unlimited context, since hidden state contains information about *all* previous words in the sequence

Limitations of RNNs

In training, backpropagation is used to minimize error

- Gradients are calculated layer by layer, and get smaller as they propagate through each layer (**vanishing gradient**) → updates to the parameters are small → network learns very slowly

- RNNs tend to be unable to make use of information distant from the current point of processing

LSTM

LSTM = Long Short-Term Memory

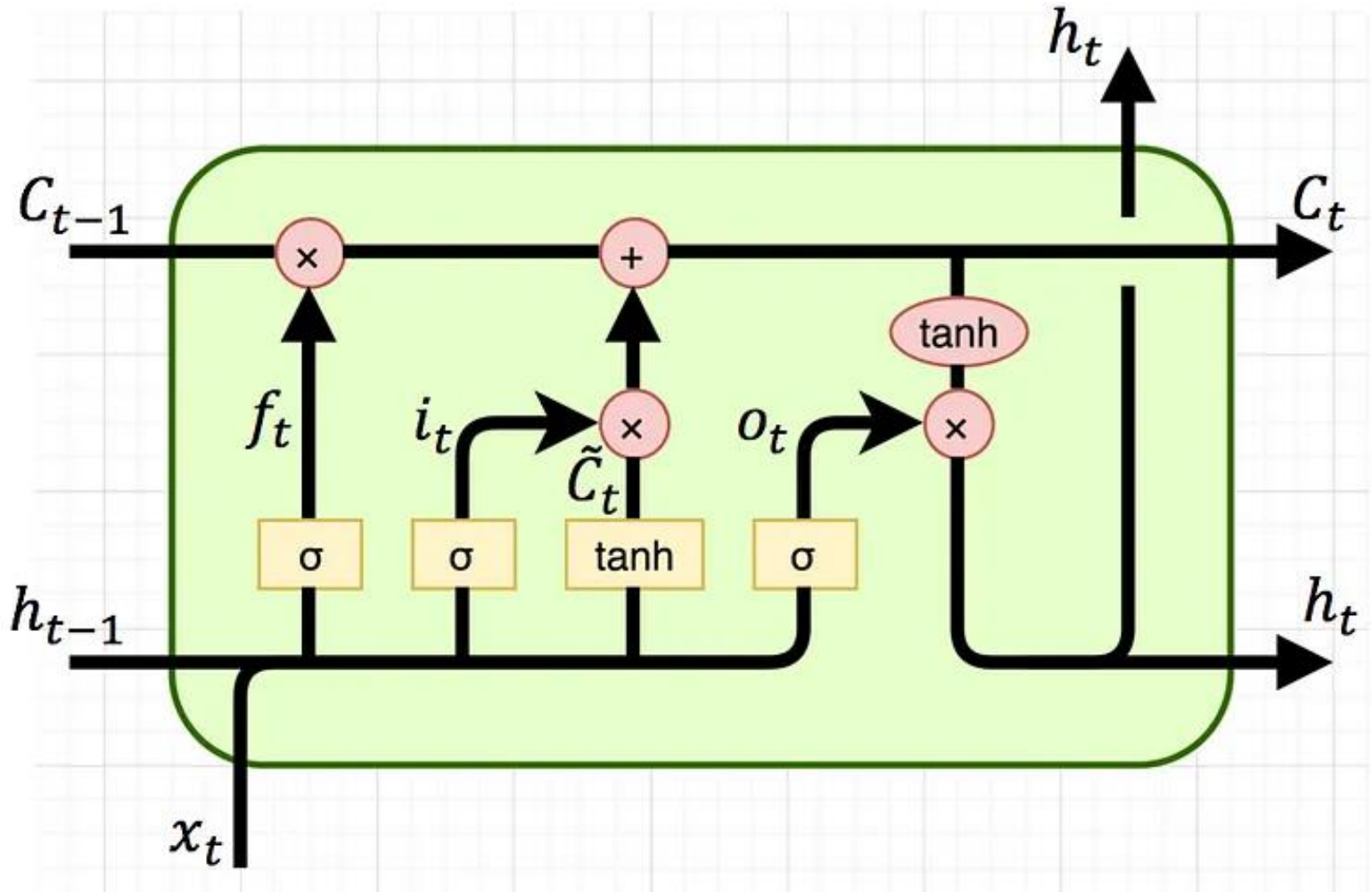
NOT

"long-term memory" + "short-term memory" (like in human beings)

but

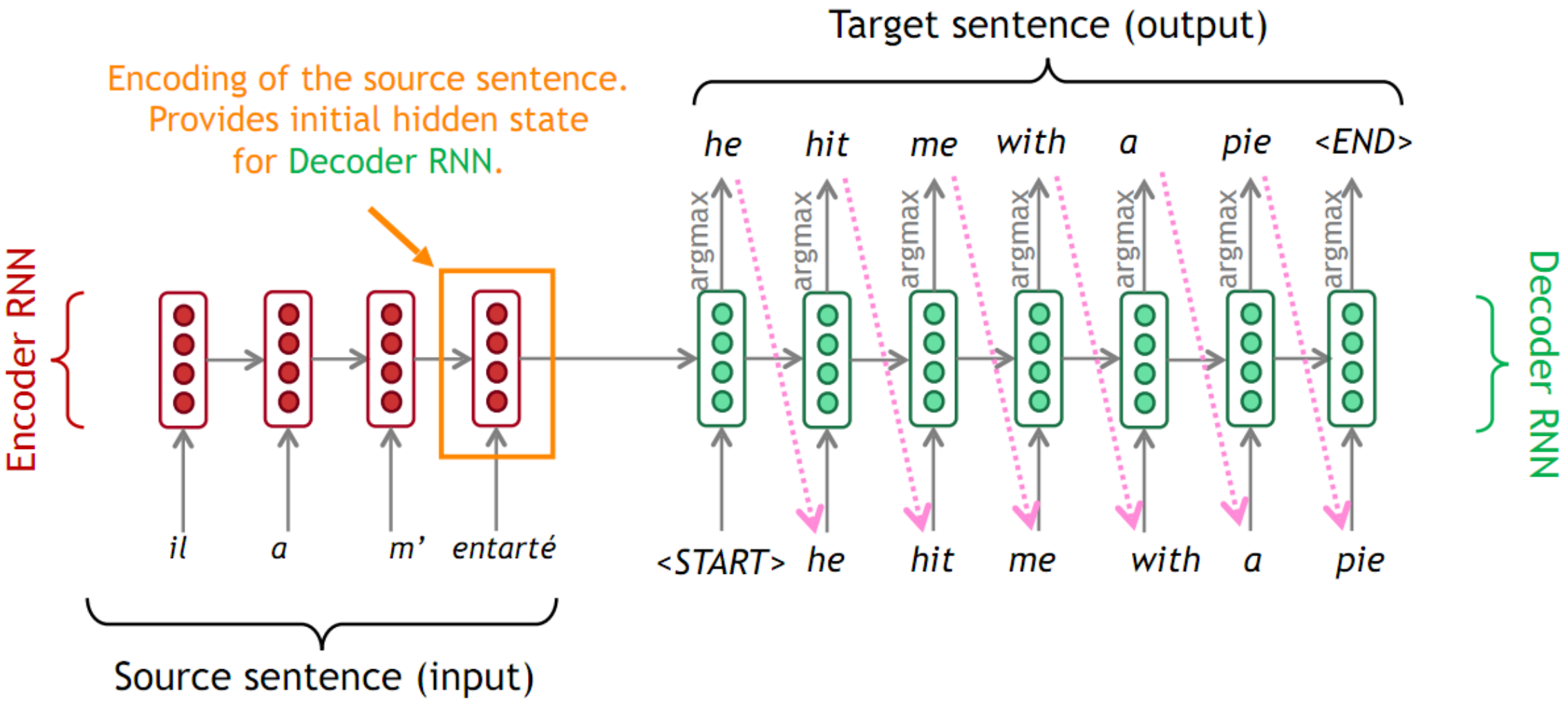
a *long STM* i.e. an STM can last thousands of timesteps

LSTM

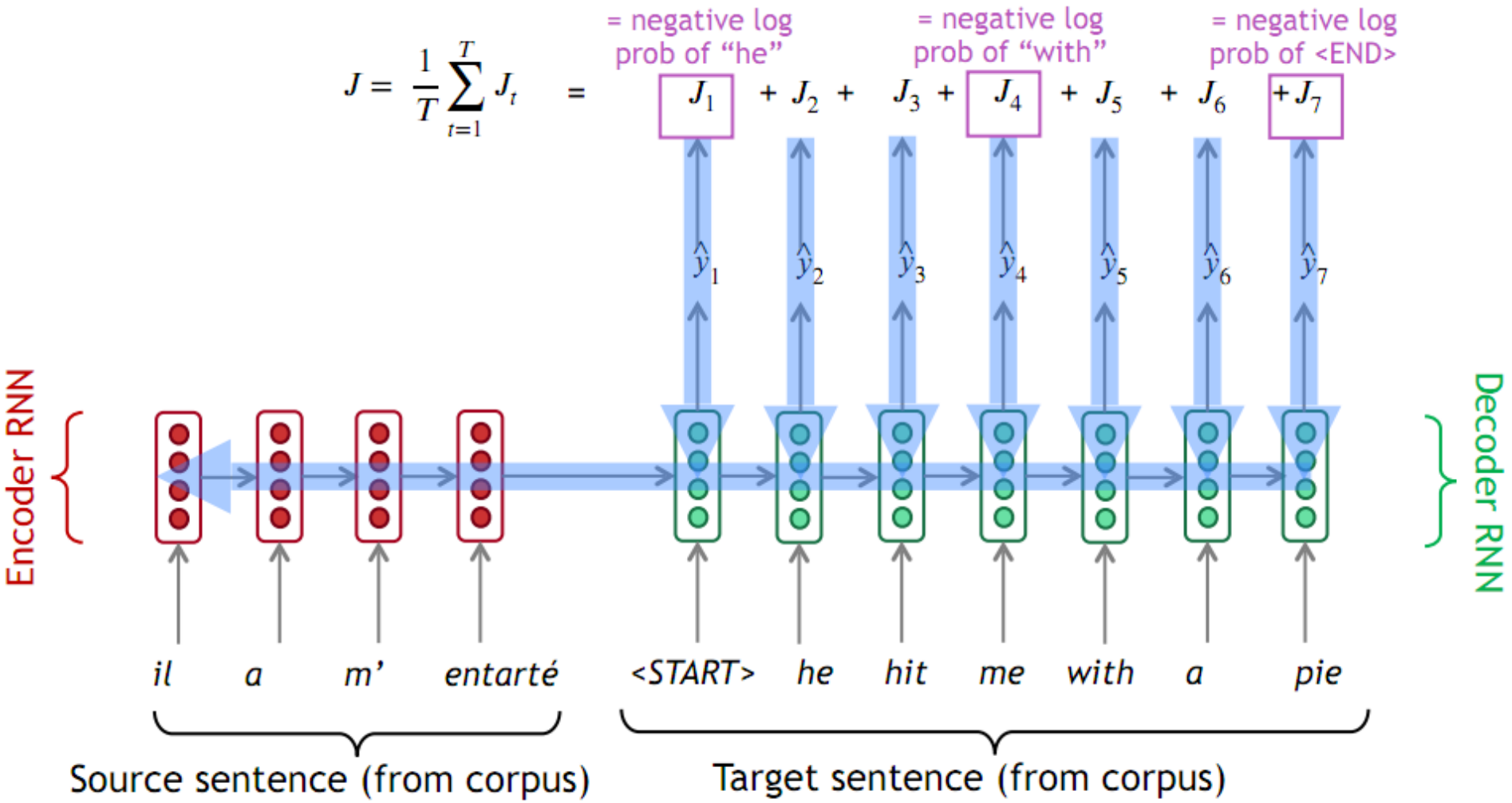


https://medium.com/@saurabh.rathor092/simple-rnn-vs-gru-vs-lstm-difference-lies-in-more-flexible-control-5f33e07b1e57

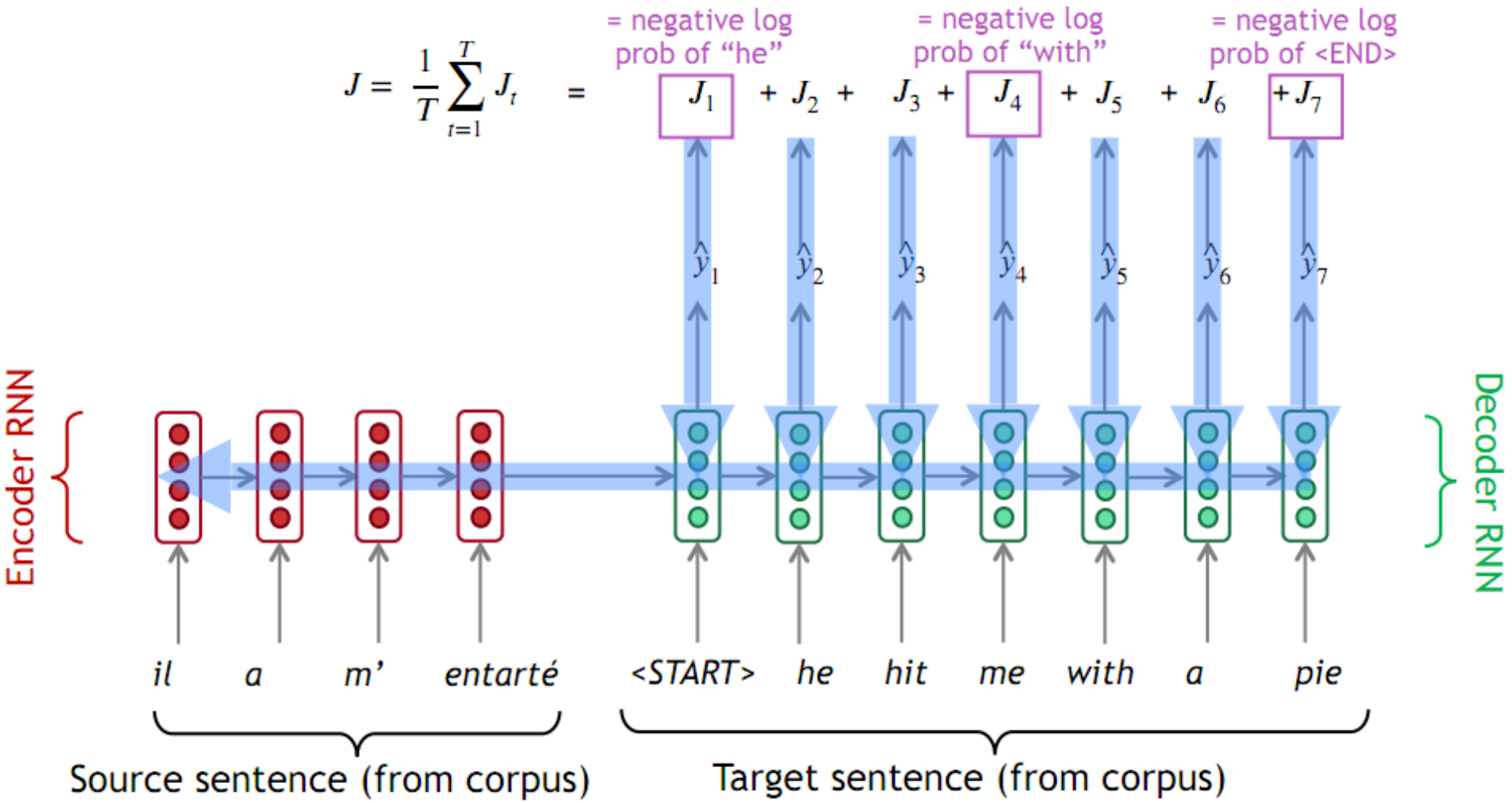
seq2seq


Target sentence (output)
Encoding of the source sentence.
Provides initial hidden state
for Decoder RNN.
Encoder RNN
Decoder RNN
argmax
he
hit
me
with
a
pie
<END>
il
a
m'
entarté
<START>
he
hit
me
with
a
pie
Source sentence (input)

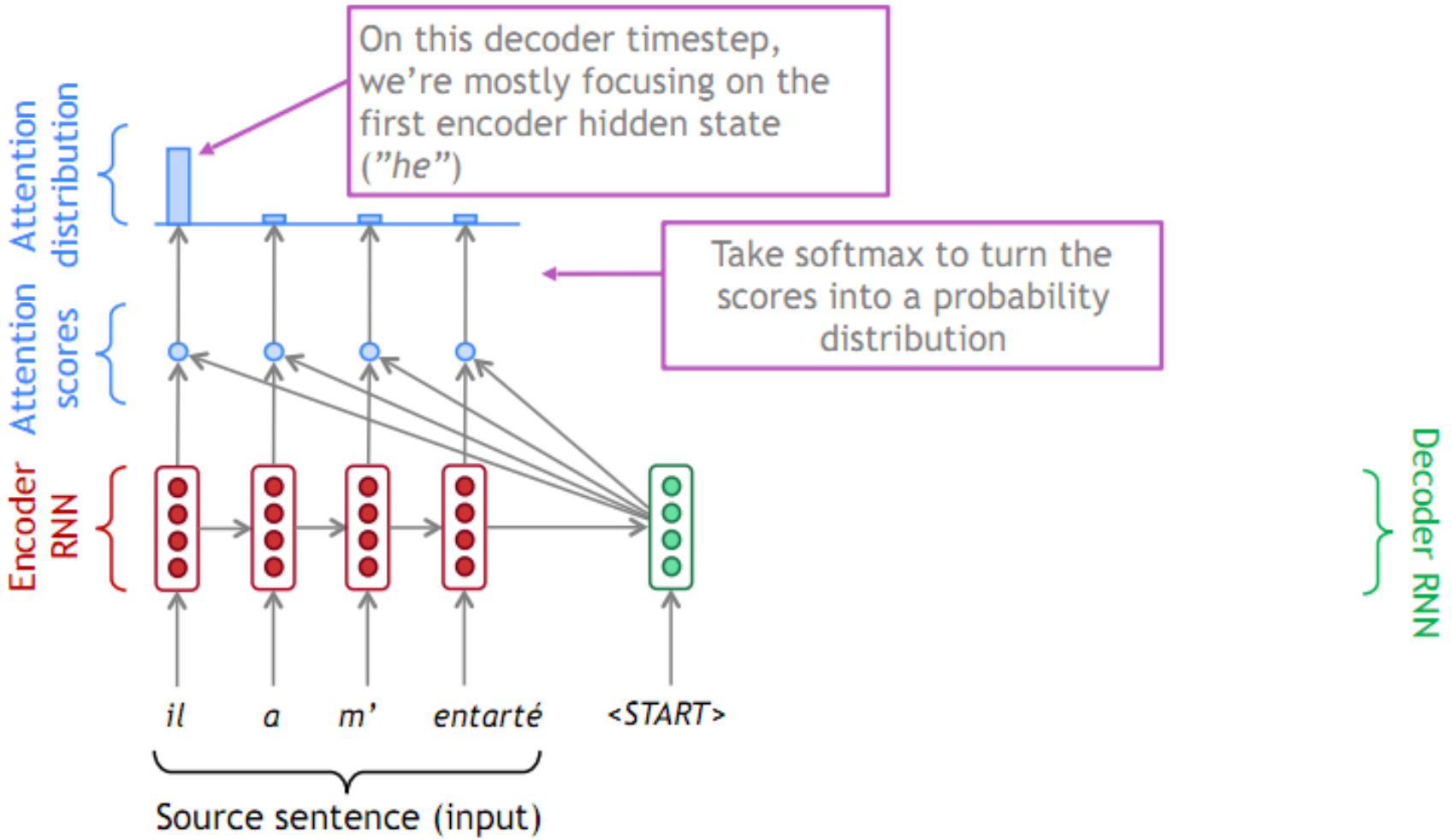
seq2seq


$J = \frac{1}{T}\sum_{t=1}^{T} J_t$ =
= negative log prob of "he"
= negative log prob of "with"
= negative log prob of <END>
J_1 + J_2 + J_3 + J_4 + J_5 + J_6 + J_7
$\hat{y}_1$ $\hat{y}_2$ $\hat{y}_3$ $\hat{y}_4$ $\hat{y}_5$ $\hat{y}_6$ $\hat{y}_7$
Encoder RNN
Decoder RNN
il a m' entarté
<START> he hit me with a pie
Source sentence (from corpus)
Target sentence (from corpus)

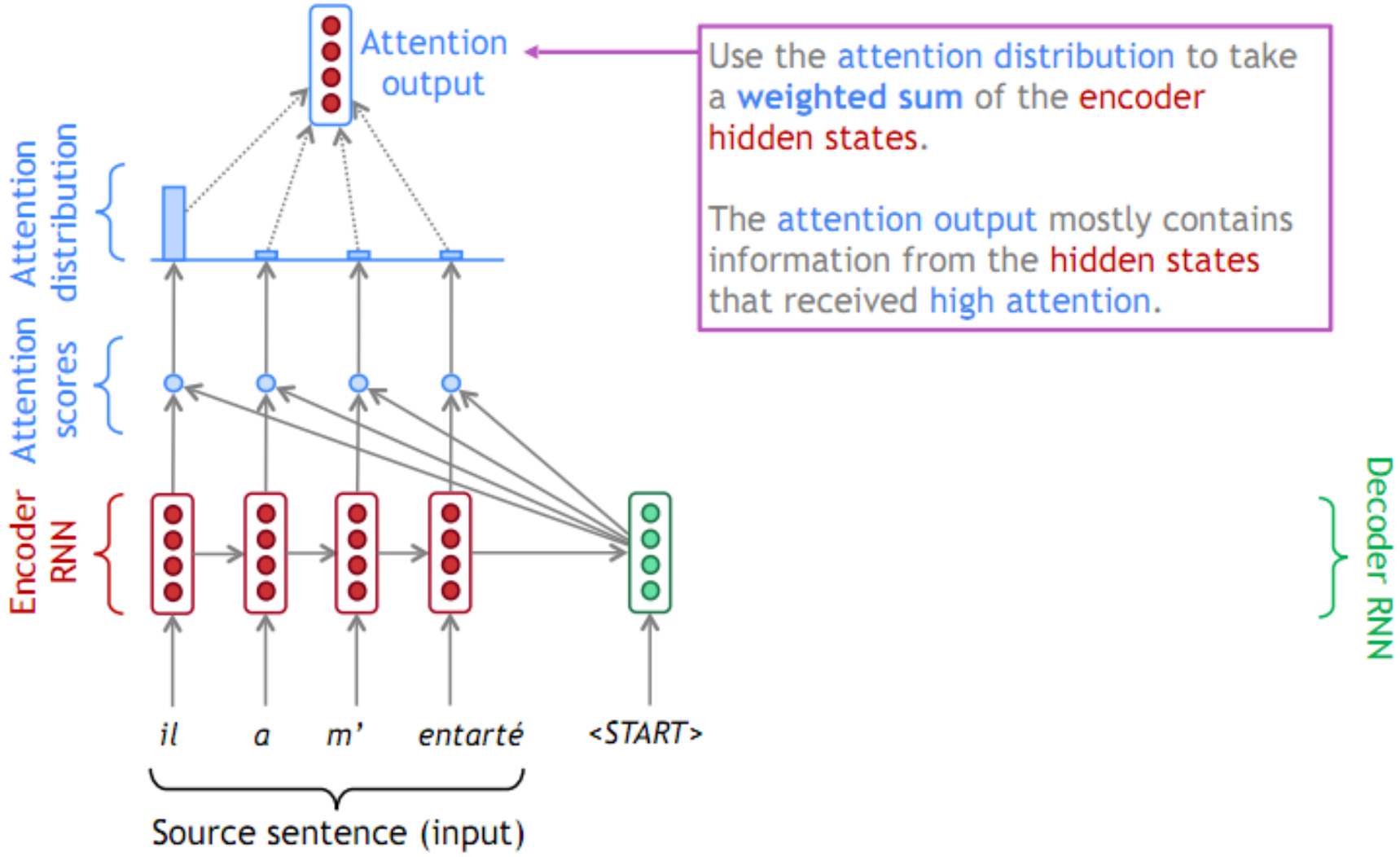
seq2seq


$J = \frac{1}{T}\sum_{t=1}^{T} J_t$ =
= negative log prob of "he"
= negative log prob of "with"
= negative log prob of <END>
J_1 + J_2 + J_3 + J_4 + J_5 + J_6 + J_7
$\hat{y}_1$ $\hat{y}_2$ $\hat{y}_3$ $\hat{y}_4$ $\hat{y}_5$ $\hat{y}_6$ $\hat{y}_7$
Encoder RNN
Decoder RNN
il a m' entarté
<START> he hit me with a pie
Source sentence (from corpus)
Target sentence (from corpus)

Attention seq2seq



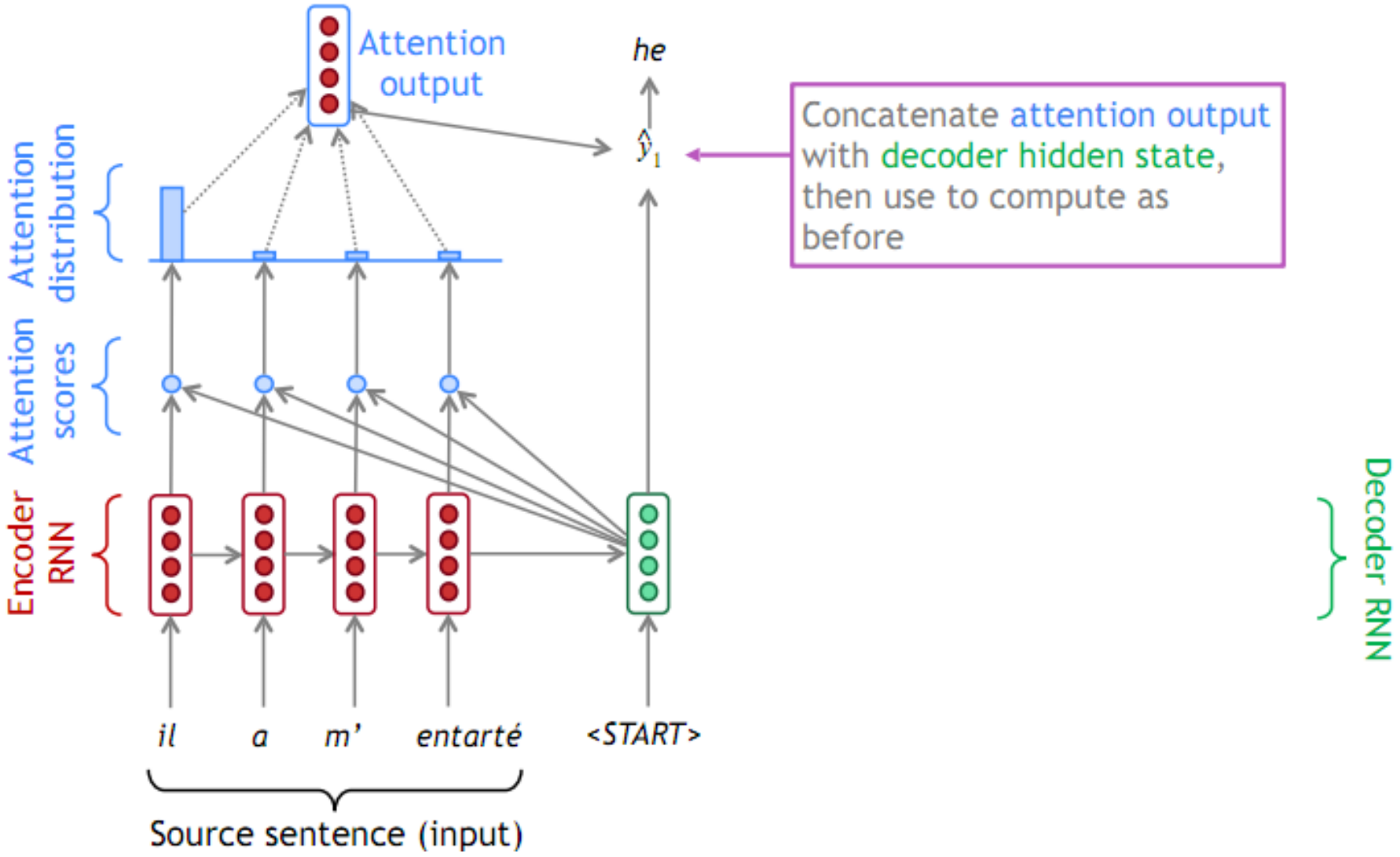
Attention seq2seq



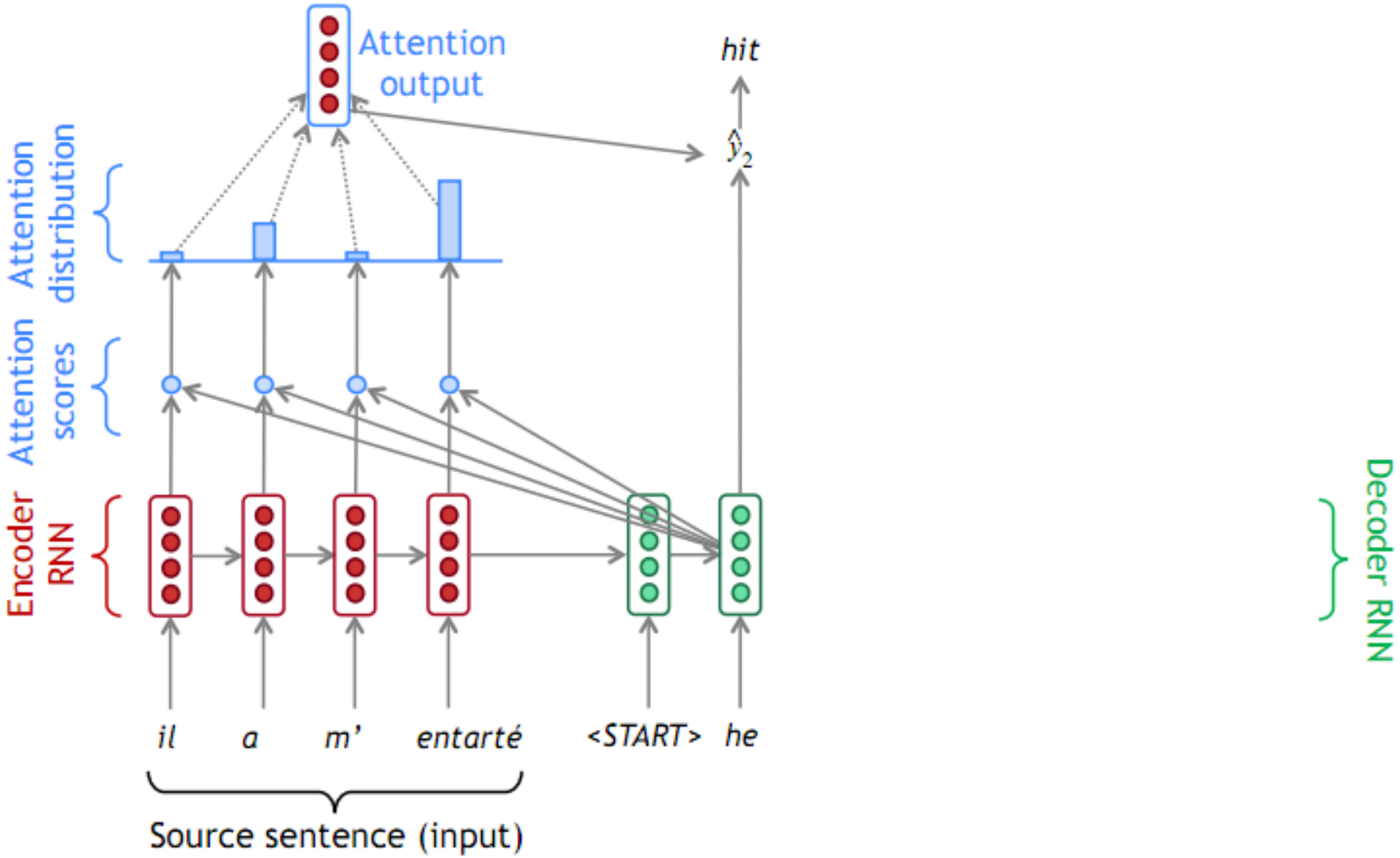
Use the attention distribution to take a **weighted sum** of the encoder hidden states.

The attention output mostly contains information from the hidden states that received high attention.

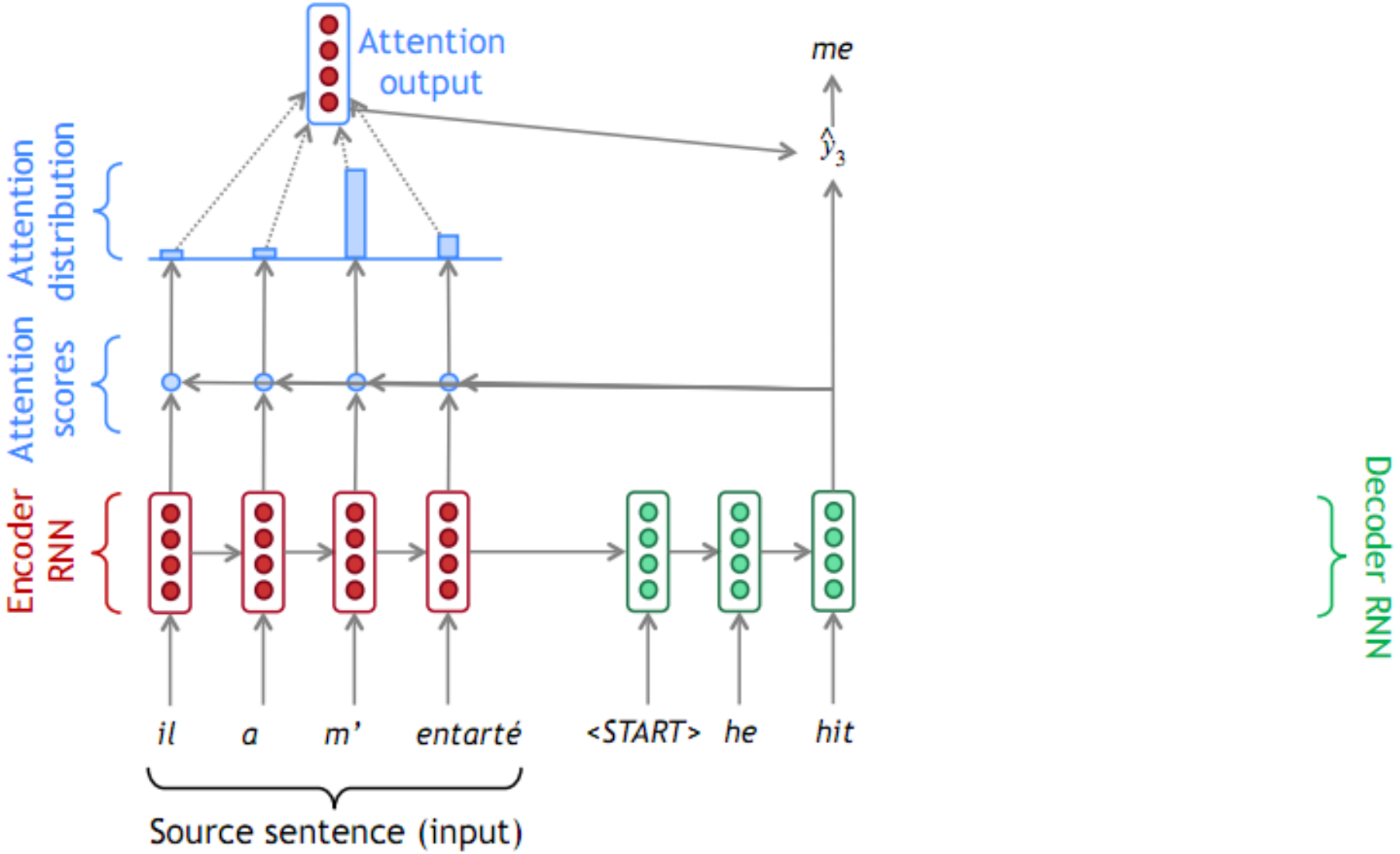
Attention seq2seq



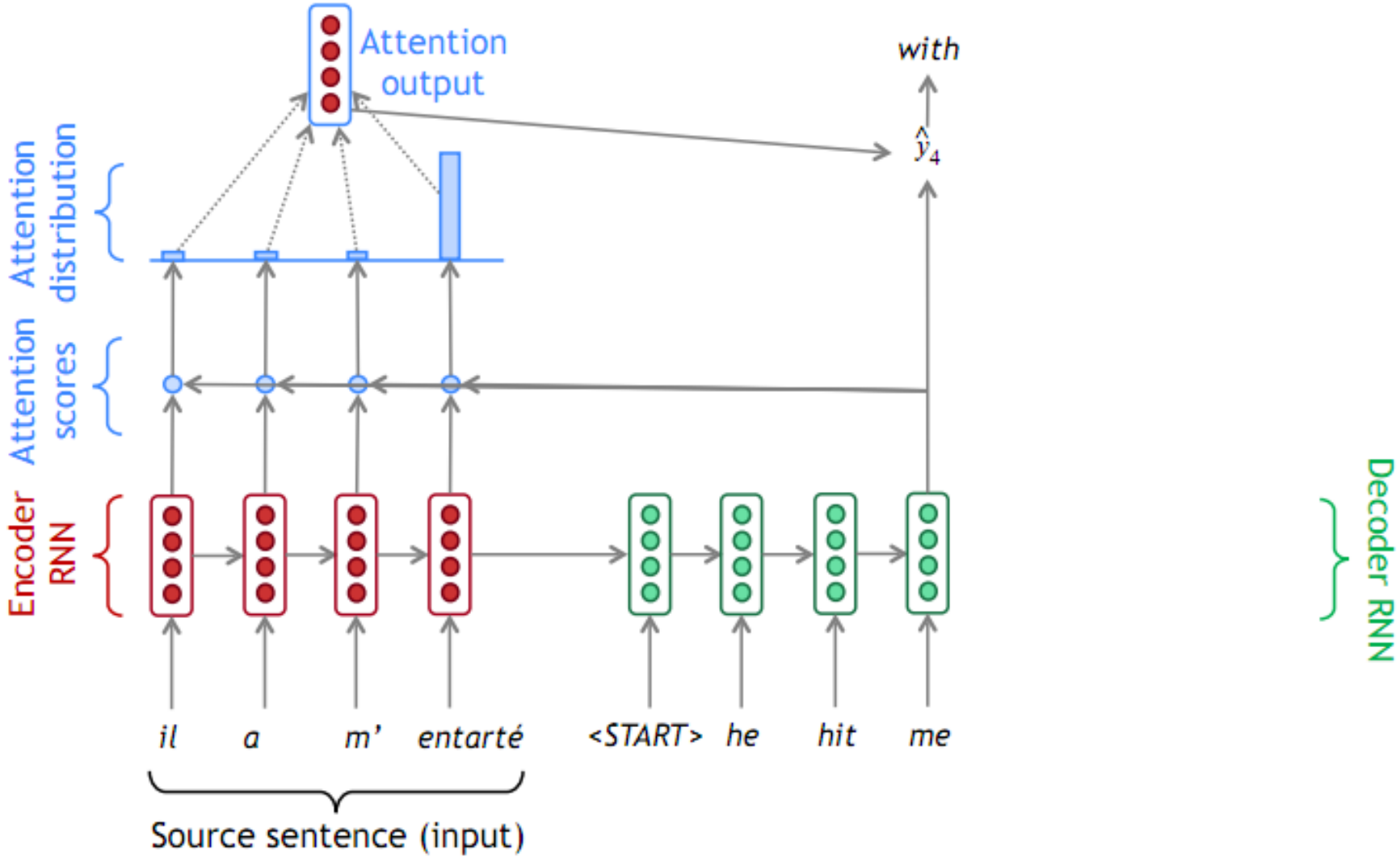
Attention seq2seq



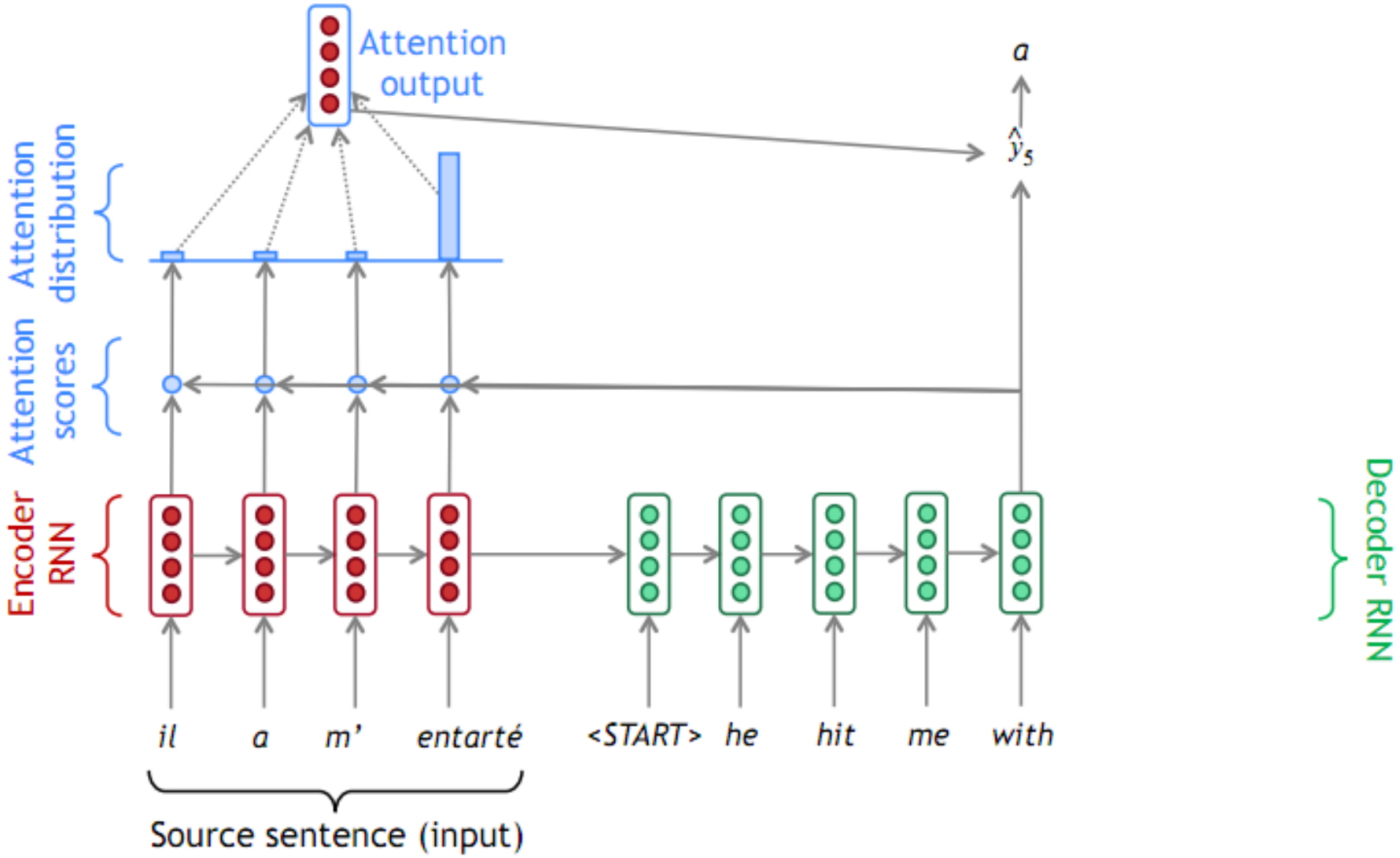
Attention seq2seq



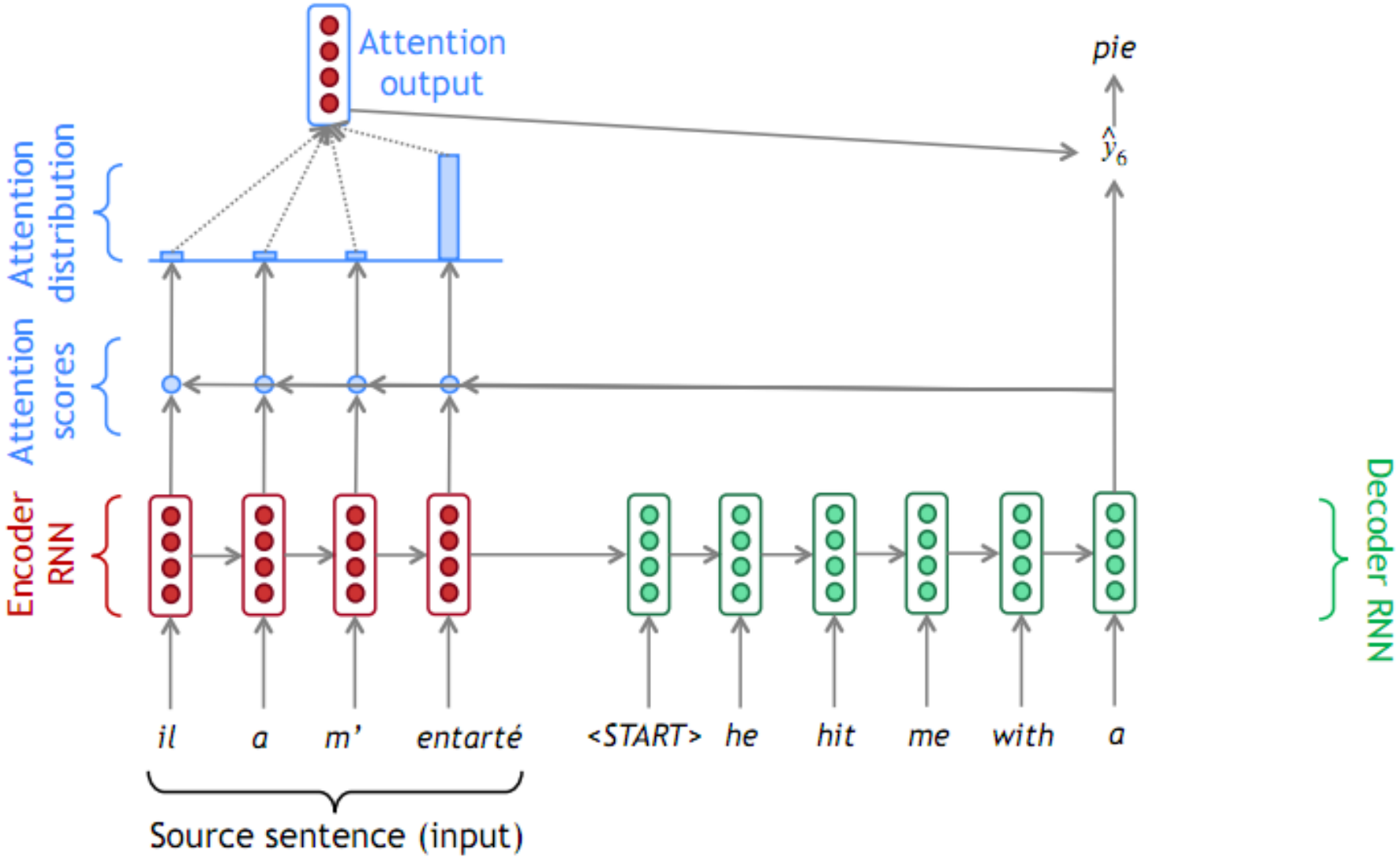
Attention seq2seq



Attention seq2seq



Attention seq2seq



Details

- We have encoder hidden states $h_1, \ldots, h_N \in \mathbb{R}^h$
- On timestep t, we have decoder hidden state $s_t \in \mathbb{R}^h$
- We get the attention scores e^t for this step:

$$\boldsymbol{e}^t = [\boldsymbol{s}_t^T \boldsymbol{h}_1, \ldots, \boldsymbol{s}_t^T \boldsymbol{h}_N] \in \mathbb{R}^N$$

- We take softmax to get the attention distribution α^t for this step (this is a probability distribution and sums to 1)

$$\alpha^t = \text{softmax}(\boldsymbol{e}^t) \in \mathbb{R}^N$$

- We use α^t to take a weighted sum of the encoder hidden states to get the attention output

$$\boldsymbol{a}_t = \sum_{i=1}^{N} \alpha_i^t \boldsymbol{h}_i \in \mathbb{R}^h$$

- Finally we concatenate the attention output $\boldsymbol{a}_t$ with the decoder hidden state s_t and proceed as in the non-attention seq2seq model

$$[\boldsymbol{a}_t; \boldsymbol{s}_t] \in \mathbb{R}^{2h}$$

HUST

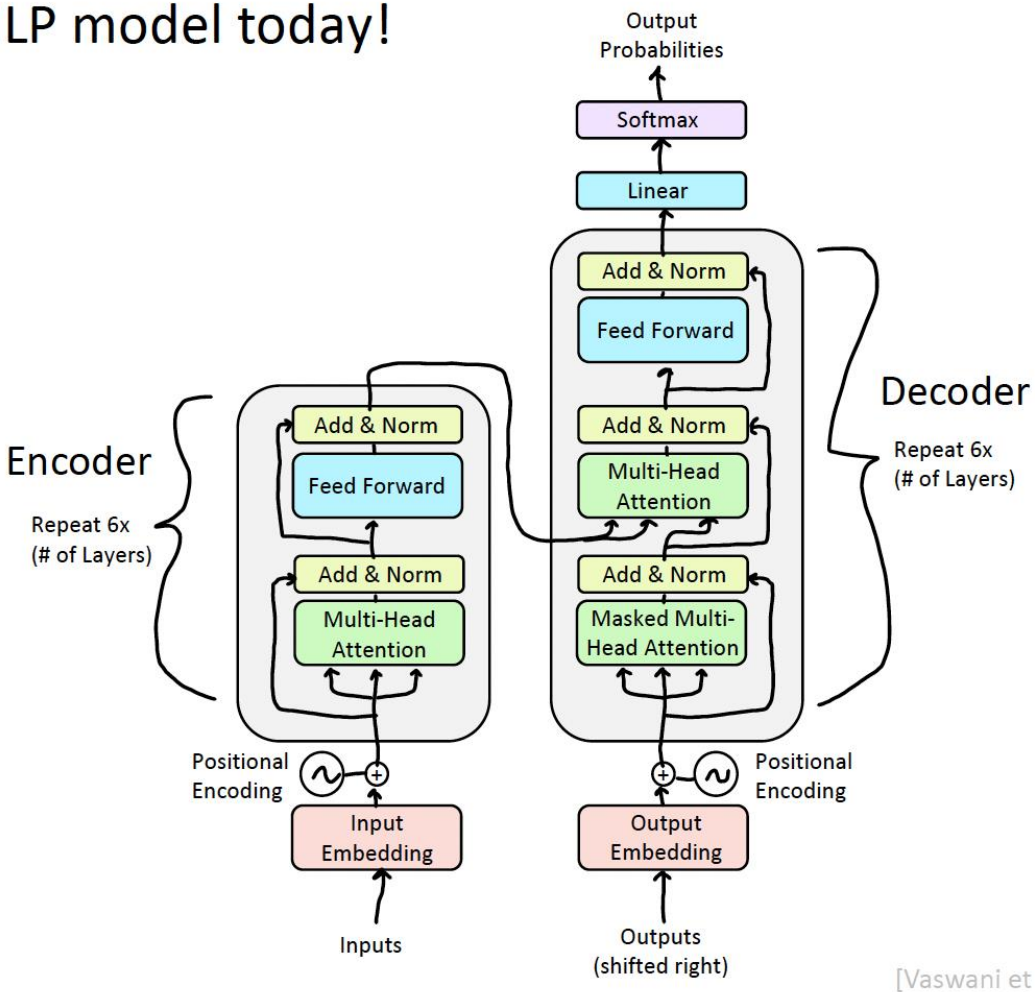
hust.edu.vn fb.com/dhbkhn

Transformer

Transformers Have Revolutionized the Field ofNLP

- By the end of this lecture, you will deeply understand the neural architecture that underpins virtually every state-of-the-art NLP model today!

Courtesy of Paramount Pictures



[Vaswani et al., 2017]

Great Results with Transformers: SuperGLUE

SuperGLUE is a suite of challenging NLP tasks, including question-answering, word sense disambiguation, coreference resolution, and natural language inference.

Rank	Name	Model	URL	Score	BoolQ	CB	COPA	MultiRC	ReCoRD	RTE	WiC	WSC	AX-b	AX-g
1	JDExplore d-team	Vega v2		91.3	90.5	98.6/99.2	99.4	88.2/62.4	94.4/93.9	96.0	77.4	98.6	-0.4	100.0/50.0
2	Liam Fedus	ST-MoE-32B		91.2	92.4	96.9/98.0	99.2	89.6/65.8	95.1/94.4	93.5	77.7	96.6	72.3	96.1/94.1
3	Microsoft Alexander v-team	Turing NLR v5		90.9	92.0	95.9/97.6	98.2	88.4/63.0	96.4/95.9	94.1	77.1	97.3	67.8	93.3/95.5
4	ERNIE Team - Baidu	ERNIE 3.0		90.6	91.0	98.6/99.2	97.4	88.6/63.2	94.7/94.2	92.6	77.4	97.3	68.6	92.7/94.7
5	Yi Tay	PaLM 540B		90.4	91.9	94.4/96.0	99.0	88.7/63.6	94.2/93.3	94.1	77.4	95.9	72.9	95.5/90.4
6	Zirui Wang	T5 + UDG, Single Model (Google Brain)		90.4	91.4	95.8/97.6	98.0	88.3/63.0	94.2/93.5	93.0	77.9	96.6	69.1	92.7/91.9
7	DeBERTa Team - Microsoft	DeBERTa / TuringNLRv4		90.3	90.4	95.7/97.6	98.4	88.2/63.7	94.5/94.1	93.2	77.5	95.9	66.7	93.3/93.8
8	SuperGLUE Human Baselines	SuperGLUE Human Baselines		89.8	89.0	95.8/98.9	100.0	81.8/51.9	91.7/91.3	93.6	80.0	100.0	76.6	99.3/99.7
9	T5 Team - Google	T5		89.3	91.2	93.9/96.8	94.8	88.1/63.3	94.1/93.4	92.5	76.9	93.8	65.6	92.7/91.9
10	SPoT Team - Google	Frozen T5 1.1 + SPoT		89.2	91.1	95.8/97.6	95.6	87.9/61.9	93.3/92.4	92.9	75.8	93.8	66.9	83.1/82.6

Great Results with Transformers: Rise of Large Language Models!

Today, Transformer-based models dominate LMSYS Chatbot Arena Leaderboard!

Rank	Model	Arena Elo	95% CI	Votes	Organization	License	Knowledge Cutoff
1	GPT-4-Turbo-2024-04-09	1258	+4/-4	26444	OpenAI	Proprietary	2023/12
1	GPT-4-1106-preview	1253	+3/-3	68353	OpenAI	Proprietary	2023/4
1	Claude 3 Opus	1251	+3/-3	71500	Anthropic	Proprietary	2023/8
2	Gemini 1.5 Pro API-0409-Preview	1249	+4/-5	22211	Google	Proprietary	2023/11
3	GPT-4-0125-preview	1248	+2/-3	58959	OpenAI	Proprietary	2023/12
6	Meta Llama 3 70b Instruct	1213	+4/-6	15809	Meta	Llama 3 Community	2023/12
6	Bard (Gemini Pro)	1208	+7/-6	12435	Google	Proprietary	Online
7	Claude 3 Sonnet	1201	+4/-2	73414	Anthropic	Proprietary	2023/8

Gemini / Bard (Google)

ChatGPT / GPT-4 (OpenAI)

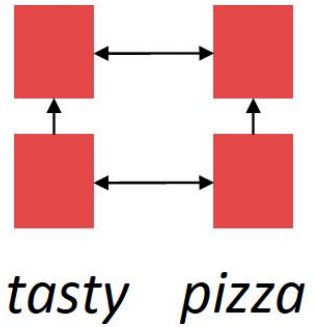
Claude 3 (Anthropic)

Llama 3 (Meta)

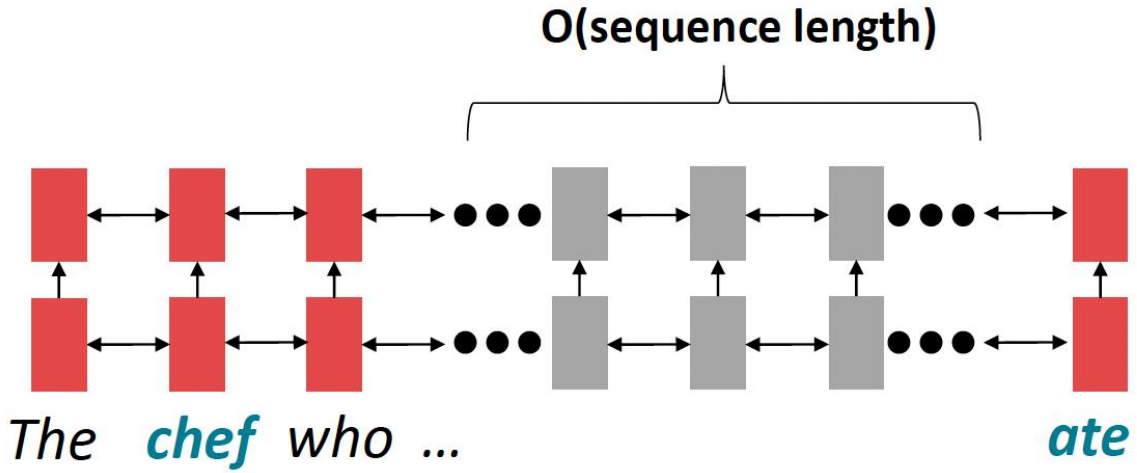
[Chiang et al., 2024]

Transformer Motivation: Minimize Linear Interaction Distance

- RNNs are unrolled "left-to-right".
- It encodes linear locality: a useful heuristic!
  - Nearby words often affect each other's meanings

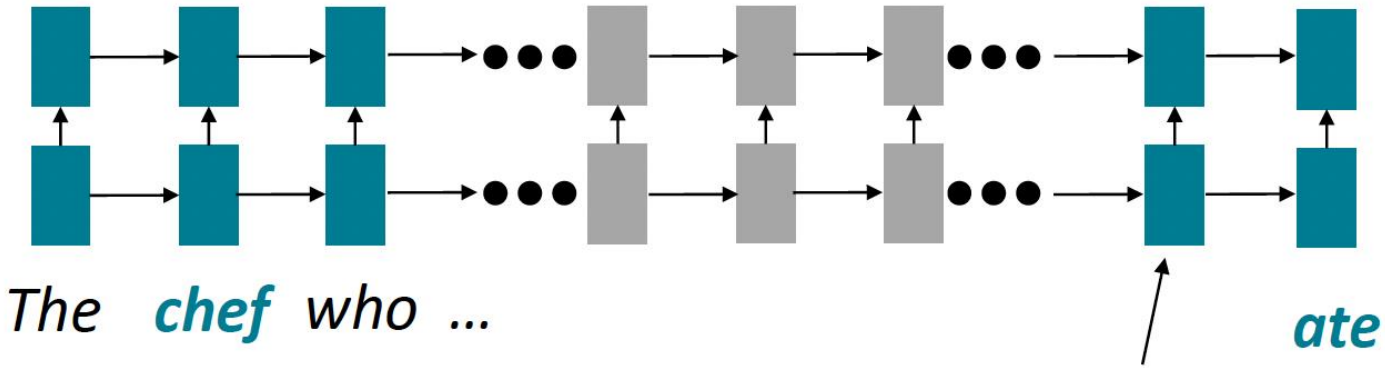


- **Problem:** RNNs take **O(sequence length)** steps for distant word pairs to interact.



Transformer Motivation: Minimize Linear Interaction Distance

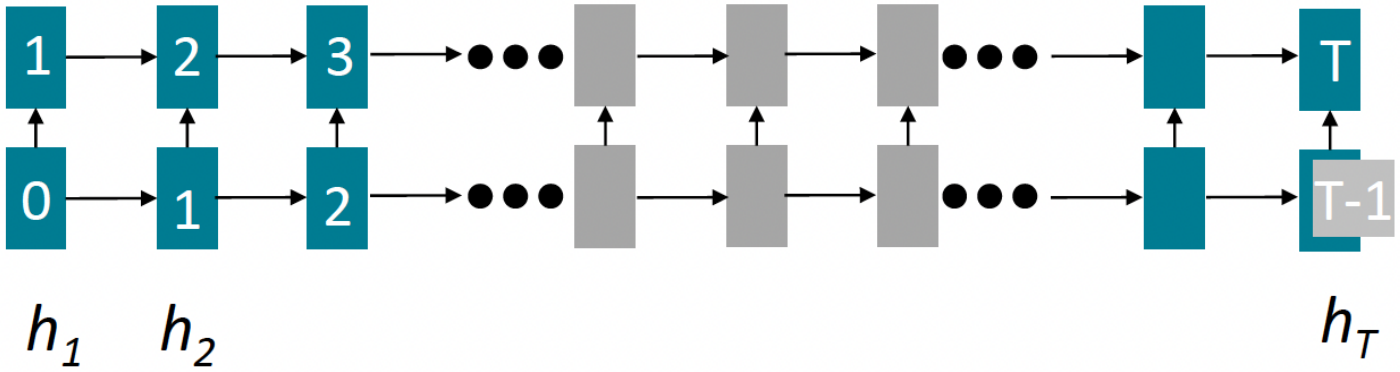
- **O(sequence length)** steps for distant word pairs to interact means:
  - Hard to learn long-distance dependencies (because gradient problems!)
  - Linear order of words is “baked in”; we already know sequential structure doesn't tell the whole story...



Info of ***chef*** has gone through O(sequence length) many layers!

Transformer Motivation: Maximize Parallelizability

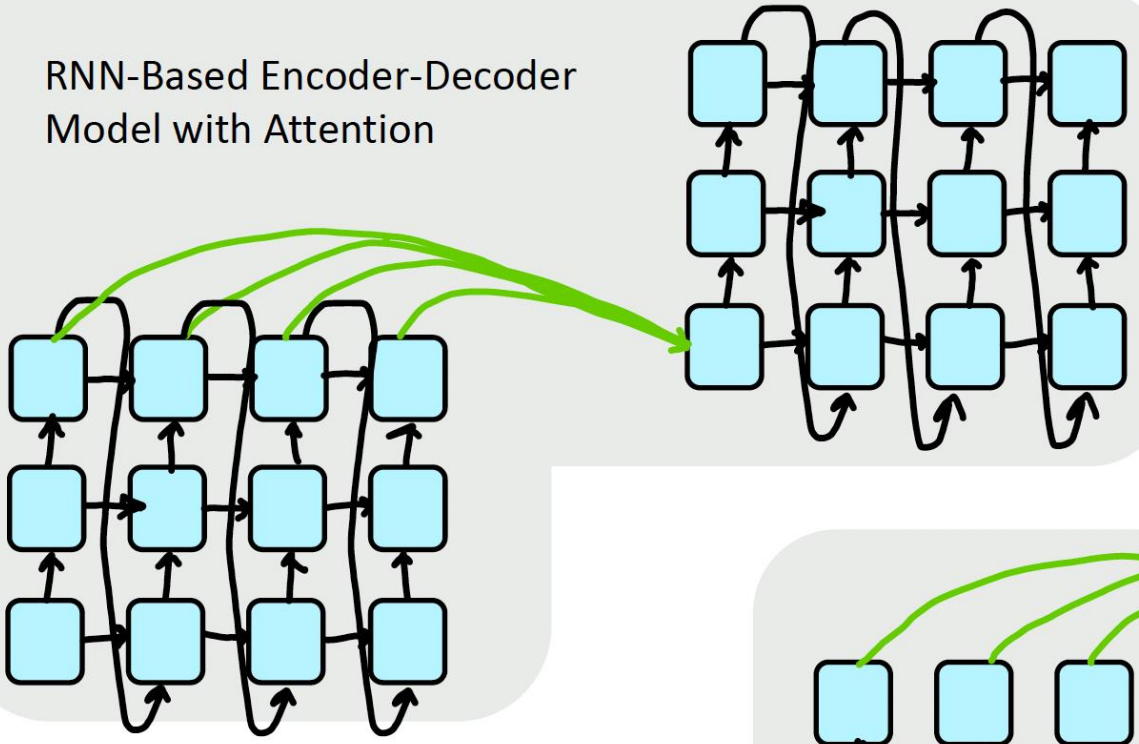
- Forward and backward passes have **O(seq length)** unparallelizable operations
  - GPUs (and TPUs) can perform many independent computations at once!
  - But future RNN hidden states can't be computed in full before past RNN hidden states have been computed
  - Inhibits training on very large datasets!
  - Particularly problematic as sequence length increases, as we can no longer batch many examples together due to memory limitations



Numbers indicate min # of steps before a state can be computed

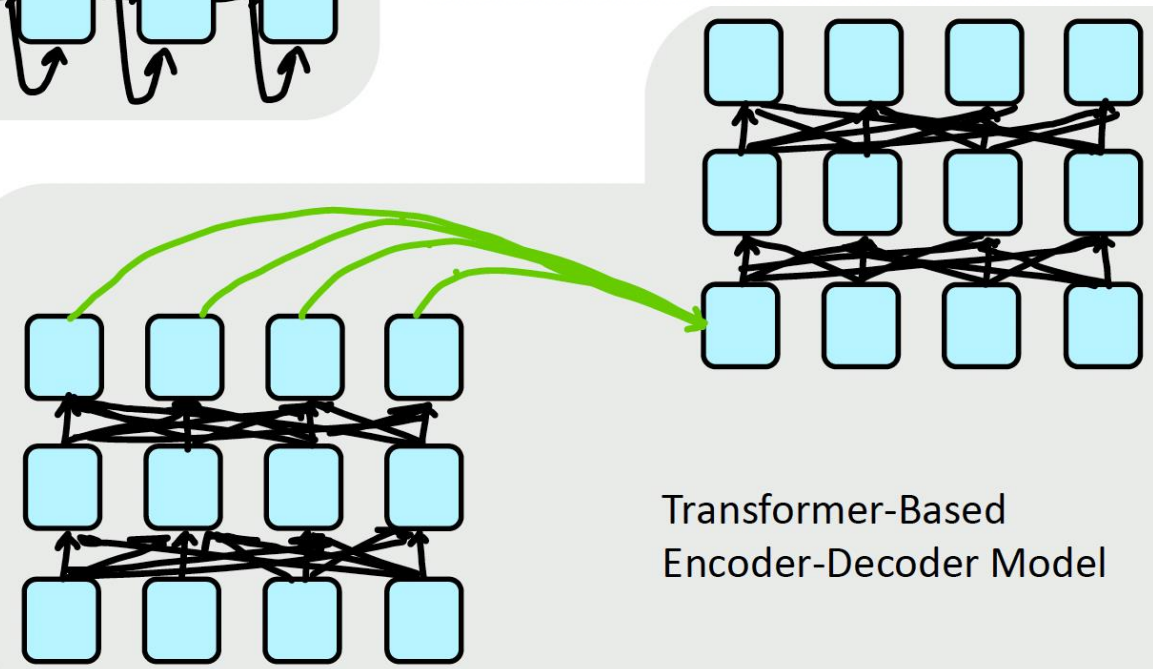
Transformer Motivation: Maximize Parallelizability

RNN-Based Encoder-Decoder Model with Attention



Transformer Advantages:

- Number of unparallelizable operations does not increase with sequence length.
- Each "word" interacts with each other, so maximum interaction distance is O(1).

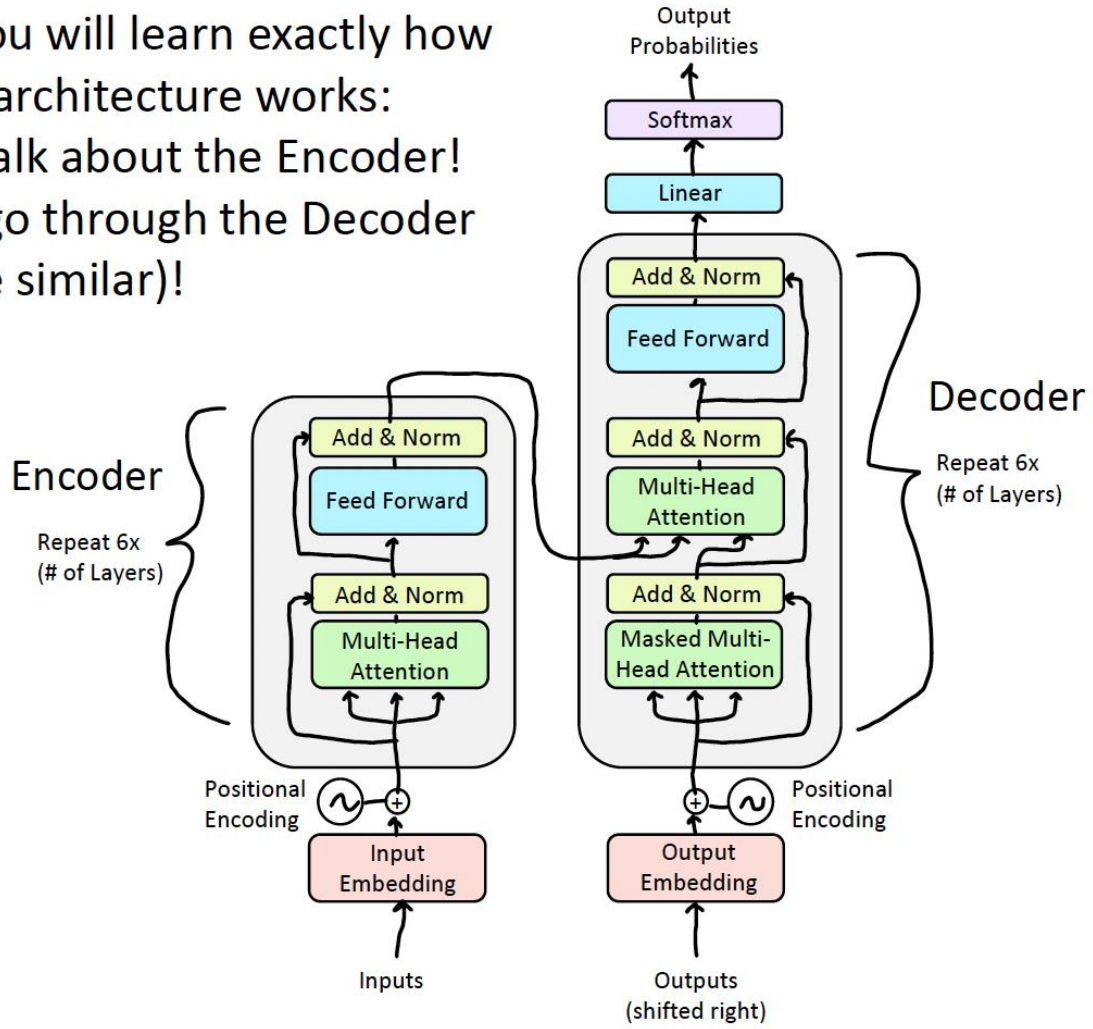


Transformer-Based Encoder-Decoder Model

The Transformer Encoder-Decoder [Vaswani et al., 2017]

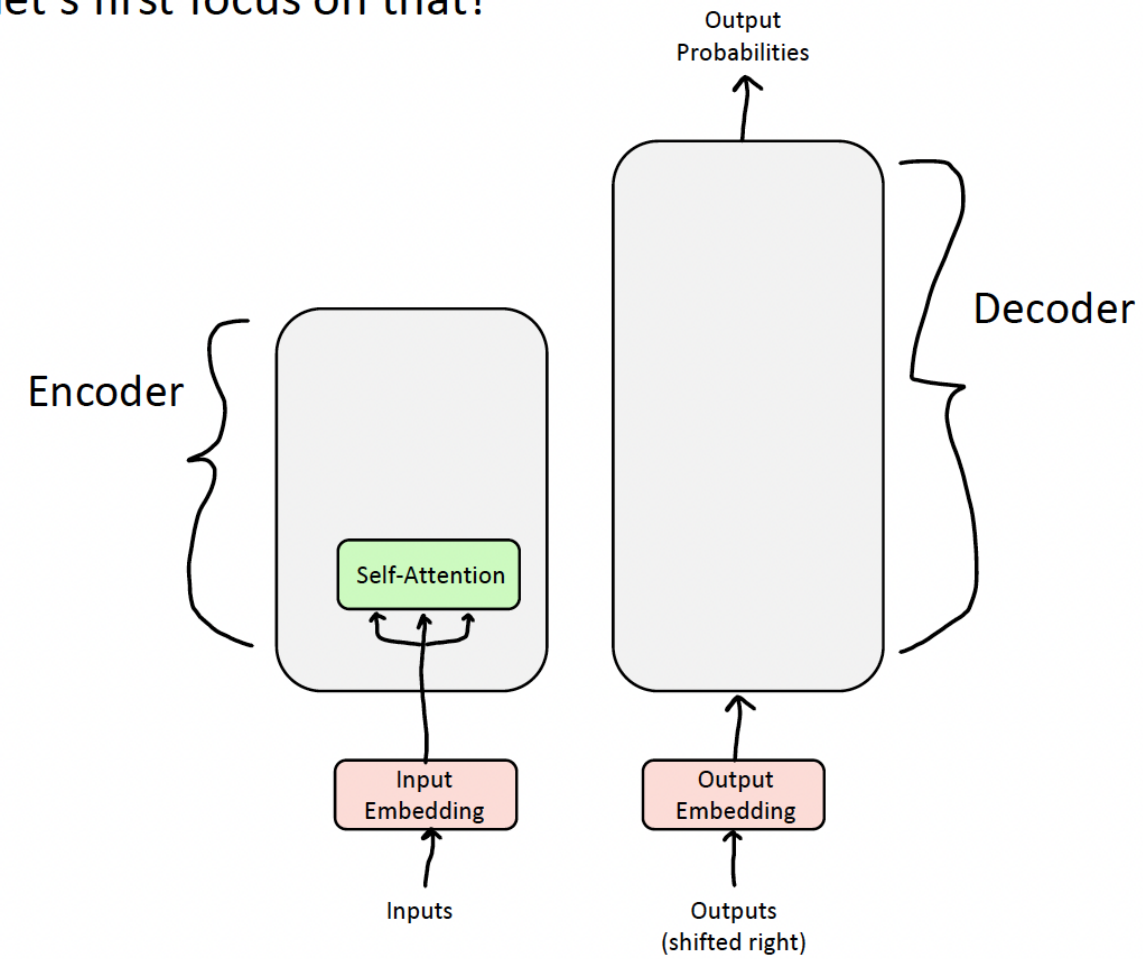
In this section, you will learn exactly how the Transformer architecture works:

- First, we will talk about the Encoder!
- Next, we will go through the Decoder (which is quite similar)!



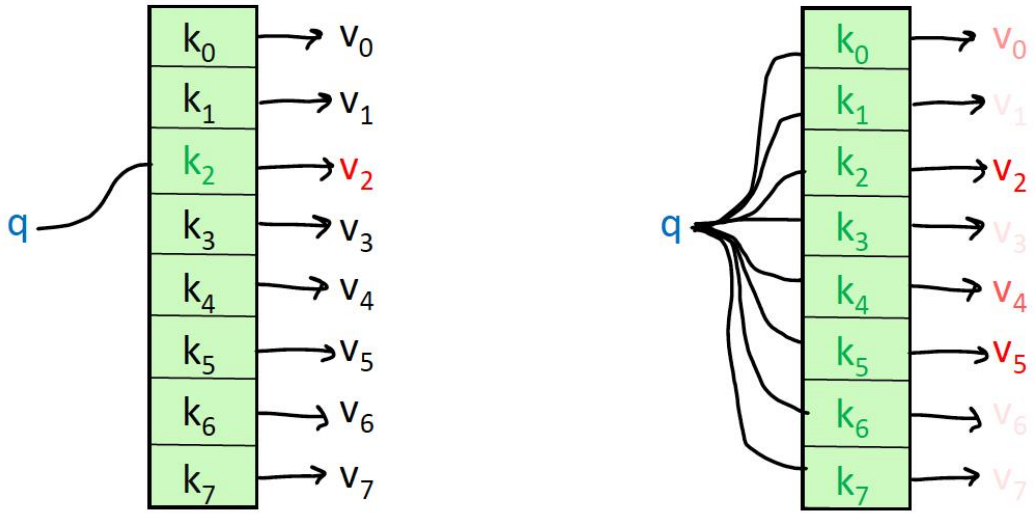
Encoder: Self-Attention

Self-Attention is the core building block of Transformer, so let's first focus on that!



Intuition for Attention Mechanism

- Let's think of attention as a "fuzzy" or approximate hashtable:
  - To look up a **value**, we compare a **query** against **keys** in a table.
  - In a hashtable (shown on the bottom left):
    - Each **query** (hash) maps to exactly one **key**-**value** pair.
  - In (self-)attention (shown on the bottom right):
    - Each **query** matches each **key** to varying degrees.
    - We return a sum of **values** weighted by the **query**-**key** match.



Recipe for Self-Attention in the Transformer Encoder

- Step 1: For each word x_i , calculate its **query**, **key**, and **value**.

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

- Step 2: Calculate attention score between **query** **and** **keys.**

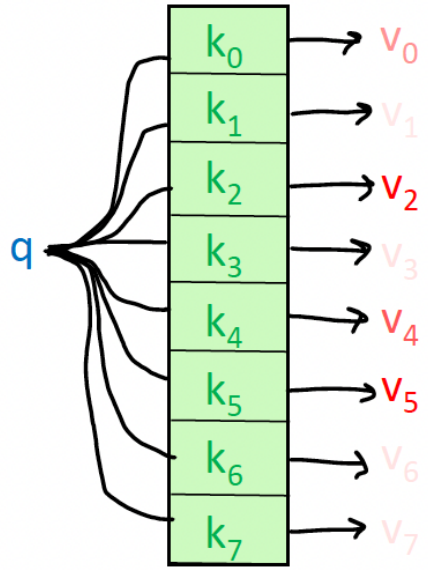
$$e_{ij} = q_i \cdot k_j$$

- Step 3: Take the softmax to normalize attention scores.

$$\alpha_{ij} = softmax(e_{ij}) = \frac{exp(e_{ij})}{\sum_k exp(e_{ik})}$$

- Step 4: Take a weighted sum of **values**.

$$Output_i = \sum_j \alpha_{ij} v_j$$



Recipe for (Vectorized) Self-Attention in the Transformer Encoder

- Step 1: With embeddings stacked in X, calculate **queries**, **keys**, and **values**.

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

- Step 2: Calculate attention scores between **query** **and** **keys.**

$$E = QK^T$$

- Step 3: Take the softmax to normalize attention scores.

$$A = softmax(E)$$

- Step 4: Take a weighted sum of **values**.

$$Output = AV$$

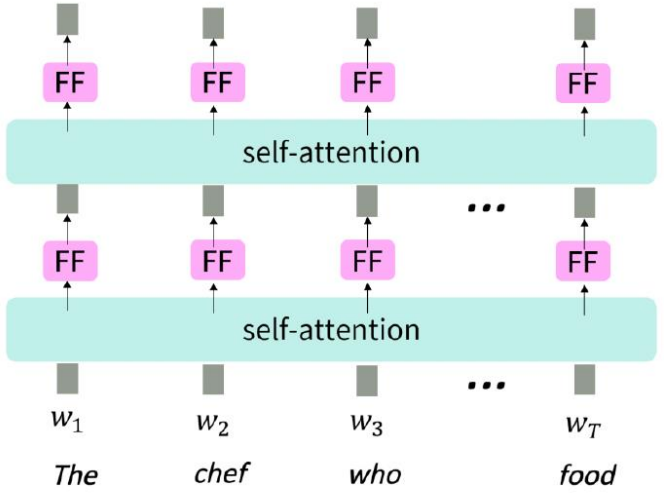
$$Output = softmax\left(QK^T\right)V$$

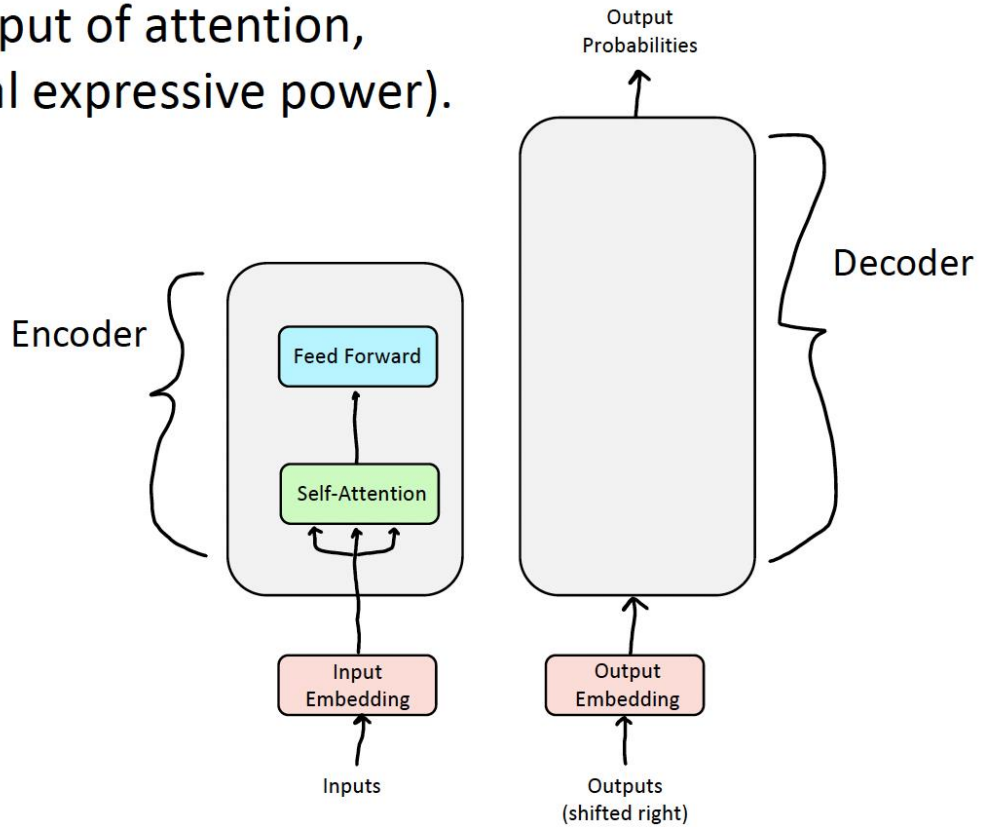
But attention isn't quite all you need!

- **Problem:** Since there are no element-wise non-linearities, self-attention is simply performing a re-averaging of the value vectors.
- **Easy fix:** Apply a feedforward layer to the output of attention, providing non-linear activation (and additional expressive power).

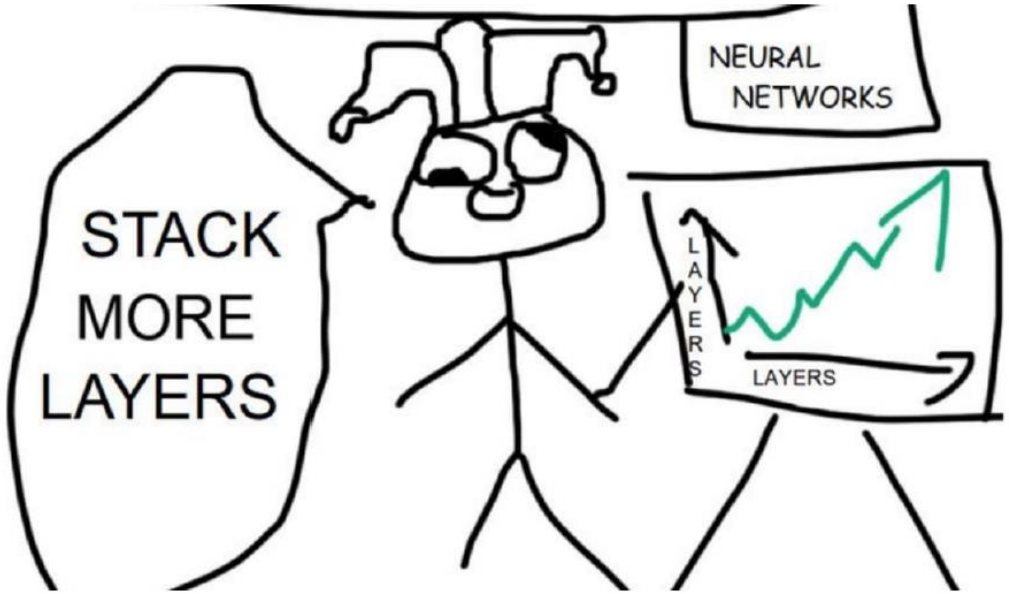
Equation for Feed Forward Layer

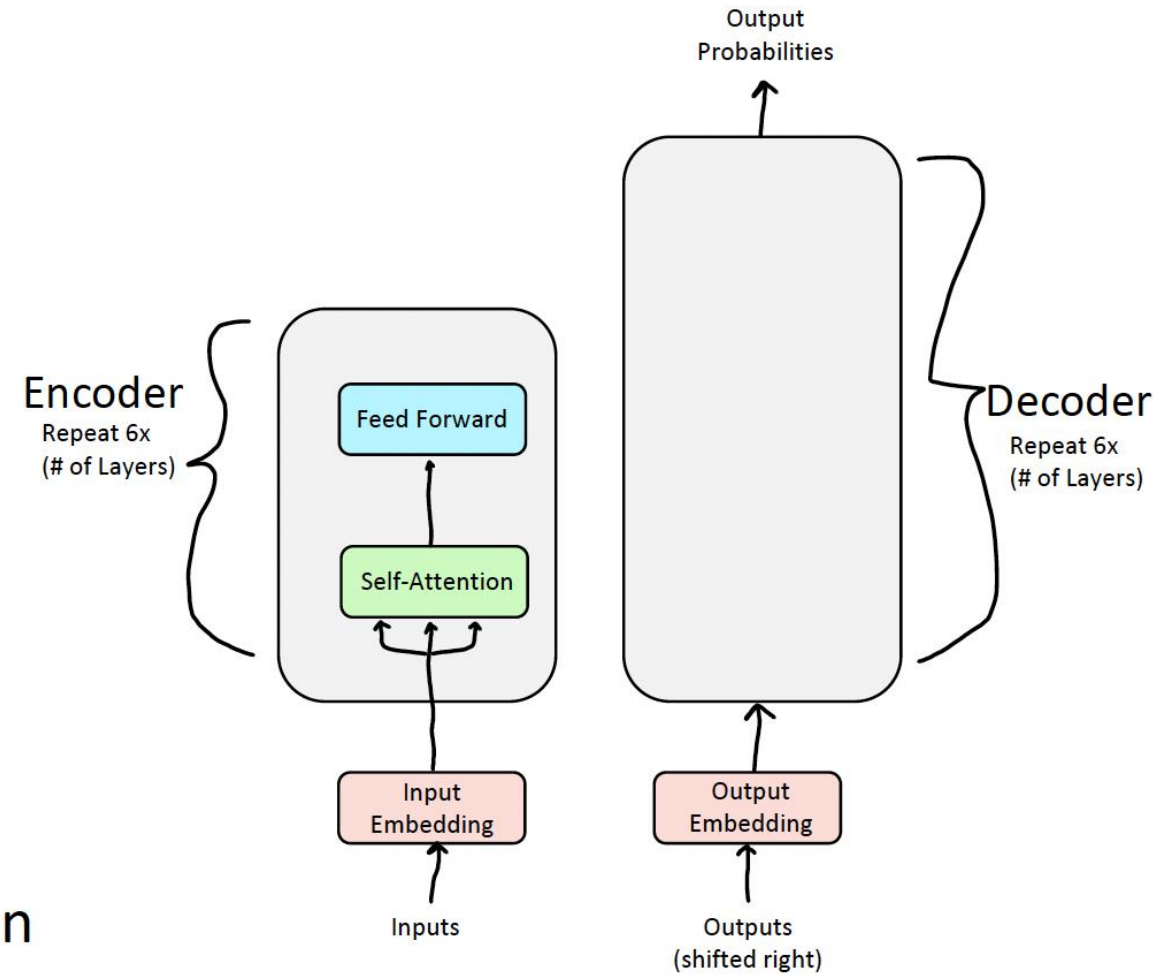
$$m_i = MLP(\text{output}_i)$$
$$= W_2 * \text{ReLU}(W_1 \times \text{output}_i + b_1) + b_2$$





But how do we make this work for deep networks?





Training Trick #1: Residual Connections

Training Trick #2: LayerNorm

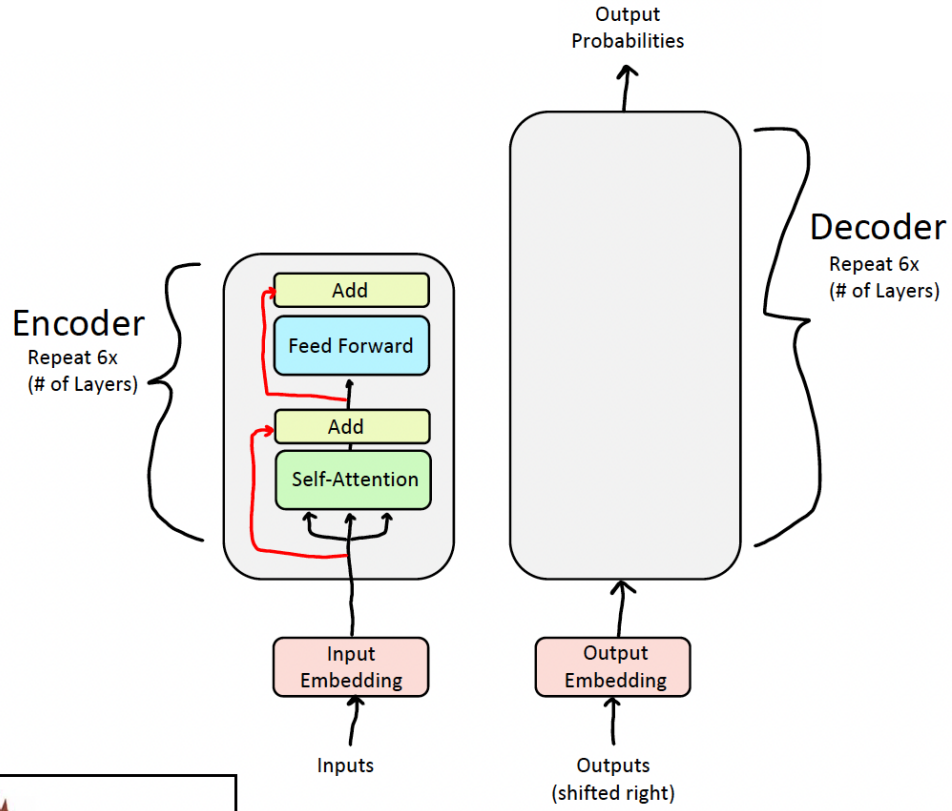
Training Trick #3: Scaled Dot Product Attention

But how do we make this work for deep networks?

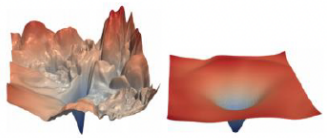
- Residual connections are a simple but powerful technique from computer vision.
- Deep networks are surprisingly bad at learning the identity function!
- Therefore, directly passing "raw" embeddings to the next layer can actually be very helpful!

$$x_\ell = F(x_{\ell-1}) + x_{\ell-1}$$

- This prevents the network from "forgetting" or distorting important information as it is processed by many layers.



Residual connections are also thought to smooth the loss landscape and make training easier!



[no residuals] [residuals]

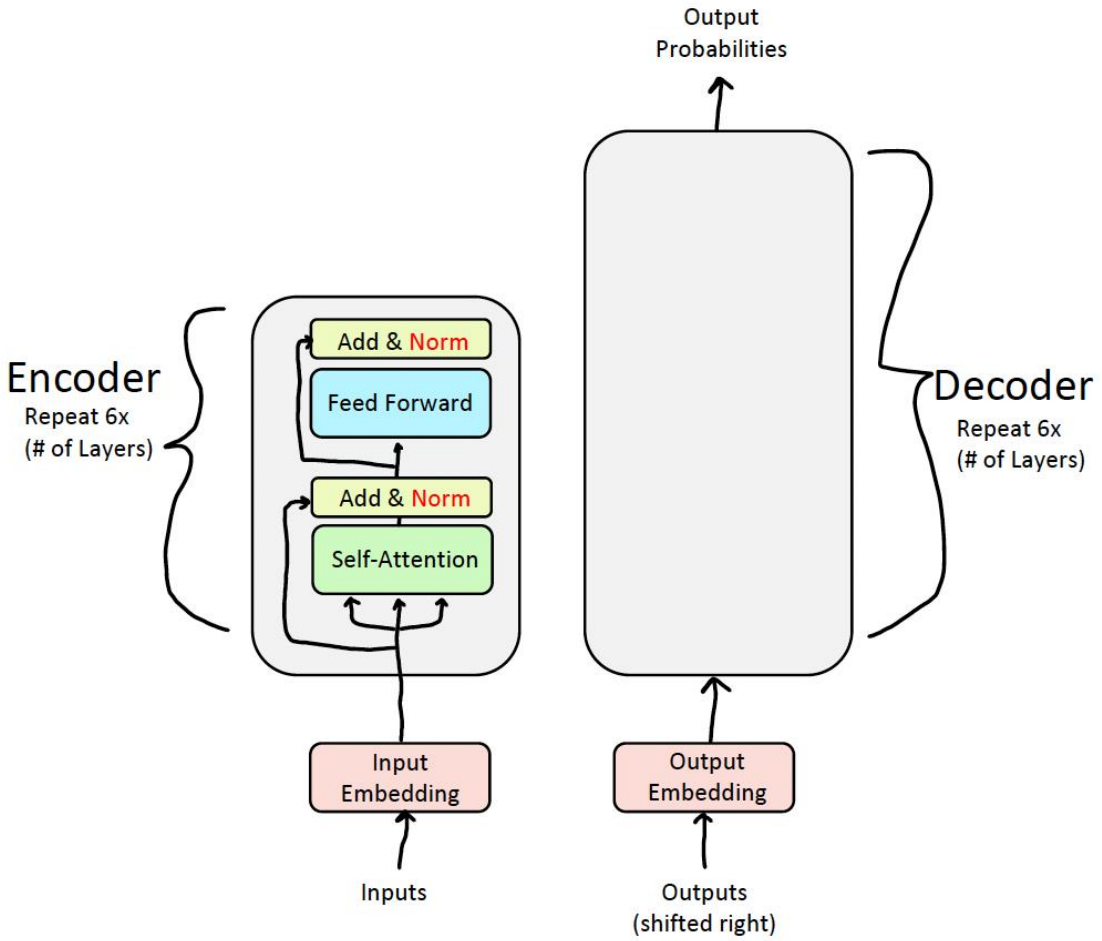
[Loss landscape visualization, Li et al., 2018, on a ResNet]

Training Trick #2: Layer Normalization [Ba et al., 2016]

- **Problem:** Difficult to train the parameters of a given layer because its input from the layer beneath keeps shifting.
- **Solution:** Reduce variation by **normalizing** to zero mean and standard deviation of one within each **layer**.

Mean: $\mu^l = \frac{1}{H}\sum_{i=1}^{H} a_i^l$ Standard Deviation: $\sigma^l = \sqrt{\frac{1}{H}\sum_{i=1}^{H}\left(a_i^l - \mu^l\right)^2}$

$$x^{\ell\prime} = \frac{x^\ell - \mu^\ell}{\sigma^\ell + \epsilon}$$

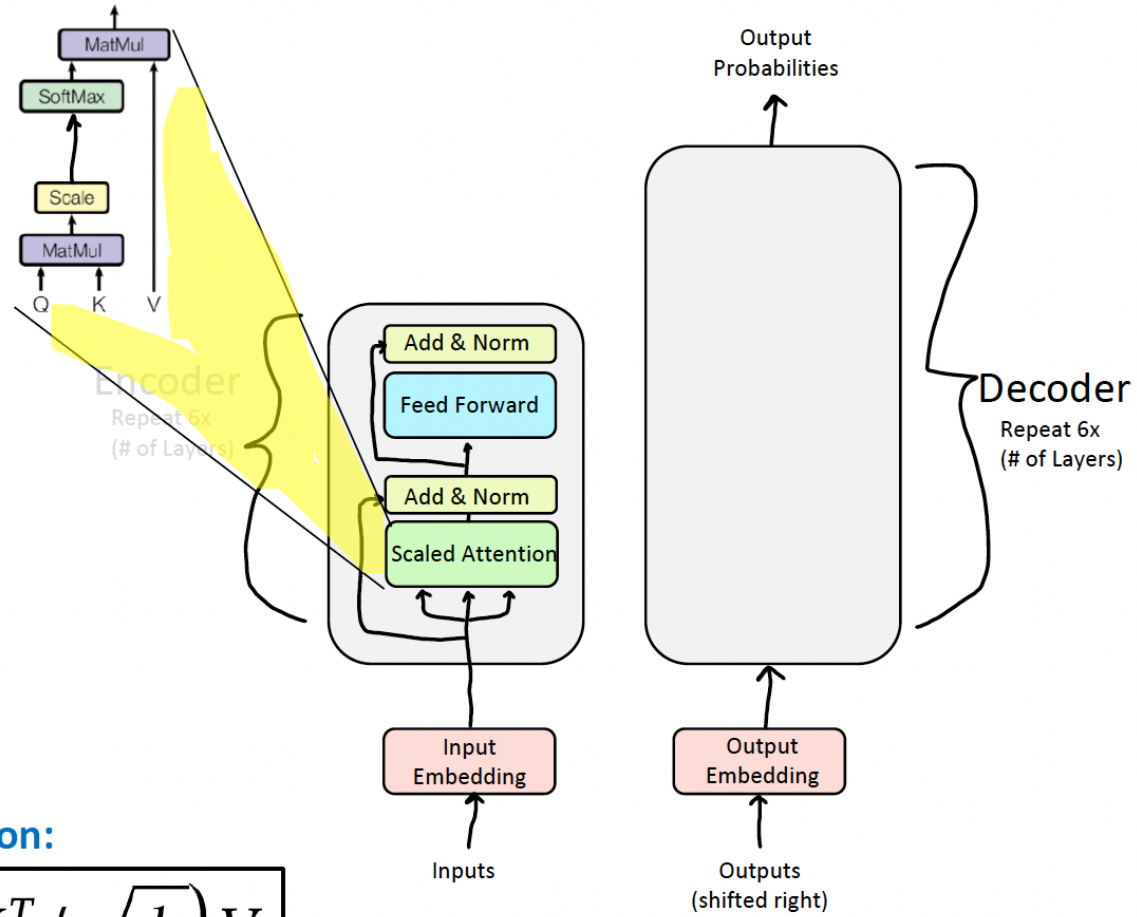


Training Trick #3: Scaled Dot Product Attention

- After LayerNorm, the mean and variance of vector elements is 0 and 1, respectively. (Yay!)
- However, the dot product still tends to take on extreme values, as its variance scales with dimensionality d_k

Quick Statistics Review:

- Mean of sum = sum of means = $d_k * 0 = 0$
- Variance of sum = sum of variances = $d_k * 1 = d_k$
- To set the variance to 1, simply divide by $\sqrt{d_k}$!



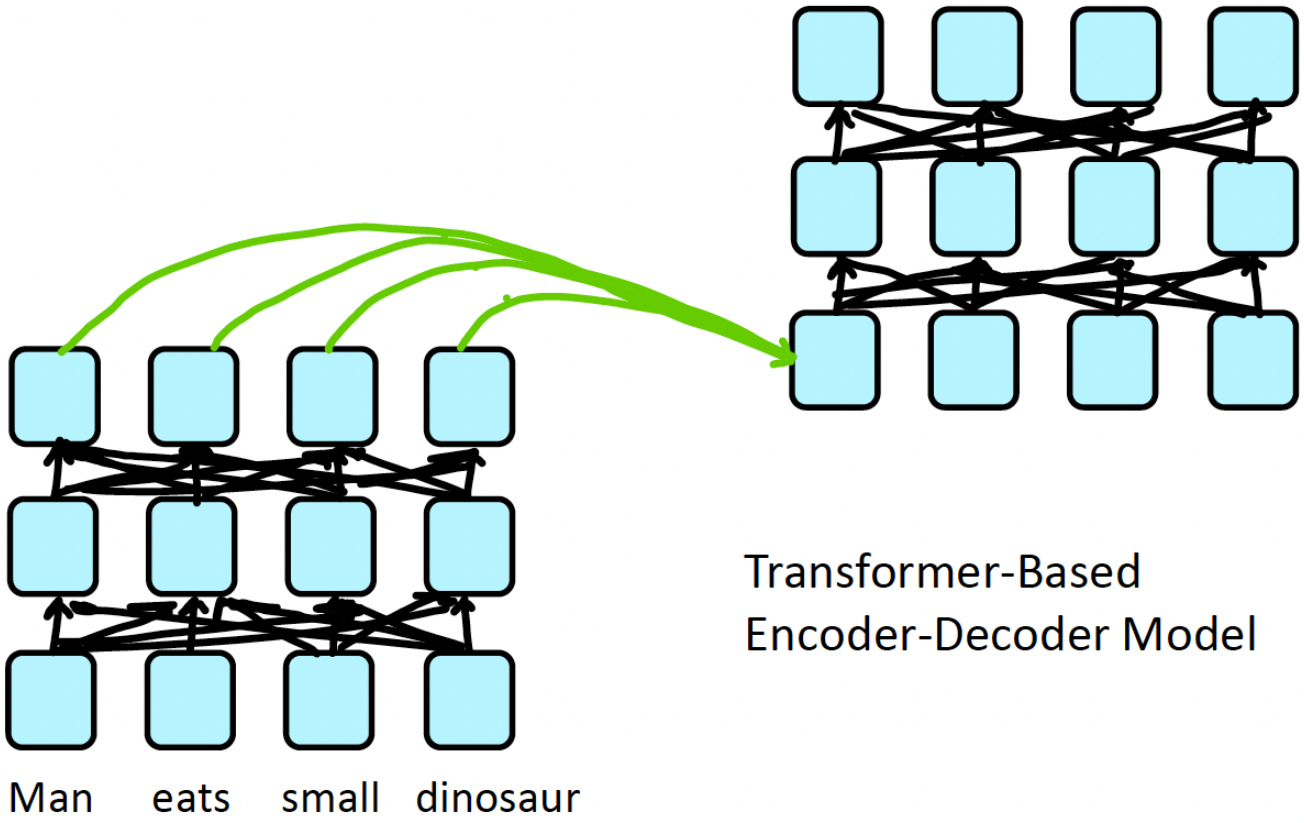
Updated Self-Attention Equation:

$$Output = softmax\left(QK^T / \sqrt{d_k}\right)V$$

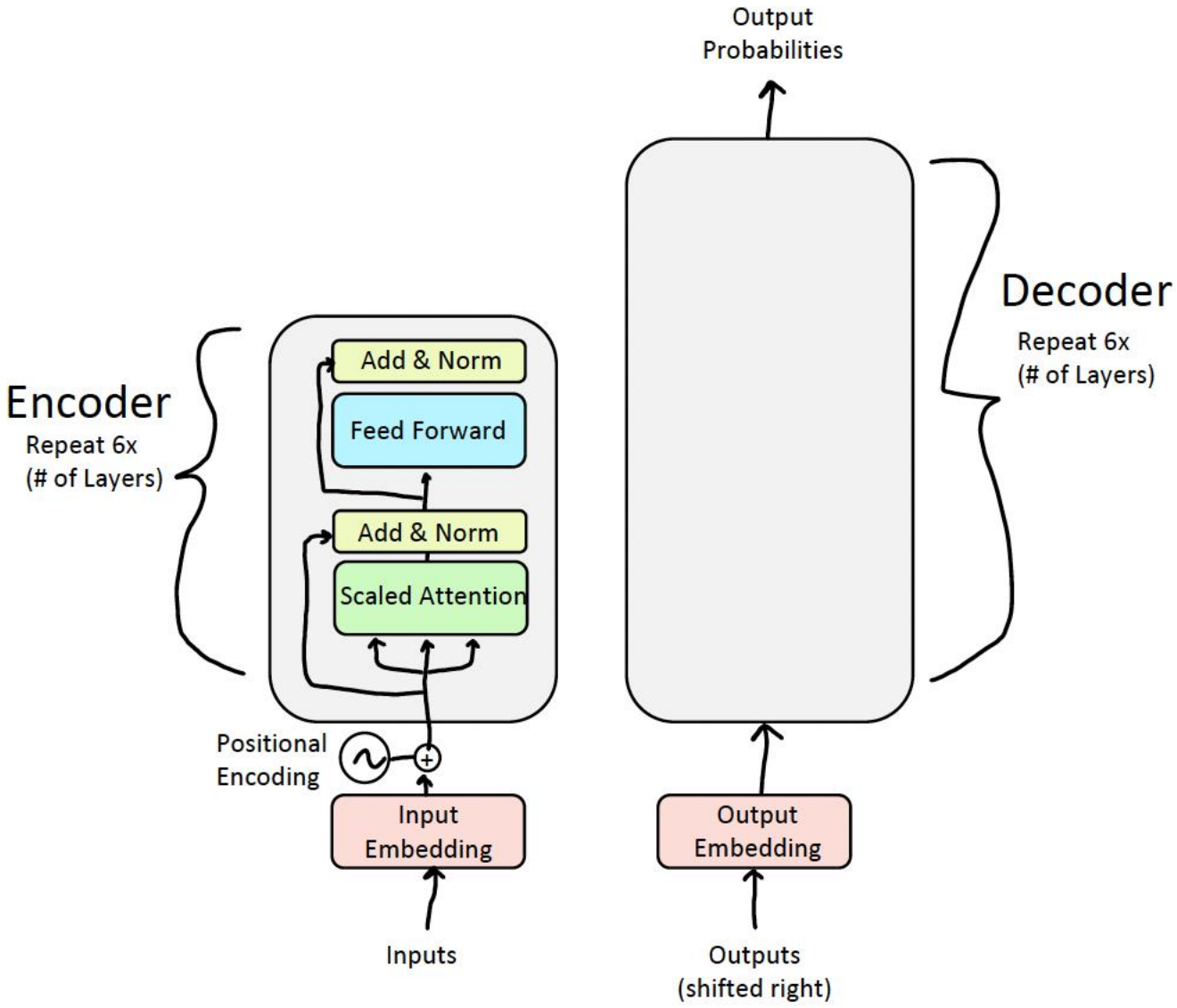
Major issue!

- We're almost done with the Encoder, but we have a major problem! Has anyone spotted it?
- Consider this sentence:
  - "Man eats small dinosaur."
- Wait a minute, order doesn't impact the network at all!
- This seems wrong given that word order does have meaning in many languages, including English!

$$Output = softmax\left(QK^T / \sqrt{d_k}\right)V$$



Solution: Inject Order Information through Positional Encodings!



Fixing the first self-attention problem: sequence order

- Since self-attention doesn't build in order information, we need to encode the order of the sentence in our keys, queries, and values.
- Consider representing each **sequence index** as a **vector**

$$p_i \in \mathbb{R}^d, \text{ for } i \in \{1,2,\dots,T\} \text{ are position vectors}$$

- Don't worry about what the p_i are made of yet!
- Easy to incorporate this info into our self-attention block: just add the p_i to our inputs!
- Let $\tilde{v}_i\ \tilde{k}_i, \tilde{q}_i$ be our old values, keys, and queries.

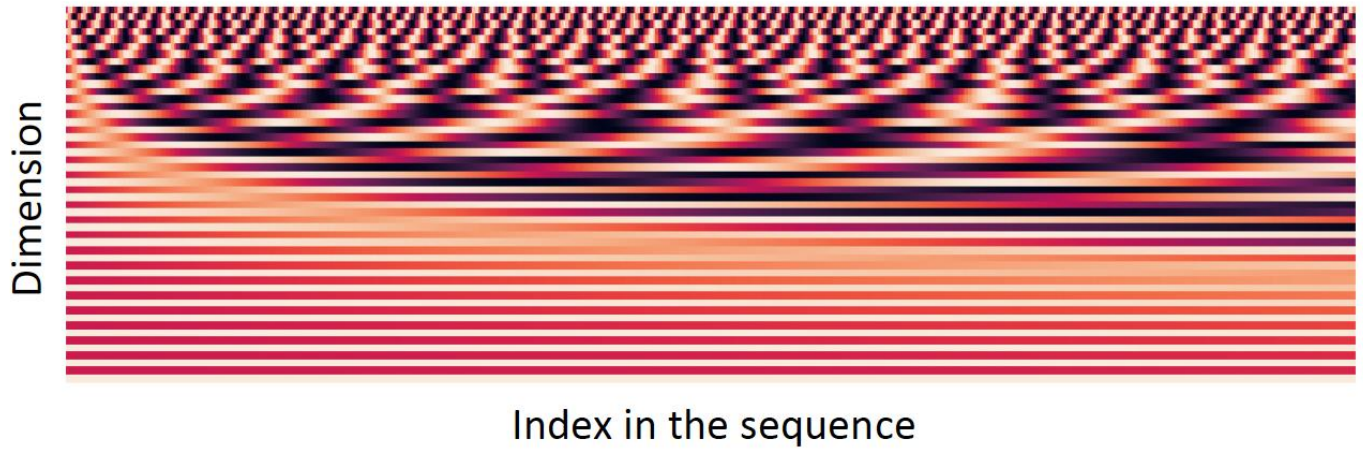
$$v_i = \tilde{v}_i + p_i$$
$$q_i = \tilde{q}_i + p_i$$
$$k_i = \tilde{k}_i + p_i$$

In deep self-attention networks, we do this at the first layer! You could concatenate them as well, but people mostly just add…

Position representation vectors through sinusoids (original)

- **Sinusoidal position representations:** concatenate sinusoidal functions of varying periods:

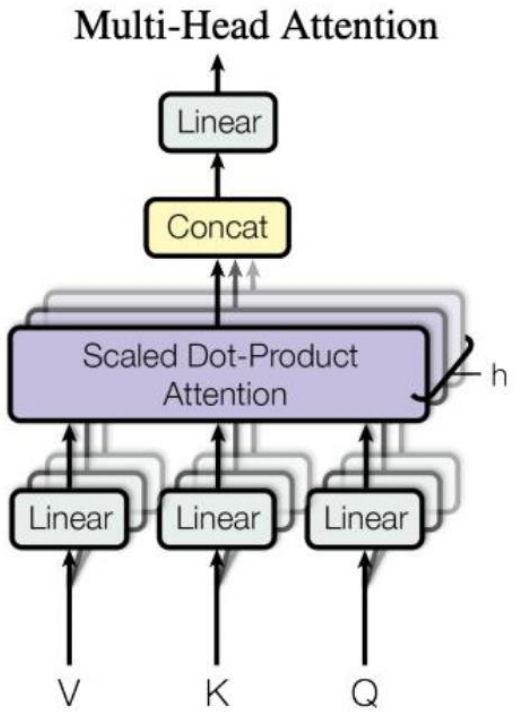
$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



- Pros:
  - Periodicity indicates that maybe "absolute position" isn't as important
  - Maybe can extrapolate to longer sequences as periods restart
- Cons:
  - Not learnable; also the extrapolation doesn't really work

Multi-Headed Self-Attention: k heads are better than 1!

- **High-Level Idea:** Let's perform self-attention multiple times in parallel and combine the results.

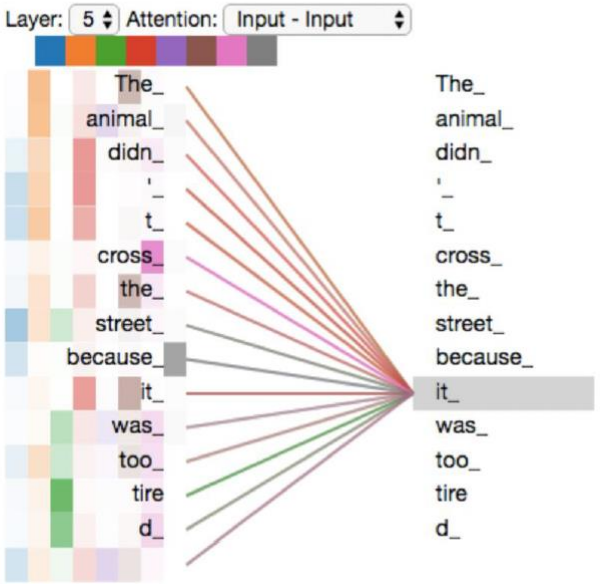


[Vaswani et al. 2017]

Wizards of the Coast, Artist: Todd Lockwood

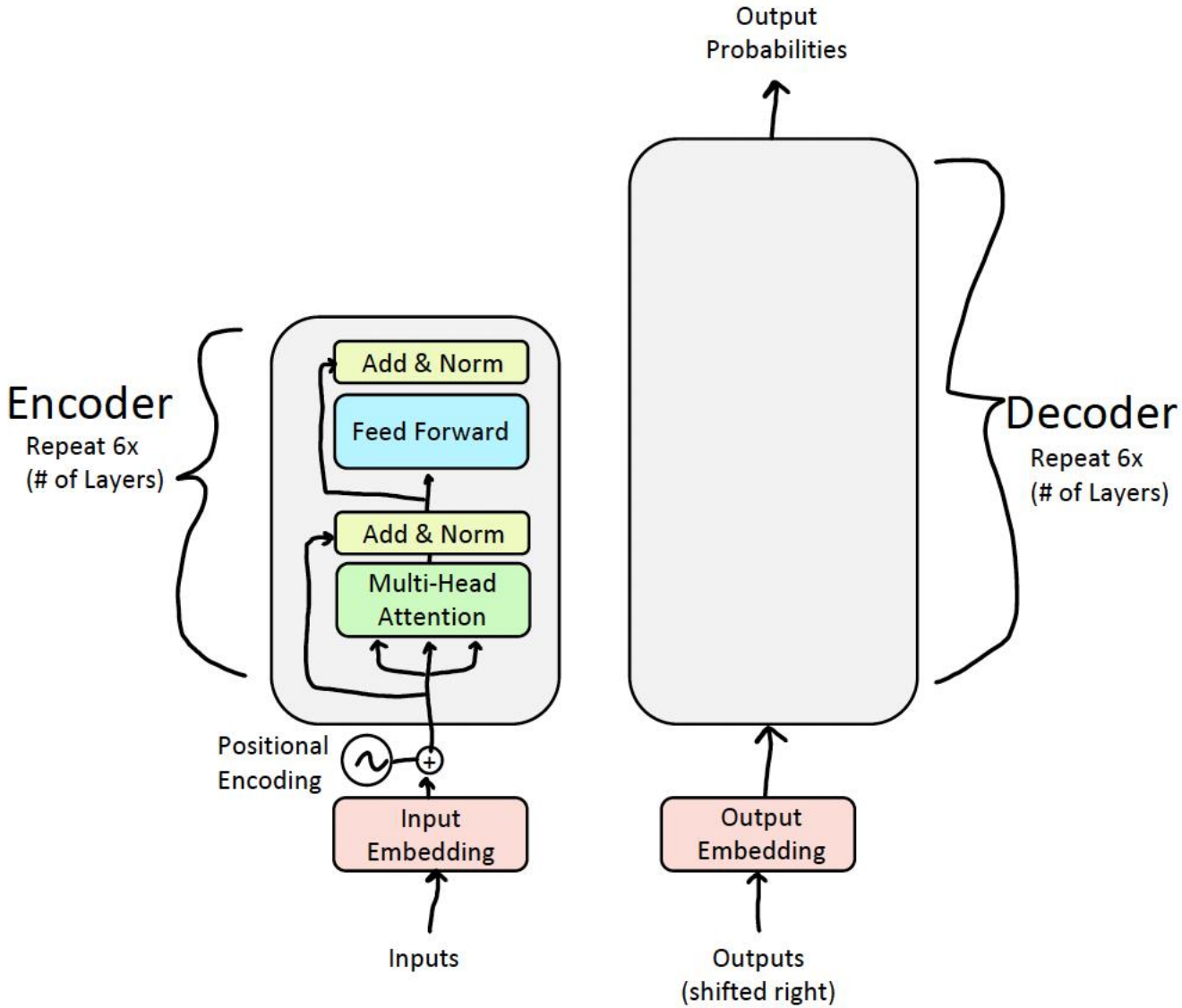
The Transformer Encoder: Multi-headed Self-Attention

- What if we want to look in multiple places in the sentence at once?
  - For word i, self-attention "looks" where $x_i^\top Q^\top K x_j$ is high, but maybe we want to focus on different j for different reasons?
- We'll define **multiple attention "heads"** through multiple Q,K,V matrices
- Let, $Q_\ell, K_\ell, V_\ell \in \mathbb{R}^{d \times \frac{d}{h}}$, where h is the number of attention heads, and ℓ ranges from 1 to h.
- Each attention head performs attention independently:
  - $\text{output}_\ell = \text{softmax}\left(XQ_\ell K_\ell^\top X^\top\right) * XV_\ell$, where $\text{output}_\ell \in \mathbb{R}^{d/h}$
- Then the outputs of all the heads are combined!
  - $\text{output} = Y[\text{output}_1; \ldots; \text{output}_h]$, where $Y \in \mathbb{R}^{d \times d}$
- Each head gets to "look" at different things, and construct value vectors differently.



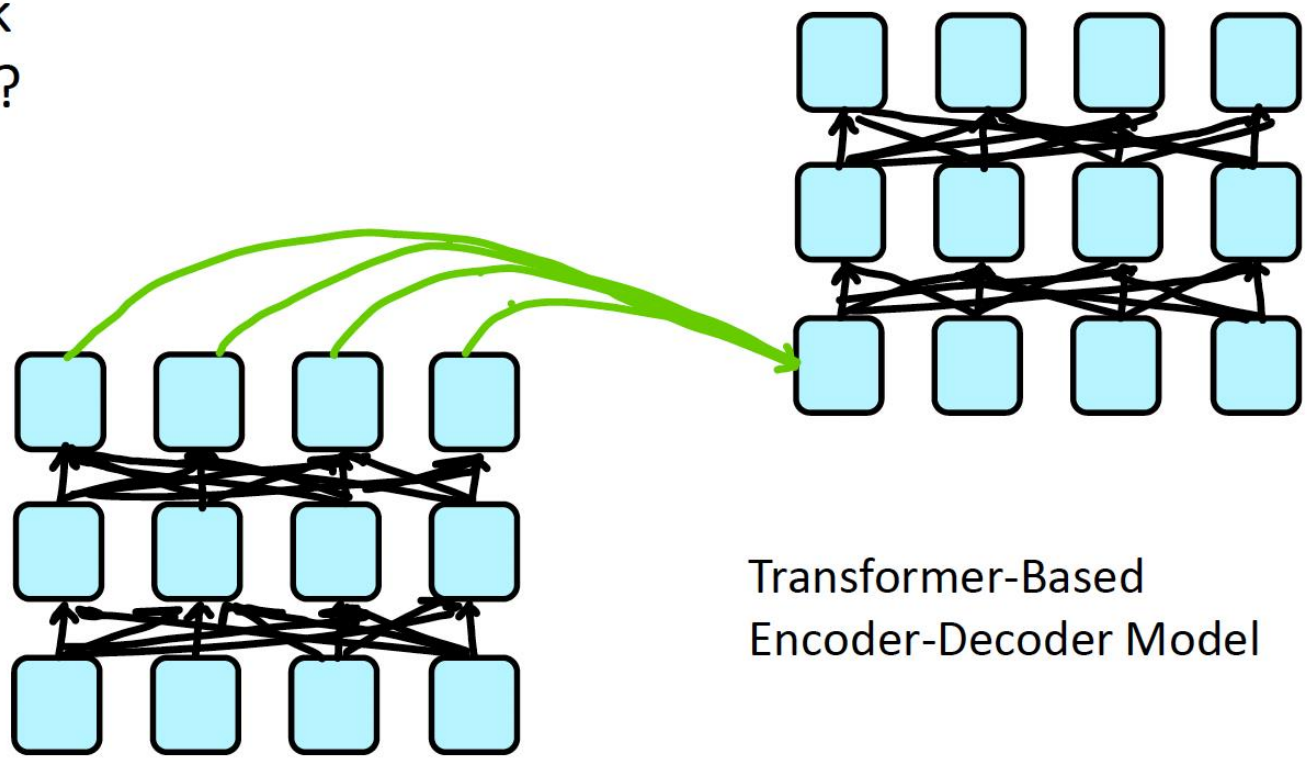
Credit to https://jalammar.github.io/illustrated-transformer/

Yay, we've completed the Encoder! Time for the Decoder...



Decoder: Masked Multi-Head Self-Attention

- **Problem:** How do we keep the decoder from “cheating”? If we have a language modeling objective, can't the network just look ahead and "see" the answer?
- **Solution:** Masked Multi-Head Attention. At a high-level, we hide (mask) information about future tokens from the model.

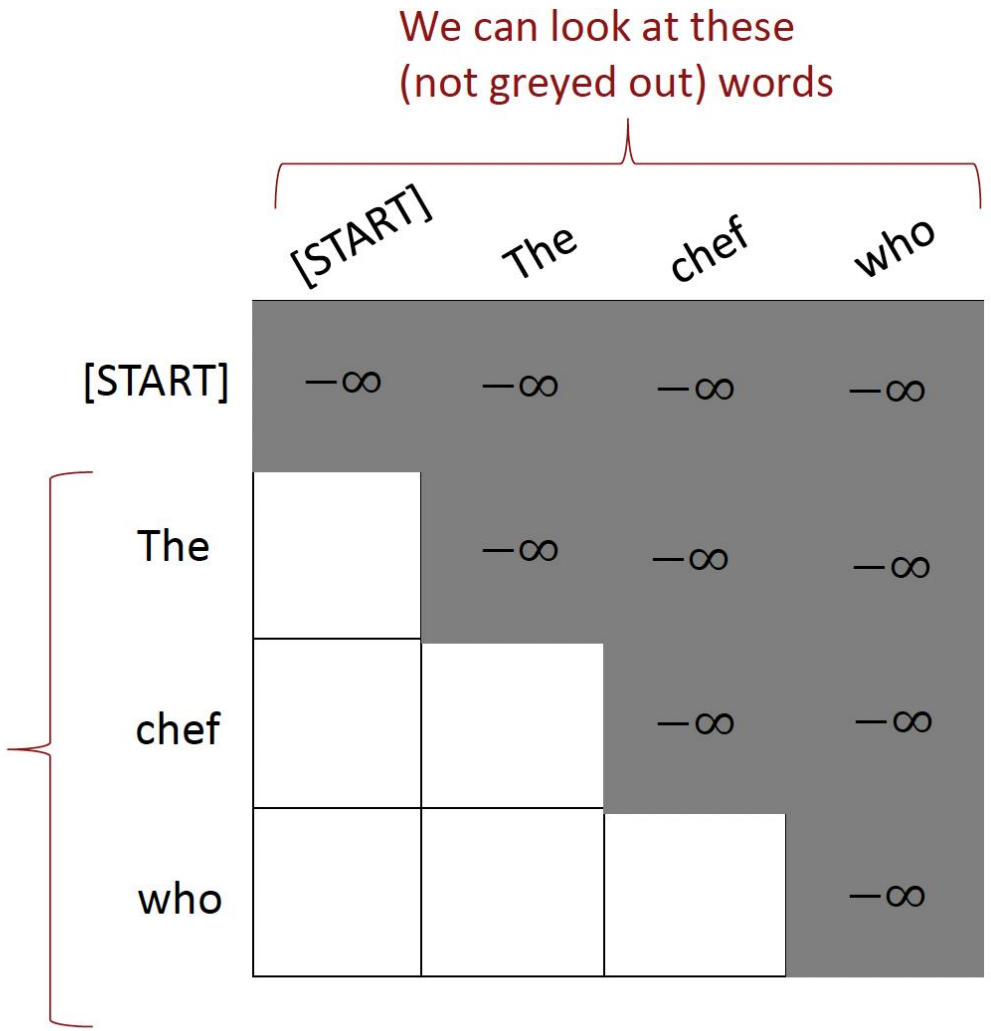


Transformer-Based Encoder-Decoder Model

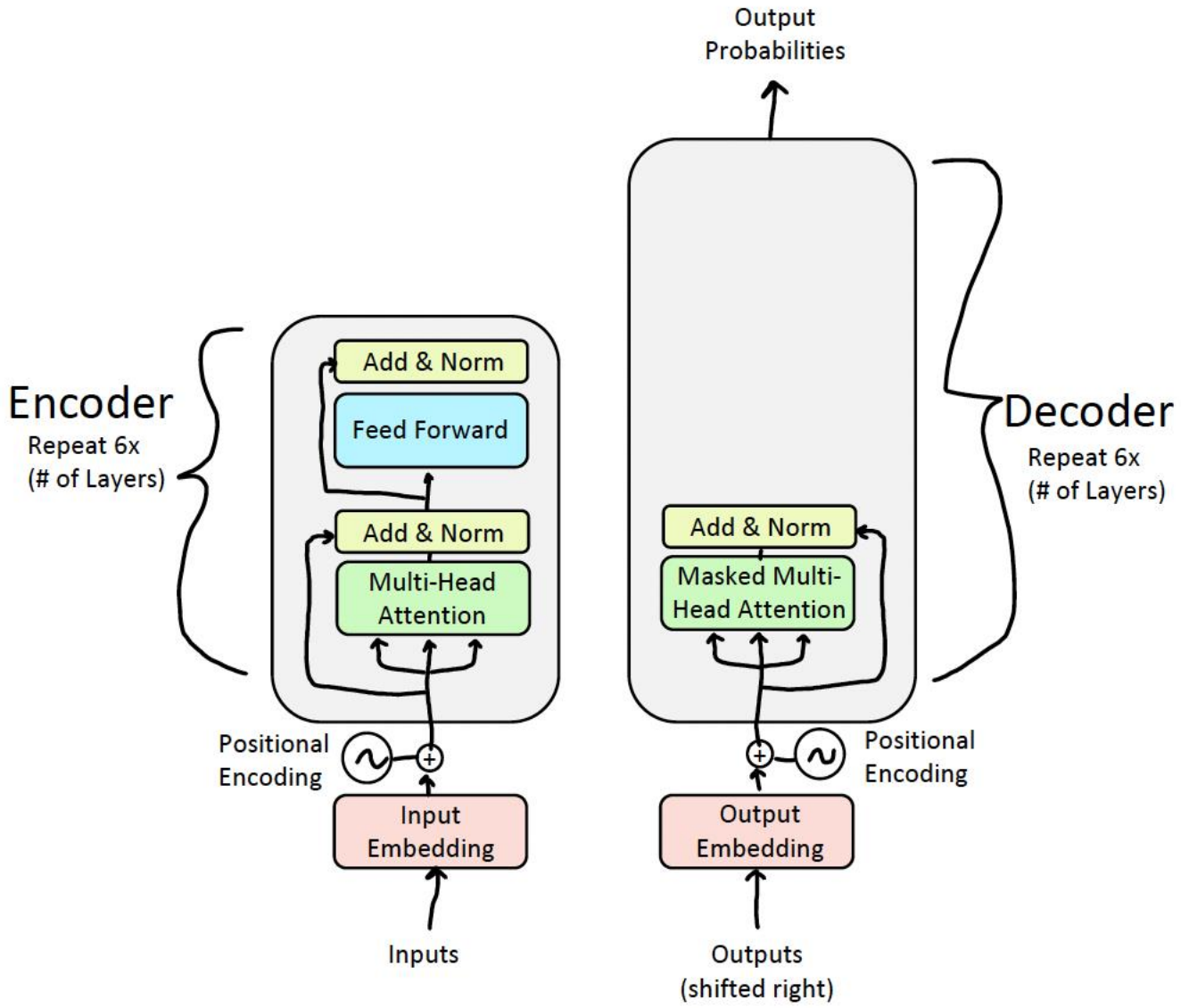
Masking the future in self-attention

- To use self-attention in **decoders**, we need to ensure we can't peek at the future.
- At every timestep, we could change the set of **keys and queries** to include only past words. (Inefficient!)
- To enable parallelization, we **mask out attention** to future words by setting attention scores to $-\infty$.

$$e_{ij} = \begin{cases} q_i^\top k_j, j < i \\ -\infty, j \geq i \end{cases}$$

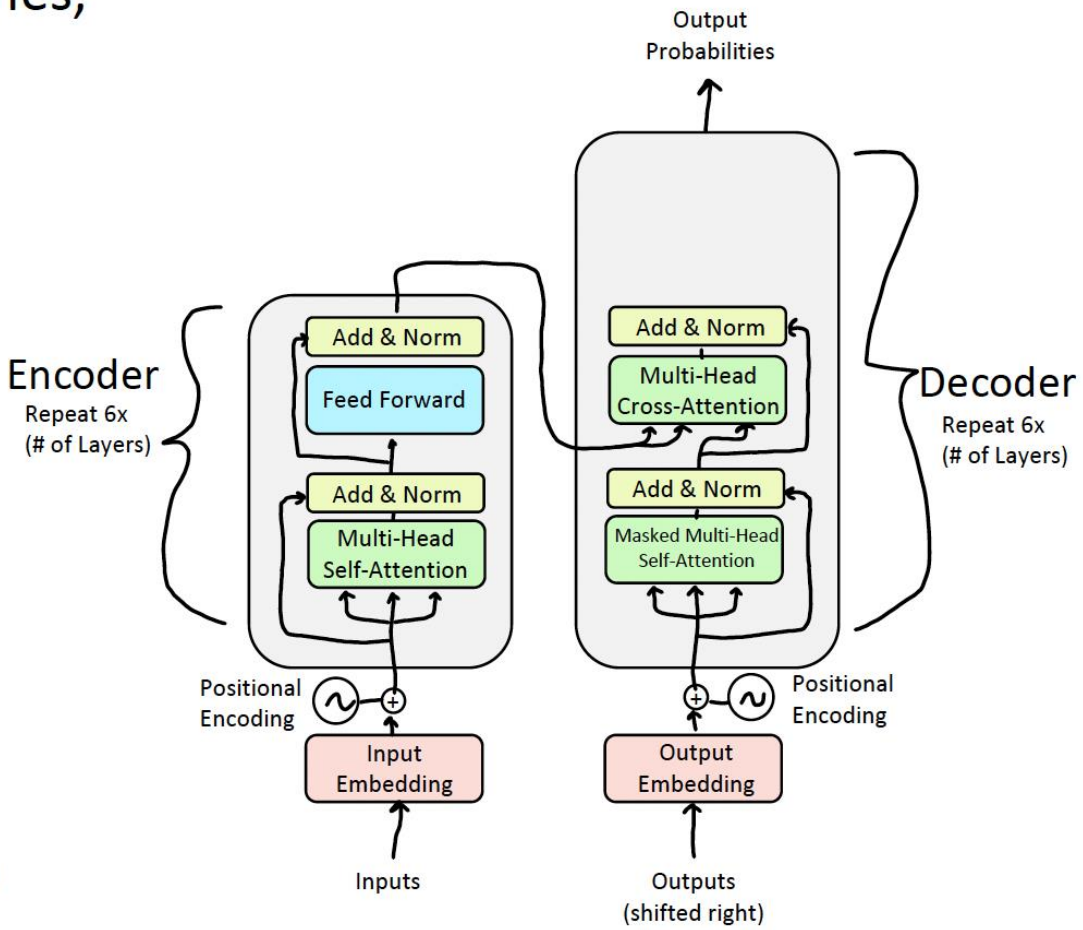


Decoder: Masked Multi-Headed Self-Attention



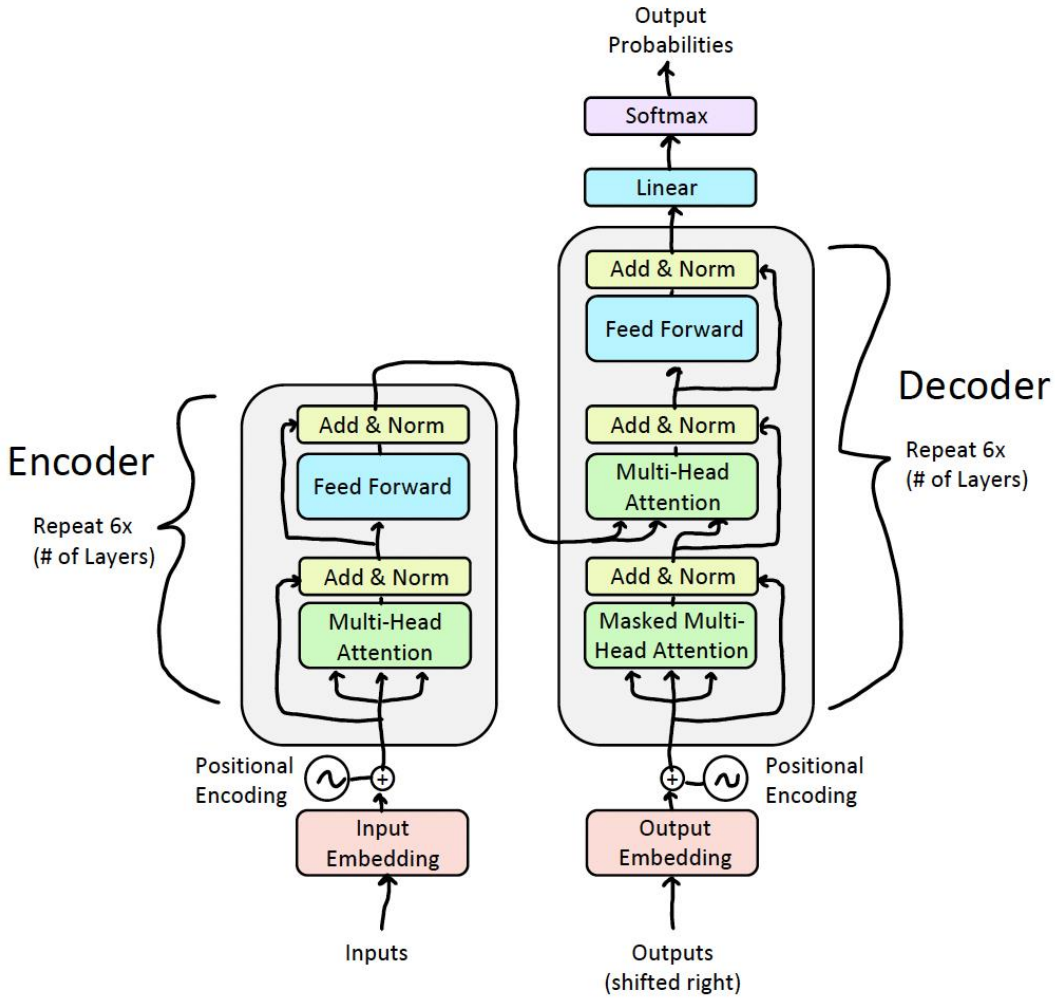
Encoder-Decoder Attention

- We saw that self-attention is when keys, queries, and values come from the same source.
- In the decoder, we have attention that looks more like what we saw last week.
- Let $h_1, \ldots, h_T$ be **output** vectors **from** the Transformer **encoder**; $x_i \in \mathbb{R}^d$
- Let $z_1, \ldots, z_T$ be input vectors from the Transformer **decoder**, $z_i \in \mathbb{R}^d$
- Then keys and values are drawn from the **encoder** (like a memory):
  - $k_i = Kh_i, v_i = Vh_i.$
- And the queries are drawn from the **decoder**, $q_i = Qz_i.$



Decoder: Finishing touches!

- Add a feed forward layer (with residual connections and layer norm)
- Add a final linear layer to project the embeddings into a much longer vector of length vocab size (logits)
- Add a final softmax to generate a probability distribution of possible next words!



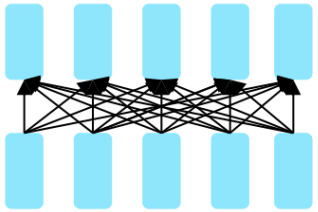
HUST

hust.edu.vn fb.com/dhbkhn

Pretraining architectures

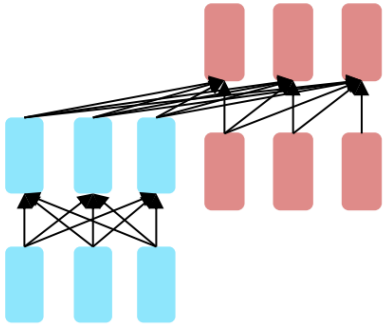
Pretraining architectures

The neural architecture influences the type of pretraining, and natural use cases.



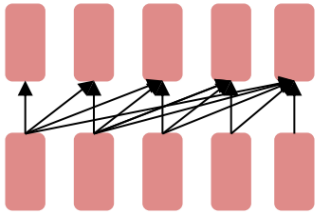
Encoders

- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



Encoder-Decoders

- Good parts of decoders and encoders?
- What's the best way to pretrain them?



Decoders

- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

Encoder-only: BERT

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

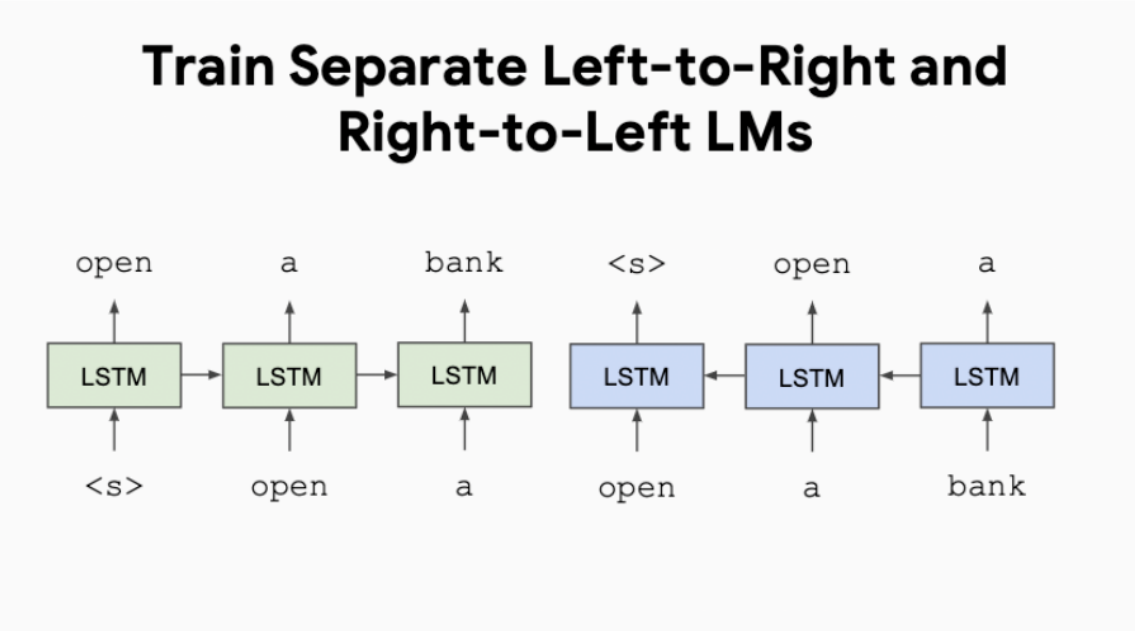
Jacob Devlin **Ming-Wei Chang** **Kenton Lee** **Kristina Toutanova**
Google AI Language
{jacobdevlin,mingweichang,kentonl,kristout}@google.com

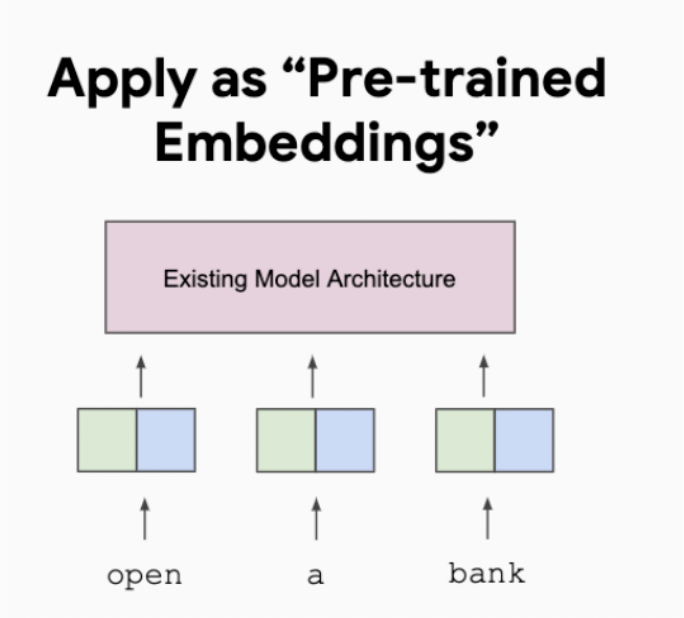
Released in 2018/10, NAACL 2019 best paper

Prior work: ELMo

ELMo (Peters et al., 2018; NAACL 2018 best paper)

- Train two separate **unidirectional** LMs (left-to-right and right-to-left) based on **LSTMs**
- **Feature-based** approach: pre-trained representations used as input to task-specific models
- Trained on **single sentences** from 1B word benchmark (Chelba et al., 2014)

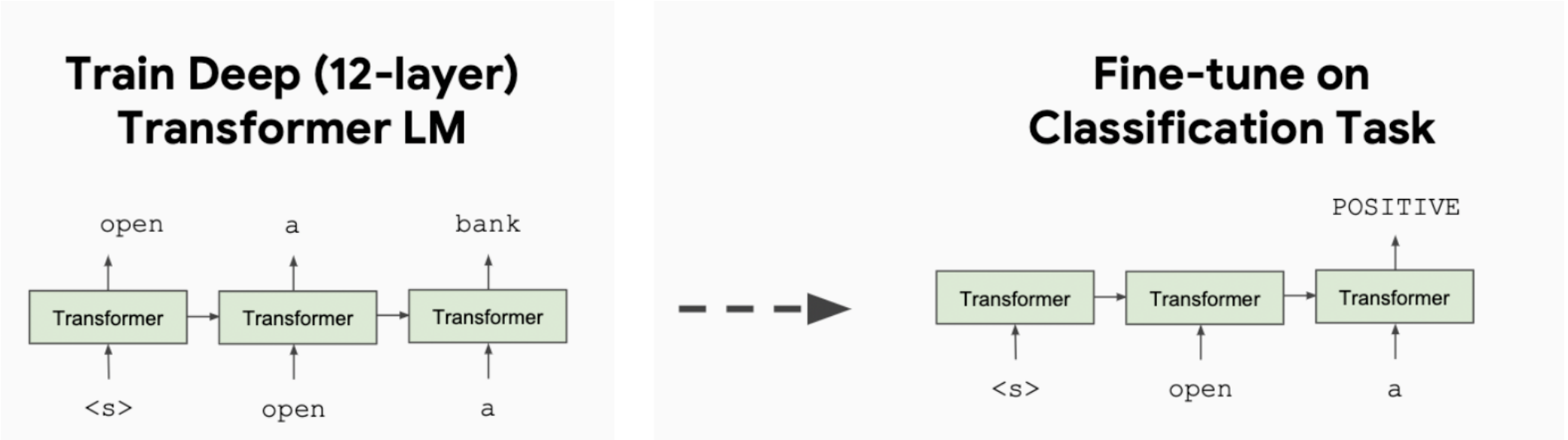




Prior work: OpenAI GPT

OpenAI GPT (Radford et al., 2018; released in 2018/6)

- Train one unidirectional LM (left-to-right) based on a deep **Transformer decoder**
- **Fine-tuning** approach: all pre-trained parameters are re-used & updated on downstream tasks
- Trained on 512-token segments on BooksCorpus — much **longer** context!



BERT: key contributions

- It is a **fine-tuning approach** based on a deep **Transformer encoder**
- The key: learn representations based on **bidirectional context**

Why? Because both left and right contexts are important to understand the meaning of words.

Example #1: we went to the river bank.

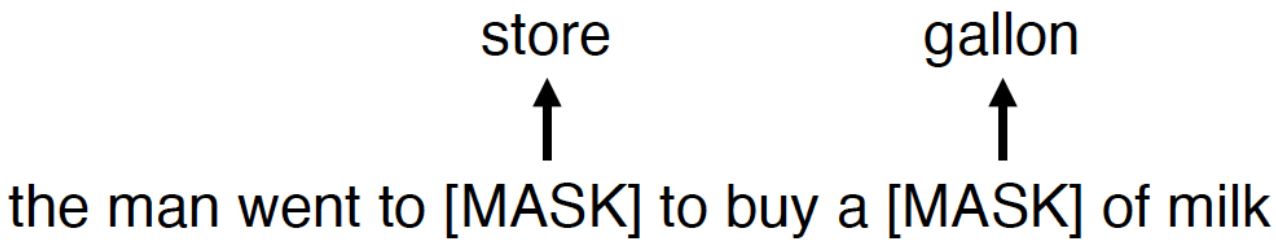
Example #2: I need to go to bank to make a deposit.

- **Pre-training objectives**: masked language modeling + next sentence prediction
- State-of-the-art performance on a large set of **sentence-level** and **token-level** tasks

Masked Language Modeling (MLM)

- Q: Why we can't do language modeling with bidirectional models?

- Solution: Mask out k% of the input words, and then predict the masked words



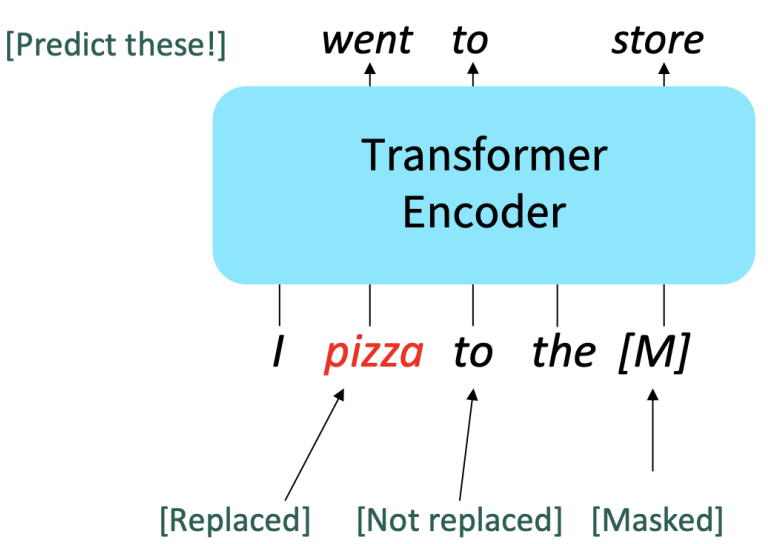
MLM: masking rate and strategy

- **Q: What is the value of k?**
  - They always use k = 15%.
  - Too little masking: computationally expensive
  - Too much masking: not enough context
  - See (Wettig et al., 2022) for more discussion of masking rates

- **Q: How are masked tokens selected?**
  - 15% tokens are uniformly sampled
  - Is it optimal? See span masking (Joshi et al., 2020) and PMI masking (Levine et al., 2021)

Example: He [MASK] from Kuala [MASK] , Malaysia.

MLM: 80-10-10 corruption



For the 15% predicted words,

- 80% of the time, they replace it with [MASK] token

  went to the store ⟶ went to the [MASK]

- 10% of the time, they replace it with a random word in the vocabulary

  went to the store ⟶ went to the running

- 10% of the time, they keep it unchanged

  went to the store ⟶ went to the store

Why? Because [MASK] tokens are never seen during fine-tuning

(See Table 8 for an ablation study)

Next Sentence Prediction (NSP)

- Motivation: many NLP downstream tasks require understanding the relationship between two sentences (natural language inference, paraphrase detection, QA)
- NSP is designed to reduce the gap between pre-training and fine-tuning

[CLS]: a special token always at the beginning

[SEP]: a special token used to separate two segments

Input = `[CLS] the man went to [MASK] store [SEP]`

`he bought a gallon [MASK] milk [SEP]`

Label = `IsNext`

Input = `[CLS] the man [MASK] to the store [SEP]`

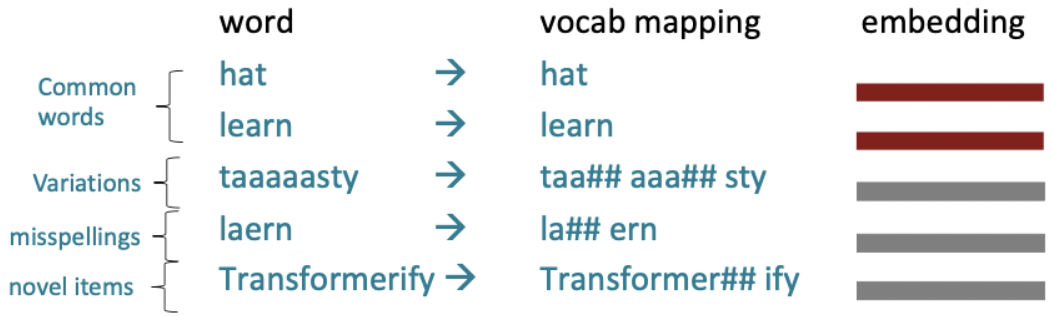
`penguin [MASK] are flight ##less birds [SEP]`

Label = `NotNext`

They sample two contiguous segments for 50% of the time and another random segment from the corpus for 50% of the time

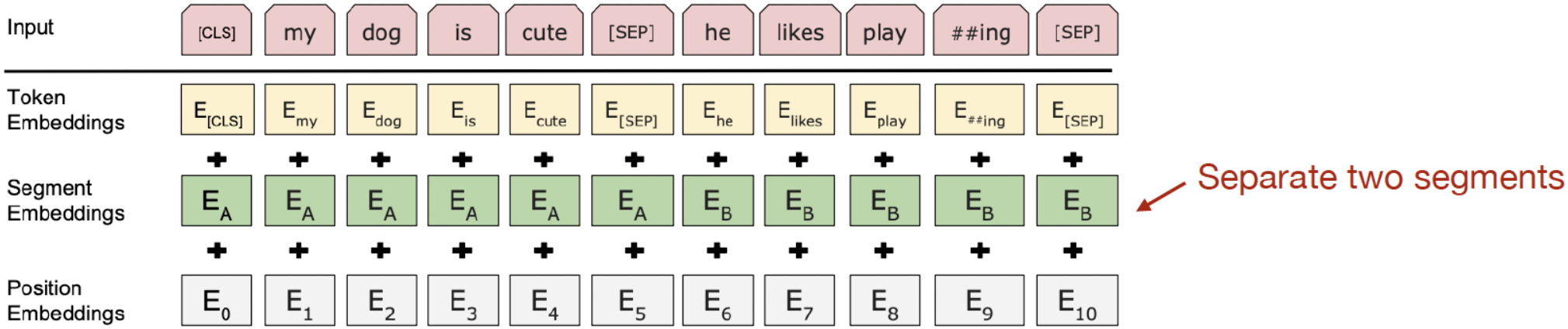
BERT pre-training: putting together

- Vocabulary size: 30,000 workpieces (common sub-word units) (Wu et al., 2016)

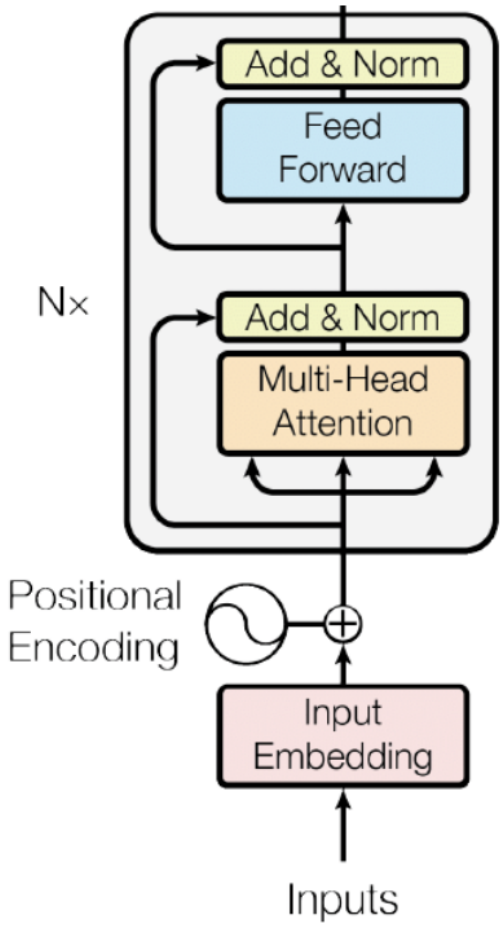


(Image: Stanford CS224N)

- Input embeddings:



BERT pre-training: putting together



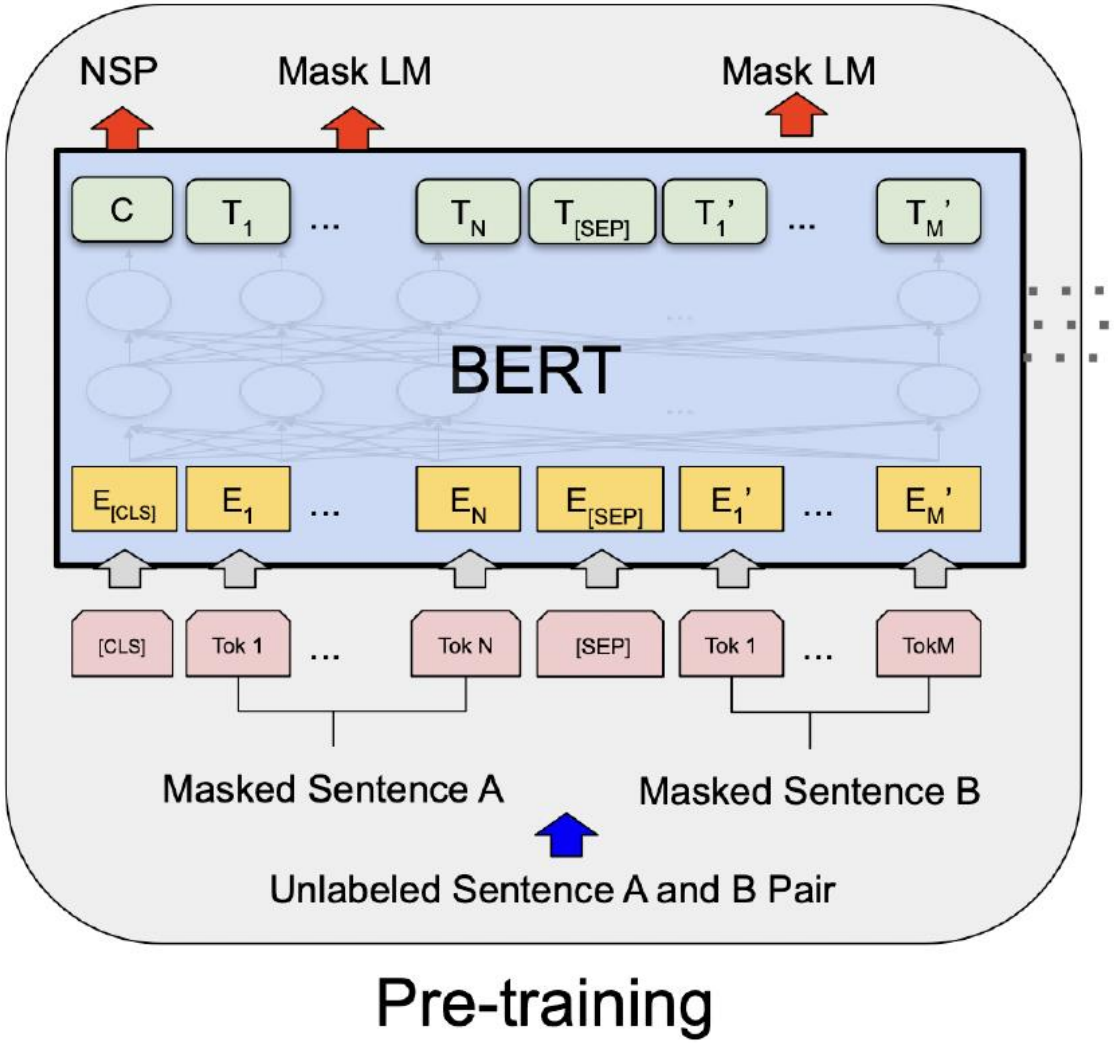
- BERT-base: 12 layers, 768 hidden size, 12 attention heads, 110M parameters
- BERT-large: 24 layers, 1024 hidden size, 16 attention heads, 340M parameters

Same as OpenAI GPT

OpenAI GPT was trained on BooksCorpus only!

- Training corpus: Wikipedia (2.5B) + BooksCorpus (0.8B)
- Max sequence size: 512 word pieces (roughly 256 and 256 for two non-contiguous sequences)
- Trained for 1M steps, batch size 128k

BERT pre-training: putting together



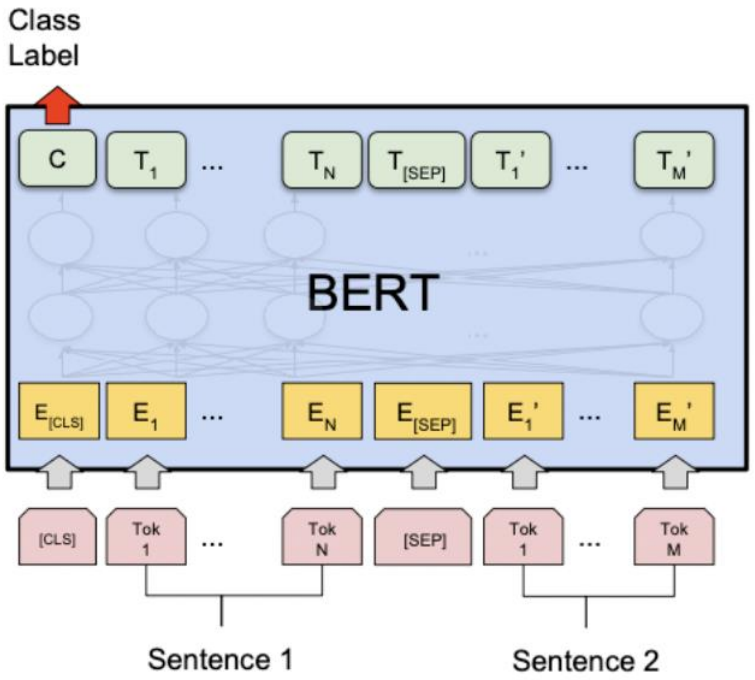
Pre-training

- MLM and NSP are trained together
- [CLS] is pre-trained for NSP
- Other token representations are trained for MLM

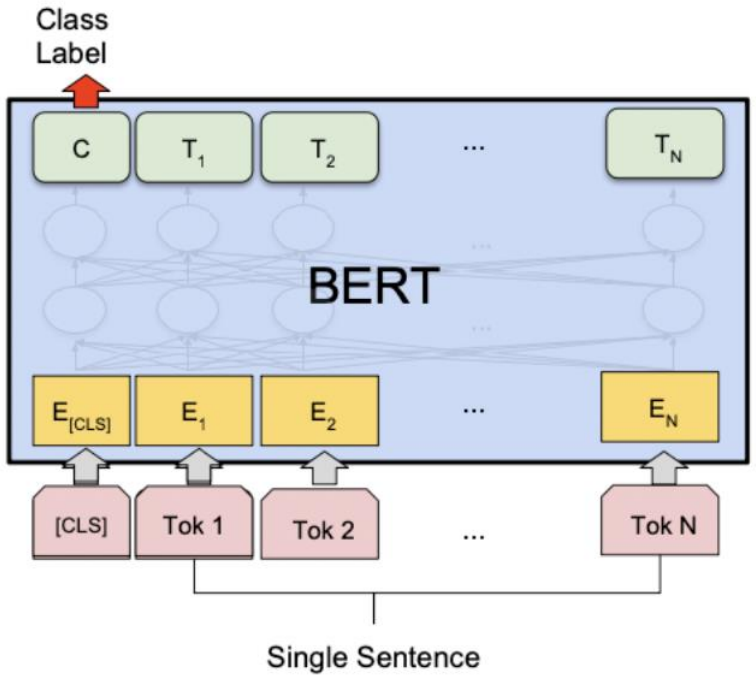
Fine-tuning BERT

"Pretrain once, finetune many times."

sentence-level tasks



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

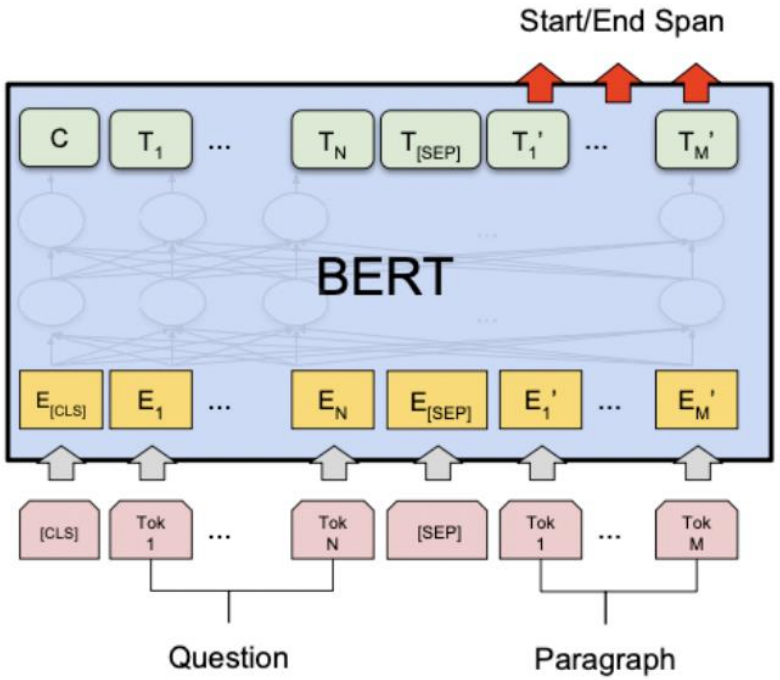


(b) Single Sentence Classification Tasks: SST-2, CoLA

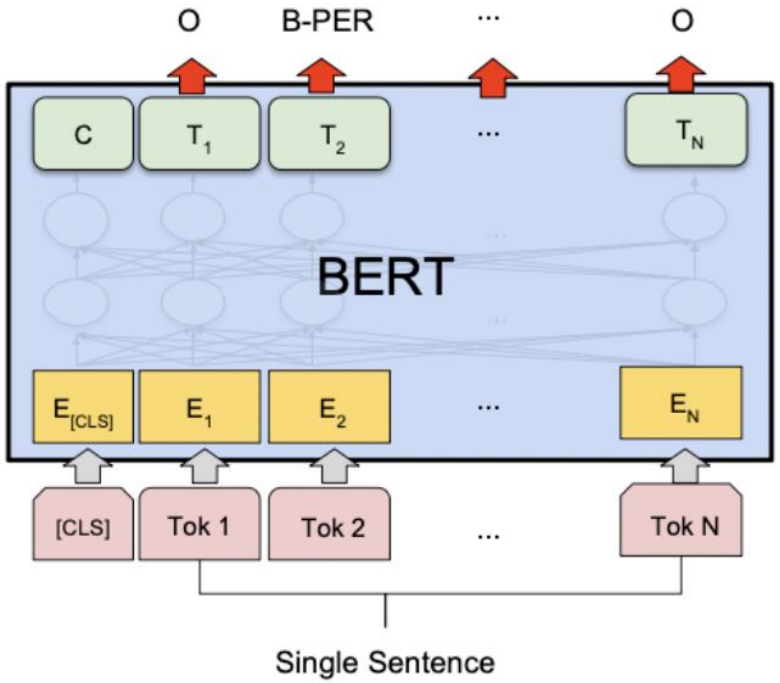
Fine-tuning BERT

"Pretrain once, finetune many times."

token-level tasks



(c) Question Answering Tasks: SQuAD v1.1



(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

Sentence-level tasks

- Sentence pair classification tasks:

MNLI

Premise: A soccer game with multiple males playing.

Hypothesis: Some men are playing a sport.

{entailment, contradiction, neutral}

QQP

Q1: Where can I learn to invest in stocks?

Q2: How can I learn more about stocks?

{duplicate, not duplicate}

- Single sentence classification tasks:

SST2

rich veins of funny stuff in this movie

{positive, negative}

GLUE (Wang et al., 2019): 6 sentence pair and 2 single-sentence tasks

Token-level tasks

- Extractive question answering e.g., SQuAD (Rajpurkar et al., 2016)

SQuAD

Question: The New York Giants and the New York Jets play at which stadium in NYC ?

Context: The city is represented in the National Football League by the New York Giants and the New York Jets , although both teams play their home games at MetLife Stadium in nearby East Rutherford , New Jersey , which hosted Super Bowl XLVIII in 2014 .

(Training example 29,883)

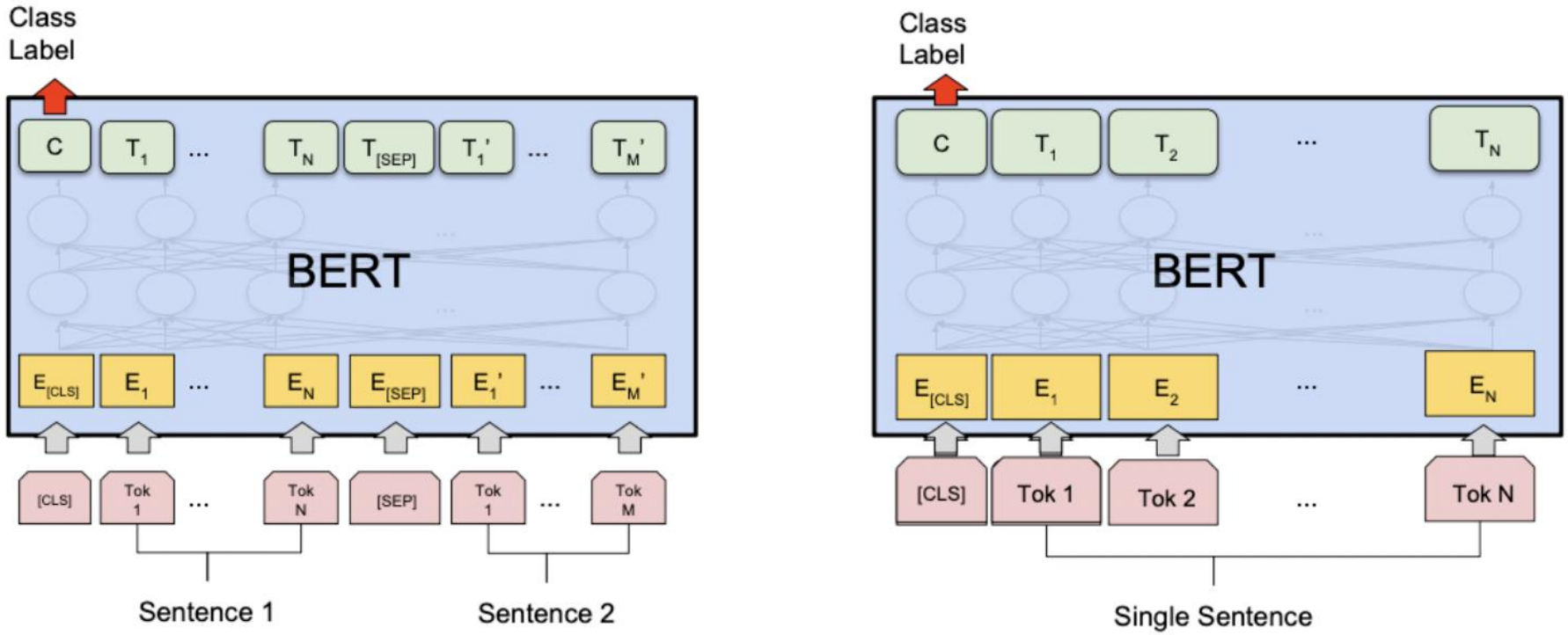
MetLife Stadium

- Named entity recognition (Tjong Kim Sang and De Meulder, 2003)

CoNLL 2003 NER

John	Smith	lives	in	New	York
B-PER	I-PER	O	O	B-LOC	I-LOC

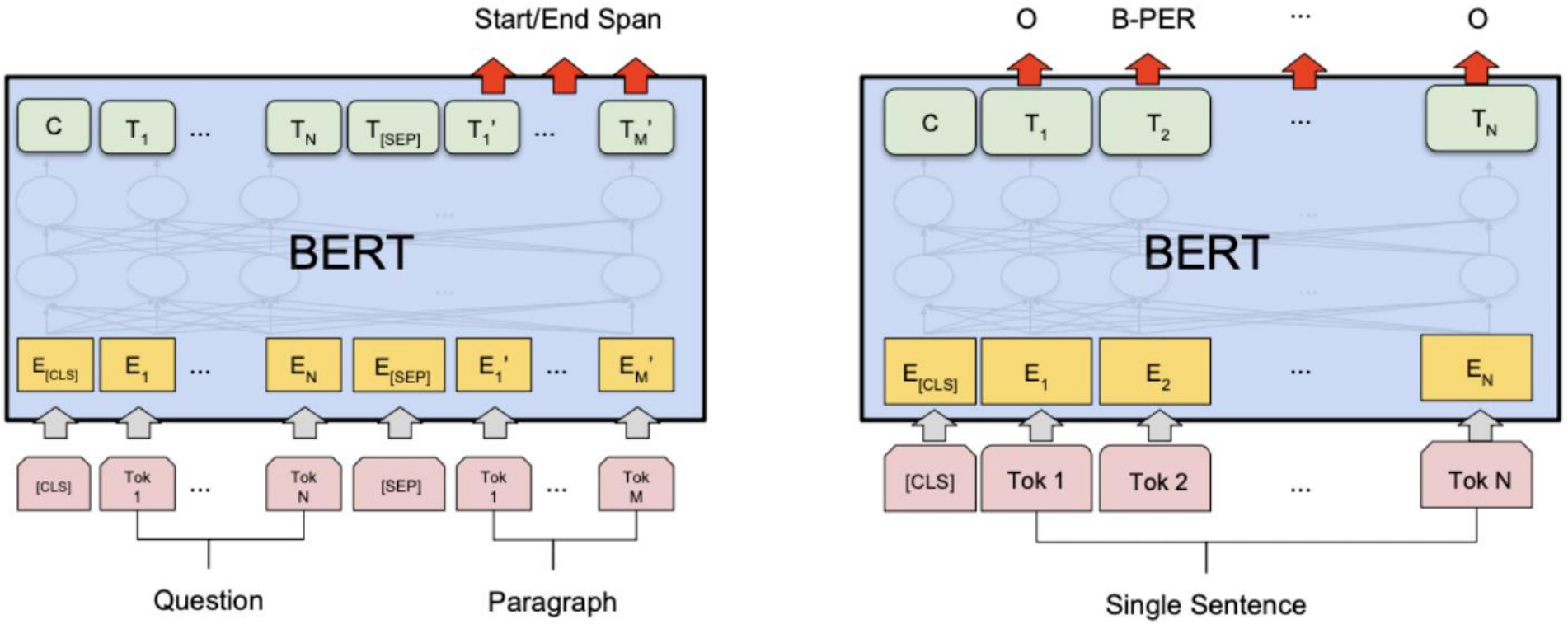
Fine-tuning BERT



- For sentence pair tasks, use [SEP] to separate the two segments with segment embeddings
- Add a linear classifier on top of [CLS] representation and introduce $C \times h$ new parameters

C: # of classes, h: hidden size

Fine-tuning BERT



- For token-level prediction tasks, add linear classifier on top of hidden representations

Q: How many new parameters?

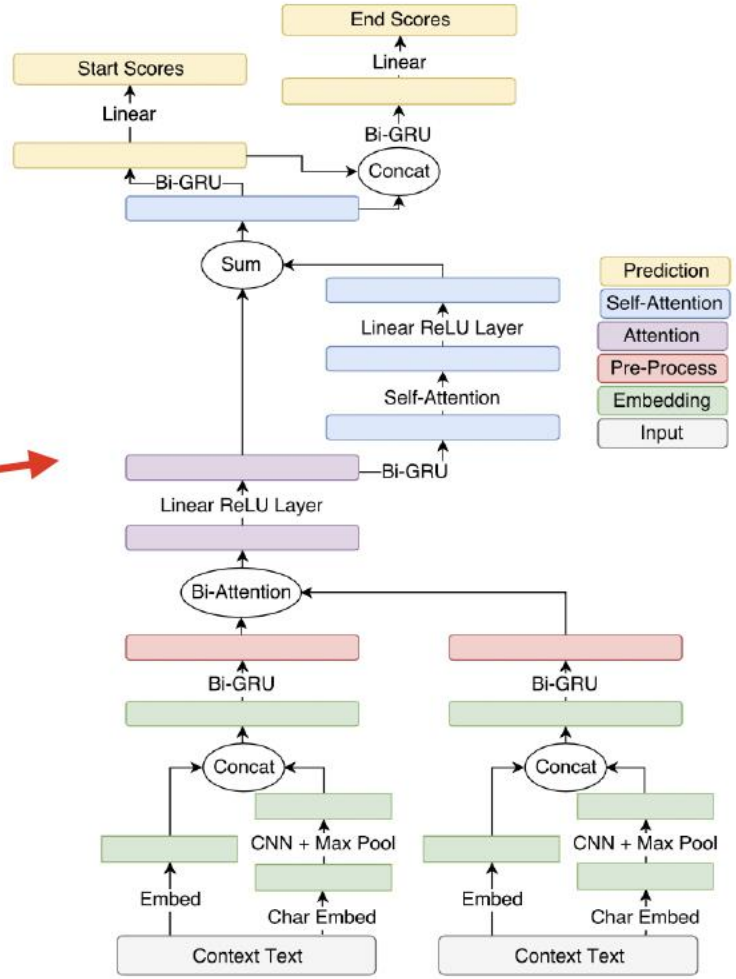
Experimental results: GLUE

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

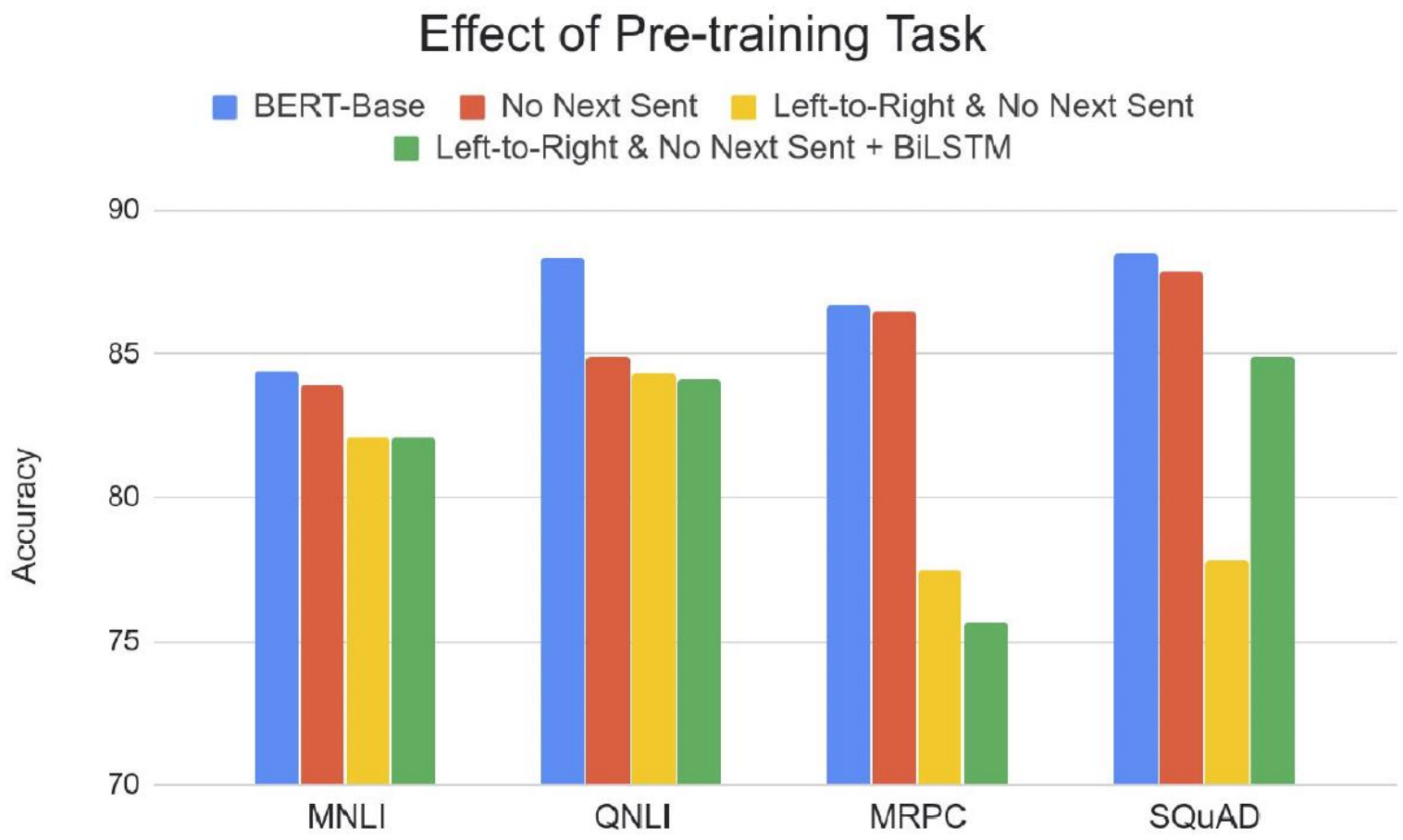
See Appendix A.4 for detailed differences between BERT and OpenAI GPT

Experimental results: SQuAD

System	Dev EM	Dev F1	Test EM	Test F1
Top Leaderboard Systems (Dec 10th, 2018)				
Human	-	-	82.3	91.2
#1 Ensemble - nlnet	-	-	86.0	91.7
#2 Ensemble - QANet	-	-	84.5	90.5
Published				
BiDAF+ELMo (Single)	-	85.6	-	85.8
R.M. Reader (Ensemble)	81.2	87.9	82.3	88.5
Ours				
$\text{BERT}_{\text{BASE}}$ (Single)	80.8	88.5	-	-
$\text{BERT}_{\text{LARGE}}$ (Single)	84.1	90.9	-	-
$\text{BERT}_{\text{LARGE}}$ (Ensemble)	85.8	91.8	-	-
$\text{BERT}_{\text{LARGE}}$ (Sgl.+TriviaQA)	**84.2**	**91.1**	**85.1**	**91.8**
$\text{BERT}_{\text{LARGE}}$ (Ens.+TriviaQA)	**86.2**	**92.2**	**87.4**	**93.2**



Ablation study: pre-training tasks



- MLM >> left-to-right LMs
- NSP improves on some tasks
- Note: later work (Joshi et al., 2020; Liu et al., 2019) argued that NSP is not useful

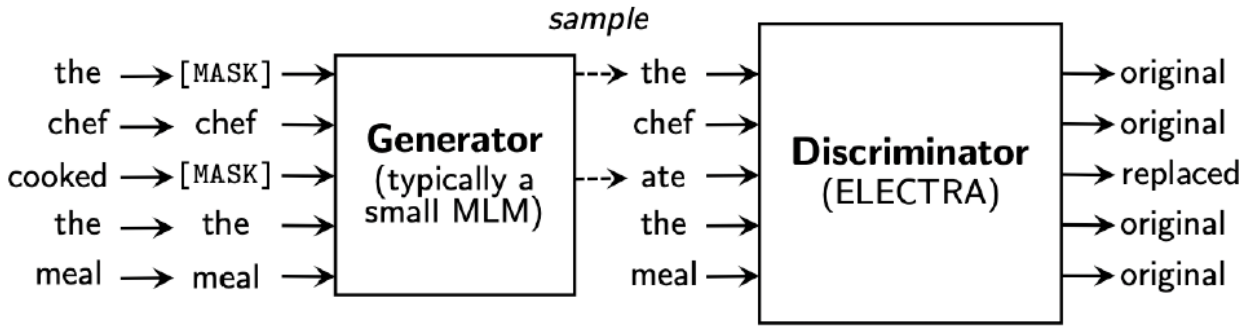
Ablation study: model sizes

Hyperparams			Dev Set Accuracy			
#L	#H	#A	LM (ppl)	MNLI-m	MRPC	SST-2
3	768	12	5.84	77.9	79.8	88.4
6	768	3	5.24	80.6	82.2	90.7
6	768	12	4.68	81.9	84.8	91.3
12	768	12	3.99	84.4	86.7	92.9
12	1024	16	3.54	85.7	86.9	93.3
24	1024	16	3.23	86.6	87.8	93.7

The bigger, the better!

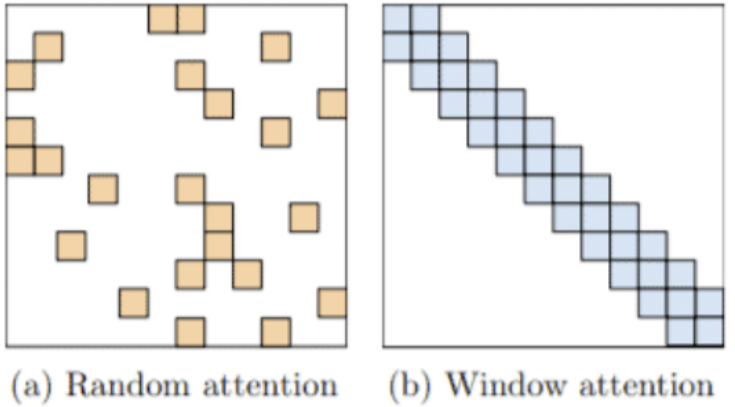
What happened after BERT?

- RoBERTa (Liu et al., 2019)
  - Trained on 10x data & longer, no NSP
  - Much stronger performance than BERT (e.g., 94.6 vs 90.9 on SQuAD)
  - Still one of the most popular models to date
- ALBERT (Lan et al., 2020)
  - Increasing model sizes by sharing model parameters across layers
  - Less storage, much stronger performance but runs slower..
- ELECTRA (Clark et al., 2020)
  - It provides a more efficient training method by predicting 100% of tokens instead of 15% of tokens



What happened after BERT?

- Models that handle long contexts (≫ 512 tokens)
  - Longformer, Big Bird, …
- Multilingual BERT
  - Trained single model on 104 languages from Wikipedia. Shared 110k WordPiece vocabulary
- BERT extended to different domains
  - SciBERT, BioBERT, FinBERT, ClinicalBERT, …
- Making BERT smaller to use
  - DistillBERT, TinyBERT, …



(a) Random attention (b) Window attention

(c) Global Attention (d) BIGBIRD

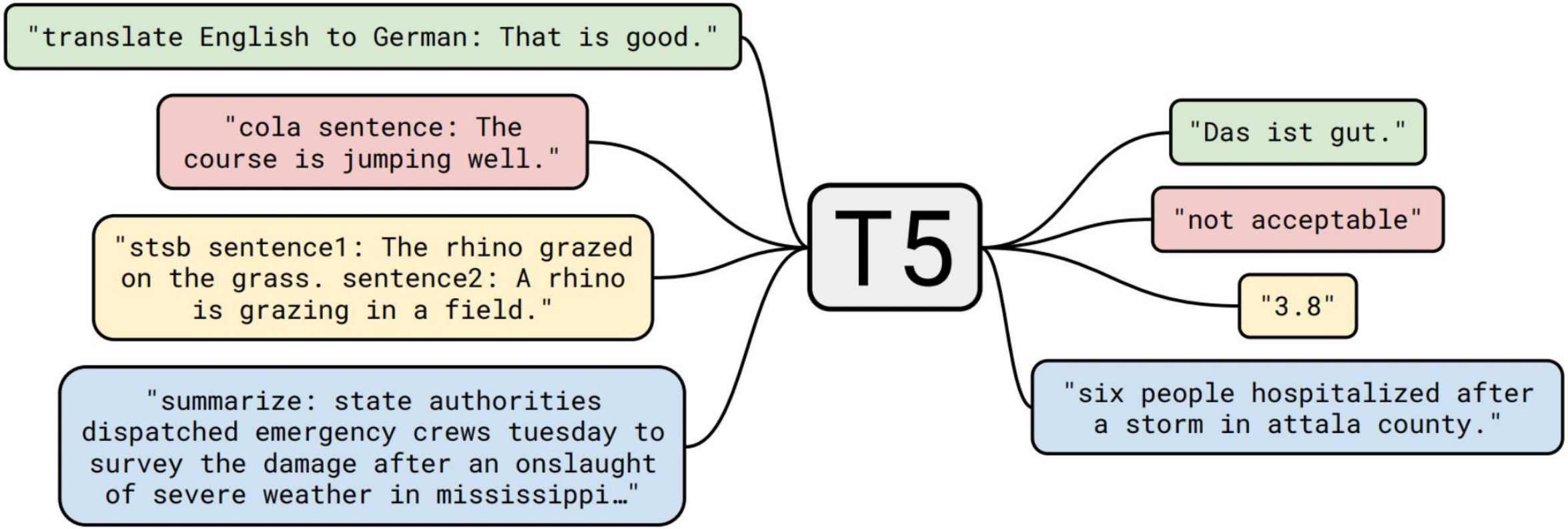
Image from the original paper

Encoder-decoder: T5

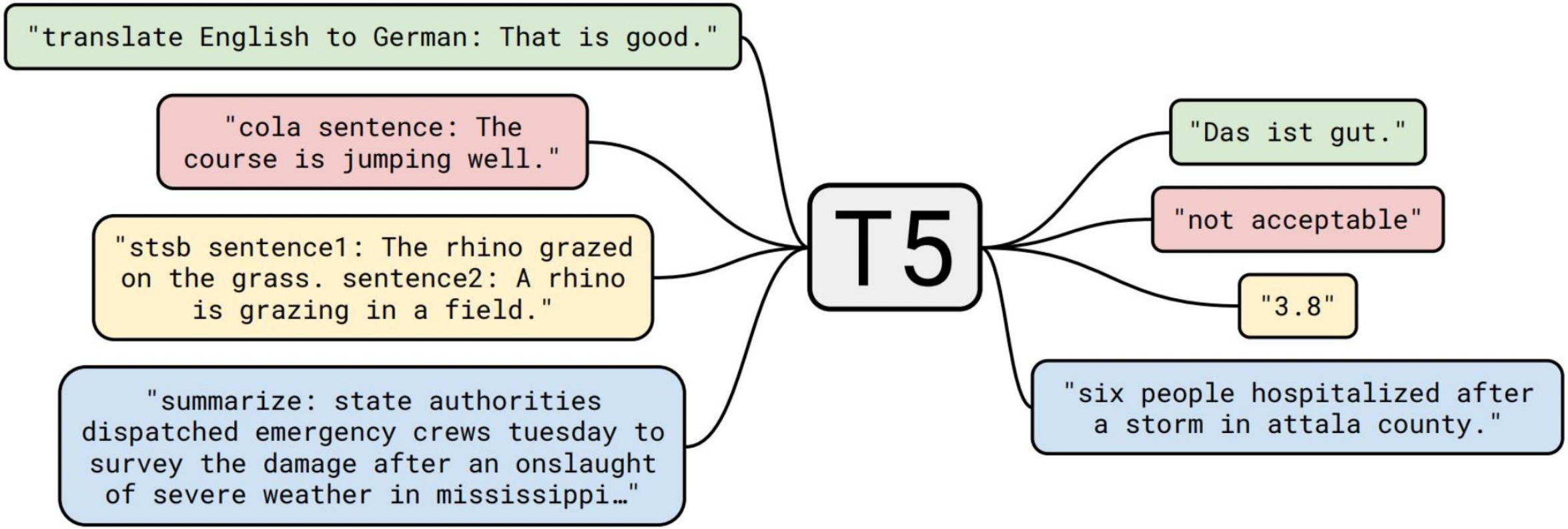
- **T**ext-**t**o-**T**ext **T**ransfer **T**ransformer
- Every task, one format!
  - Previous attempts included:
    - Question answering
    - Language modeling
    - Span extraction

    … but had limitations
- “[Task-specific prefix]: [Input text]” -> “[output text]”

Encoder-decoder: T5



Encoder-decoder: T5



Some tasks

- CoLA (GLUE): Sentence acceptability
  - **Input**: sentence, **output**: labels “acceptable” or “not acceptable”
  - Ex: “The course is jumping well.” -> not acceptable
- STS-B (GLUE): Sentence similarity
  - **Input**: pair of sentences, **output**: similarity score [1,5]
  - Ex: “sentence1: The rhino grazed. sentence2: A rhino is grazing.” -> 3.8

Some tasks

- COPA (SuperGLUE): Causal reasoning
  - **Input**: premise and 2 alternatives, **output**: alternative1 or alternative2
  - Ex: “Premise: I tipped the bottle. What happened as a RESULT?
    Alternative 1: The liquid in the bottle froze.
    Alternative 2: The liquid in the bottle poured out.”
    -> alternative2
- ReCoRD/MultiRC (SuperGLUE): Question answering/Reading comprehension

Input/Output

[Task-specific prefix]: [Input text]

- EnDe (Translation):
  "translate English to German: That is good" -> "Das ist gut"

- CNNDM (Summarization):
  "summarize: state authorities dispatched…" -> "six people hospitalized after storm"

Input/Output

[Task-specific prefix]: [Input text]

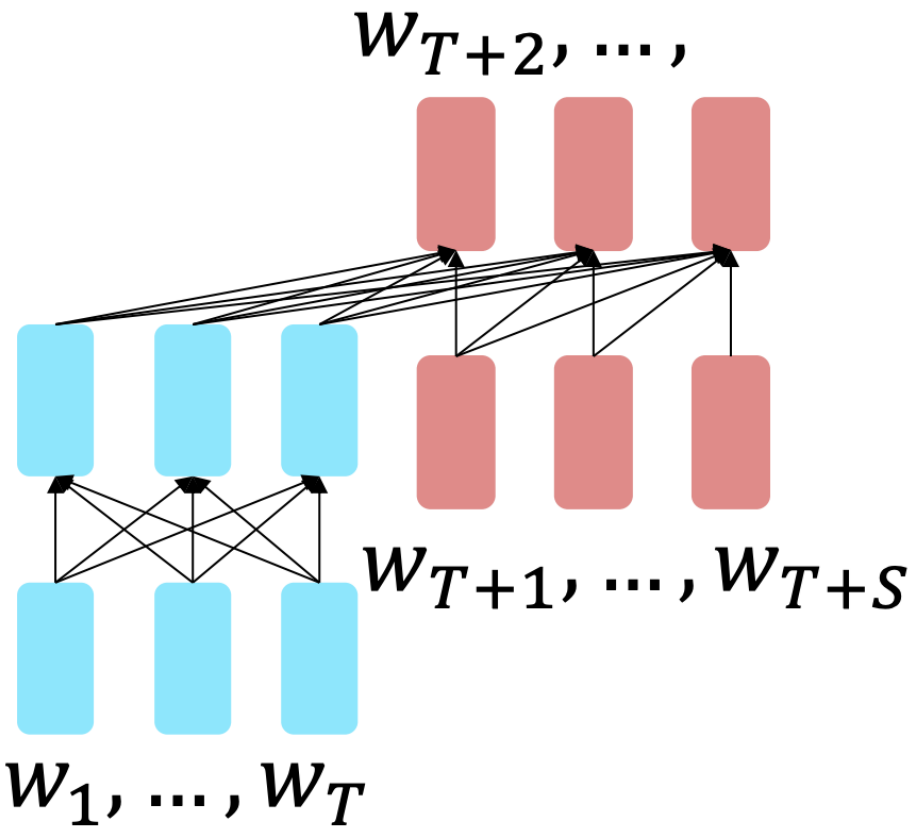
- CoLA (GLUE; Classification):
  "cola sentence: The course is jumping well." -> "not acceptable"

- STS-B (GLUE; Regression):
  "stsb sentence1: The rhino grazed. sentence2: A rhino is grazing." -> "3.8"

Pretraining encoder-decoders

$$h_1, \ldots, h_T = \text{Encoder}(w_1, \ldots, w_T)$$
$$h_{T+1}, \ldots, h_{T+S} = Decoder(w_{T+1}, \ldots, w_{T+S}, h_1, \ldots, h_T)$$
$$y_i \sim Ah_i + b, i > T$$

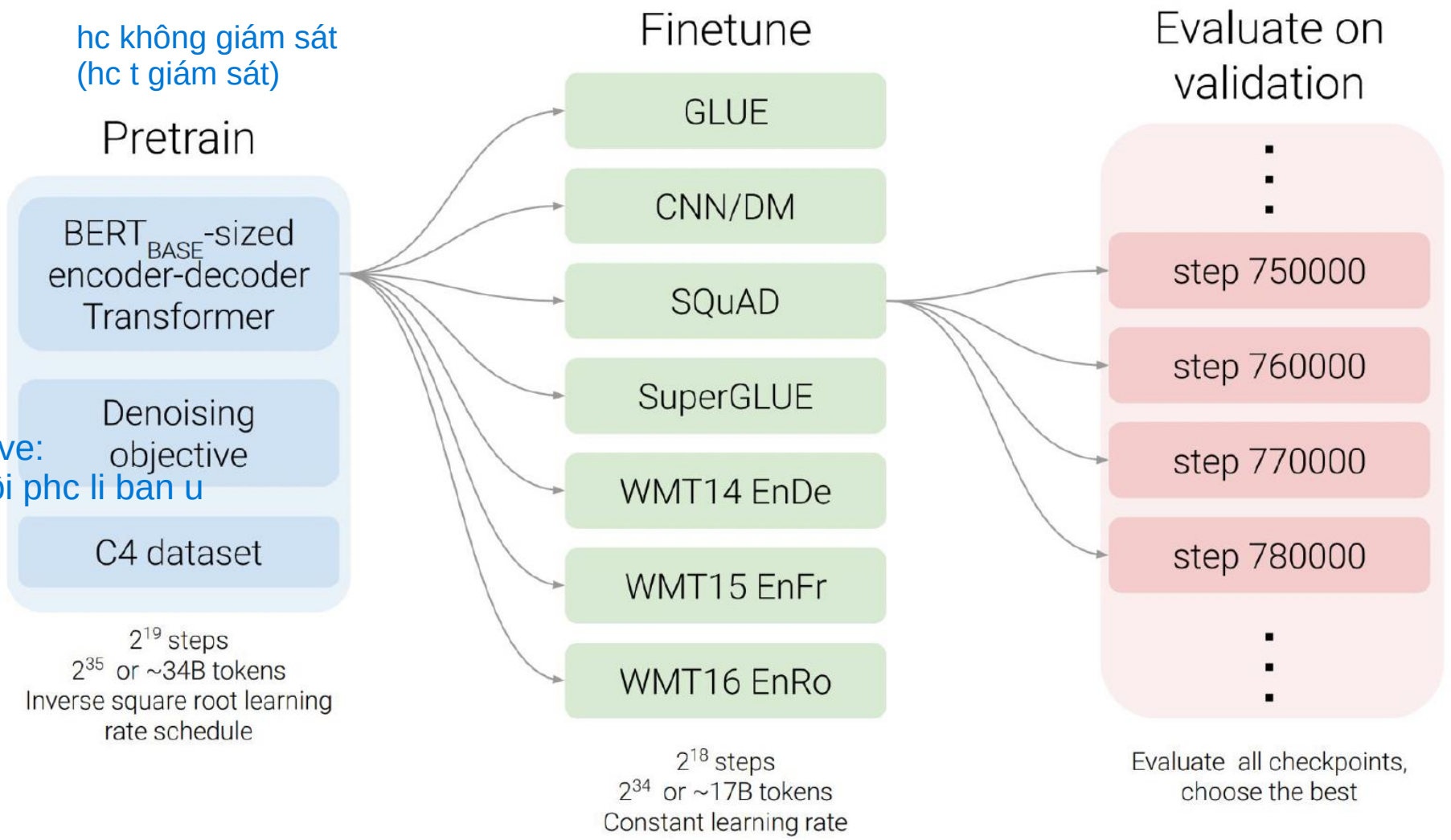
The **encoder** portion can benefit from bidirectional context; the **decoder** portion is used to train the whole model through language modeling, autoregressively predicting and then conditioning on one token at a time.



Workflow

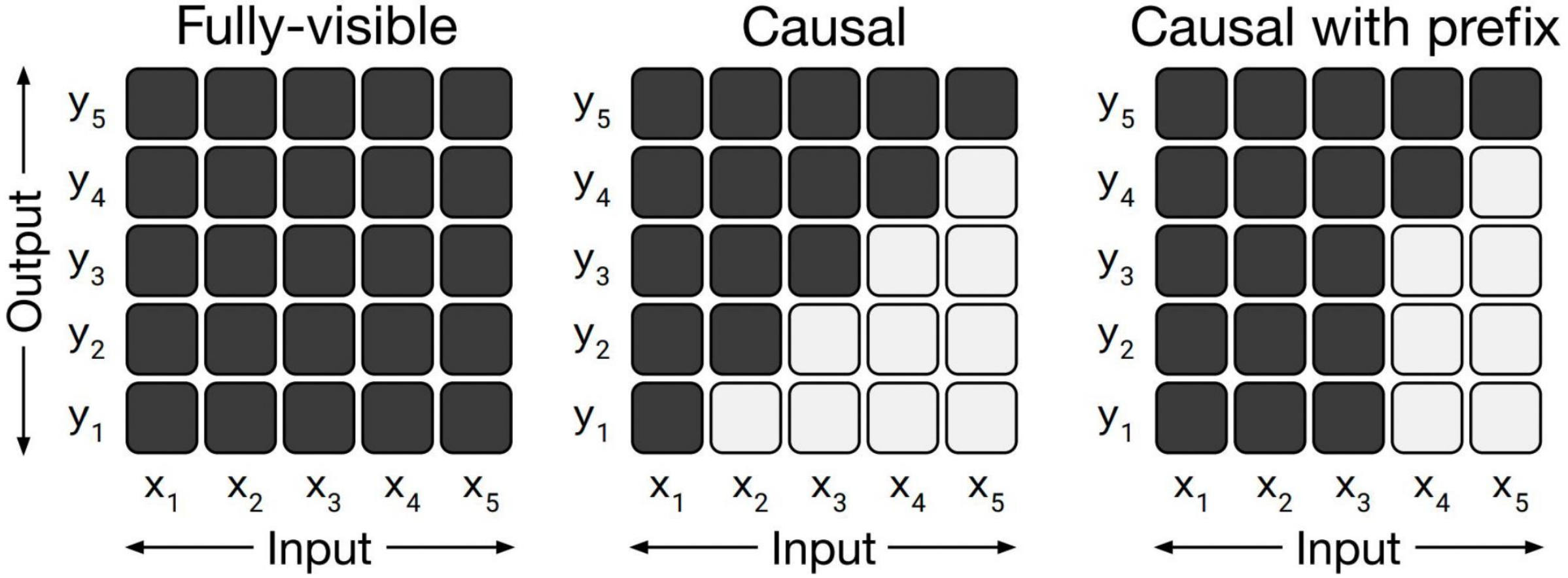
hc không giám sát
(hc t giám sát)

hc có giám sát

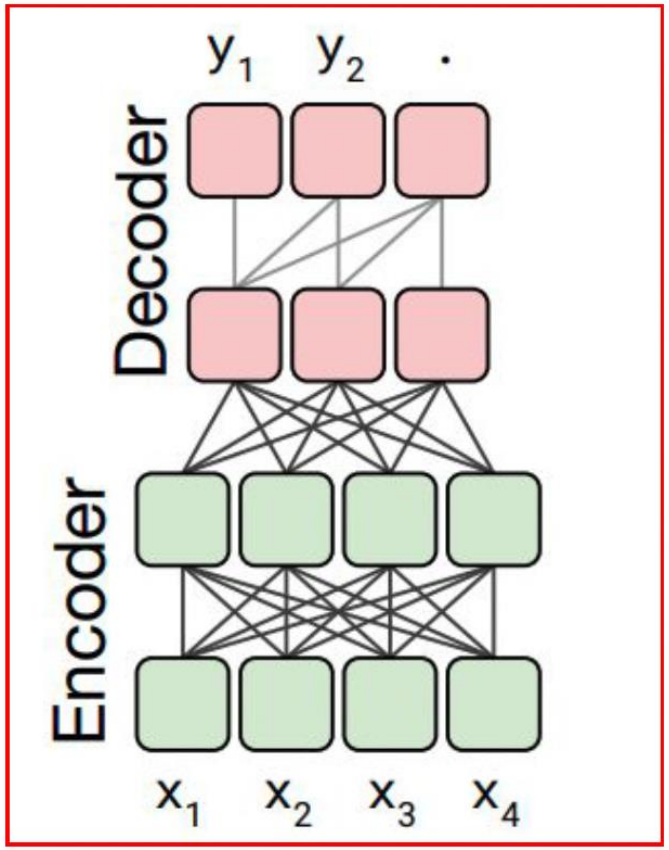


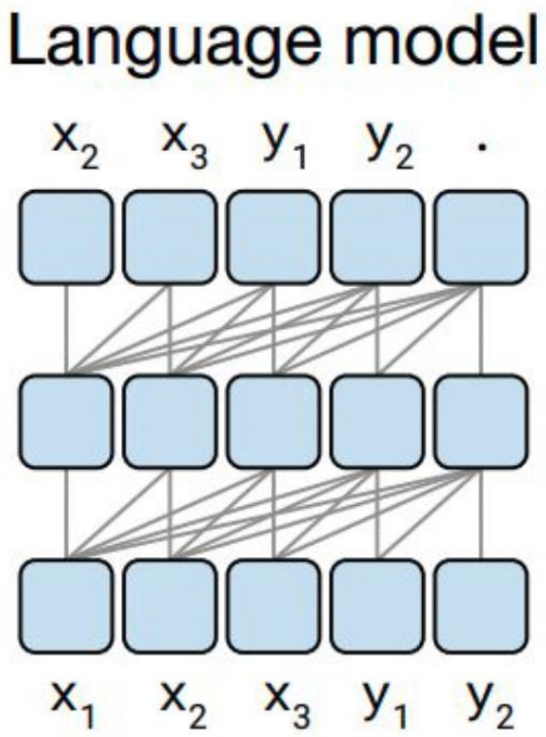
denoising objective:
thêm nhiu, bt khôi phc li ban u

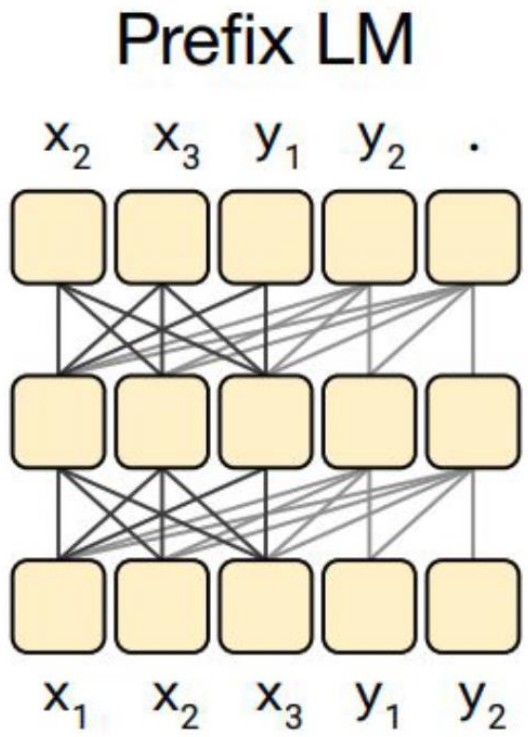
Architectures



Architectures



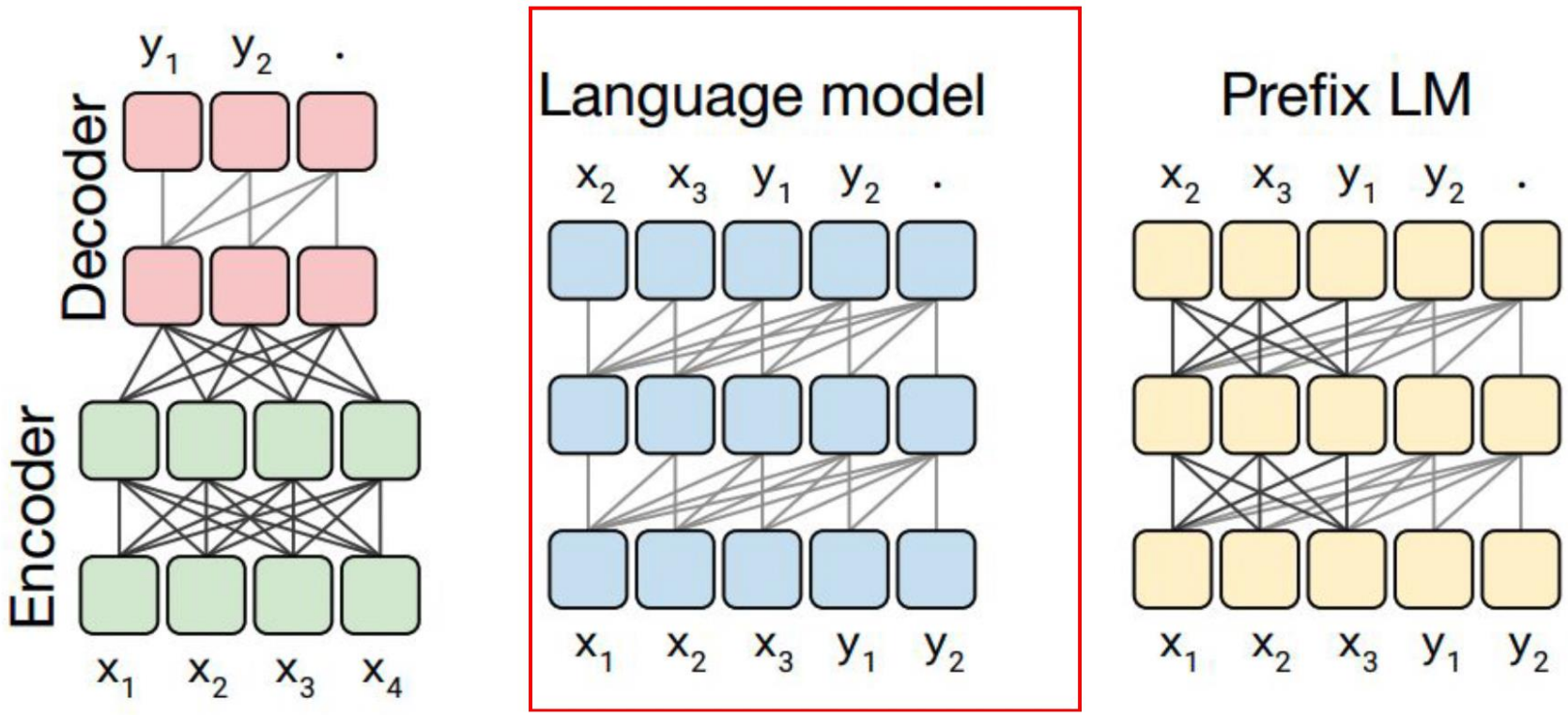




Translation: That is good -> Das ist gut.

Translate English to German: That is good. Target: Das is gut.

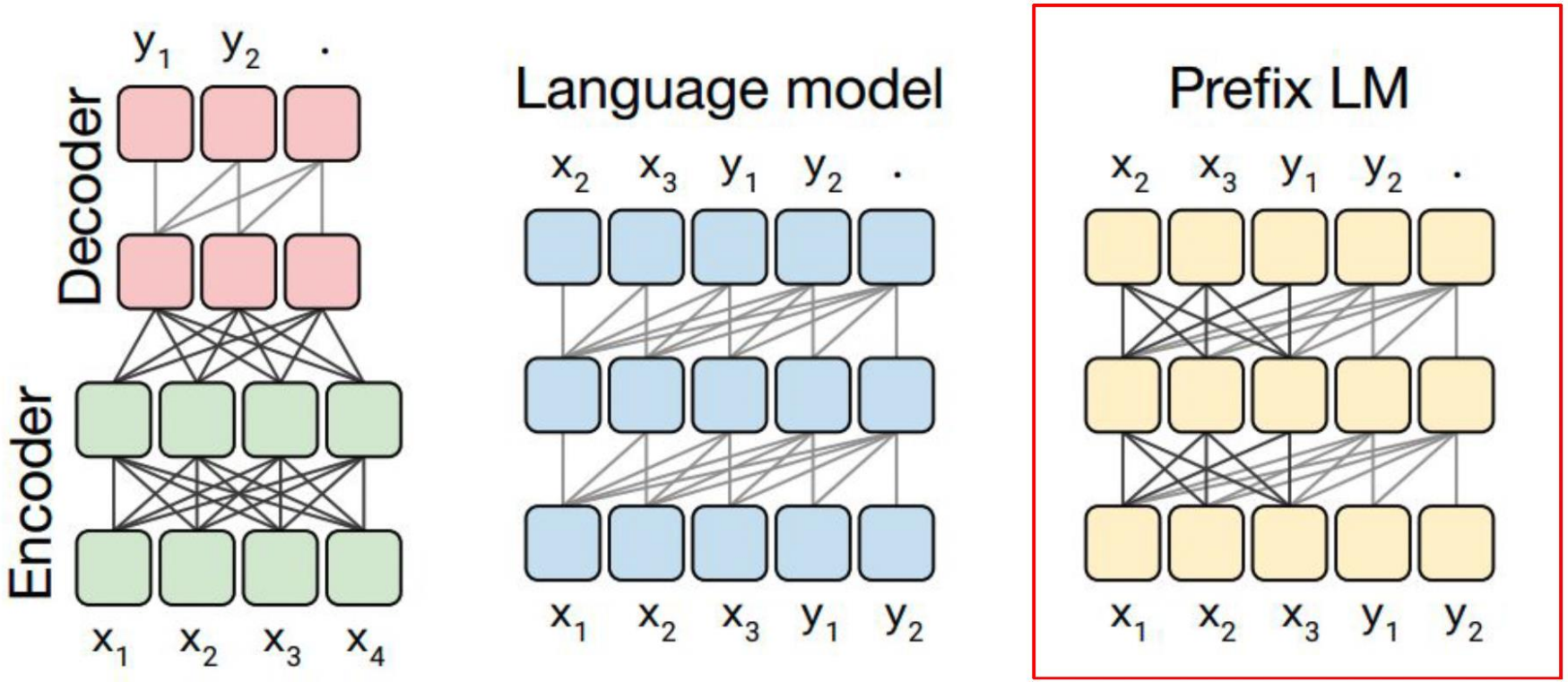
Architectures



Translation: That is good -> Das ist gut.

- Translate English to German: That is good. Target: Das is gut.
  - "Good" representation can only look at "Translate English to German: That is".

Architectures



Translation: That is good -> Das ist gut.

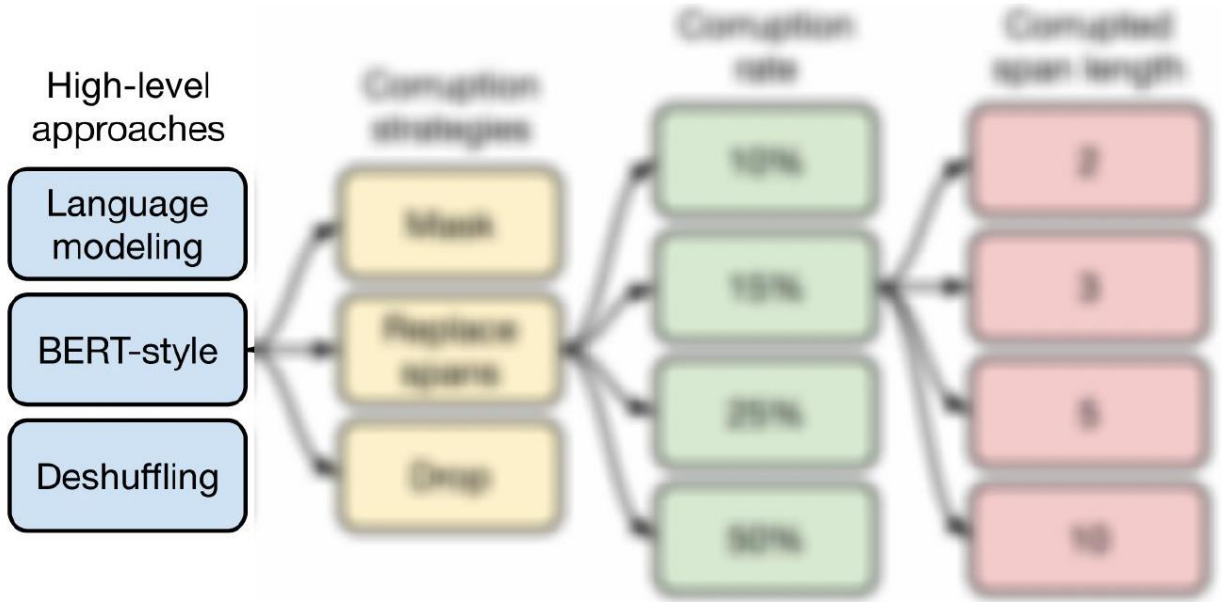
- Translate English to German: That is good. Target: Das is gut.
  - "Good" representation can look at "Translate English to German: That is. Target:".

Performance of different Architectures

Architecture	Objective	Params	Cost	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Encoder-decoder	Denoising	$2P$	M	**83.28**	**19.24**	**80.88**	**71.36**	**26.98**	**39.82**	**27.65**
Enc-dec, shared	Denoising	P	M	82.81	18.78	**80.63**	**70.73**	26.72	39.03	**27.46**
Enc-dec, 6 layers	Denoising	P	$M/2$	80.88	18.97	77.59	68.42	26.38	38.40	26.95
Language model	Denoising	P	M	74.70	17.93	61.14	55.02	25.09	35.28	25.86
Prefix LM	Denoising	P	M	81.82	18.61	78.94	68.11	26.43	37.98	27.39

1. Sharing parameters in encoder and decoder models perform nearly as well as the baseline.
2. Halving the number of layers in encoder and decoder hurts the performance.
3. Performance of Encoder and Decoder with shared parameters is better than decoder only LM and prefix LM.

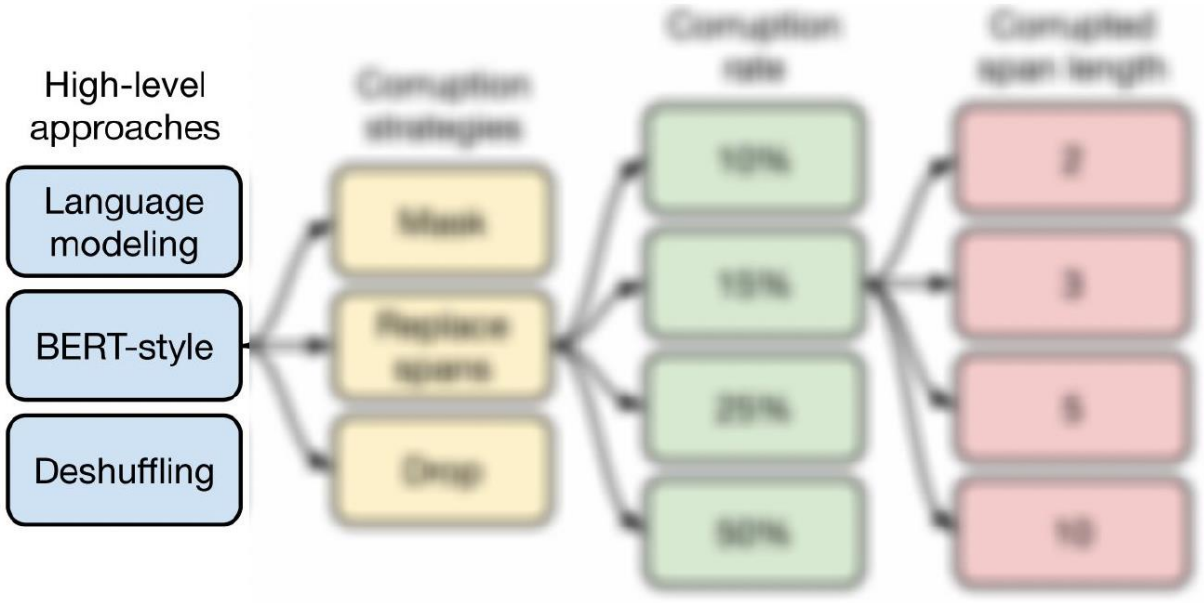
Different Unsupervised Objectives



3 kiu

	Objective	Inputs	Targets
1	Prefix language modeling	Thank you for inviting	me to your party last week .
2	BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	*(original text)*
3	Deshuffling	party me for your to . last fun you inviting week Thank	*(original text)*

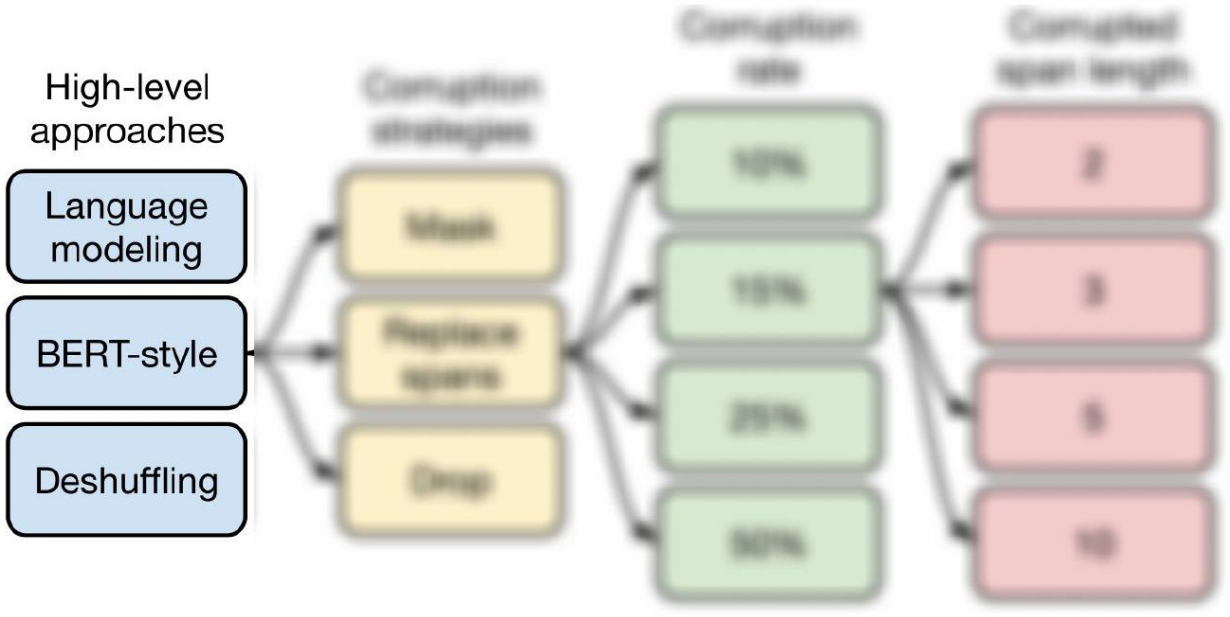
Different Unsupervised Objectives



Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	*(original text)*
Deshuffling	party me for your to . last fun you inviting week Thank	*(original text)*

- Thank you for inviting me to your party last week.

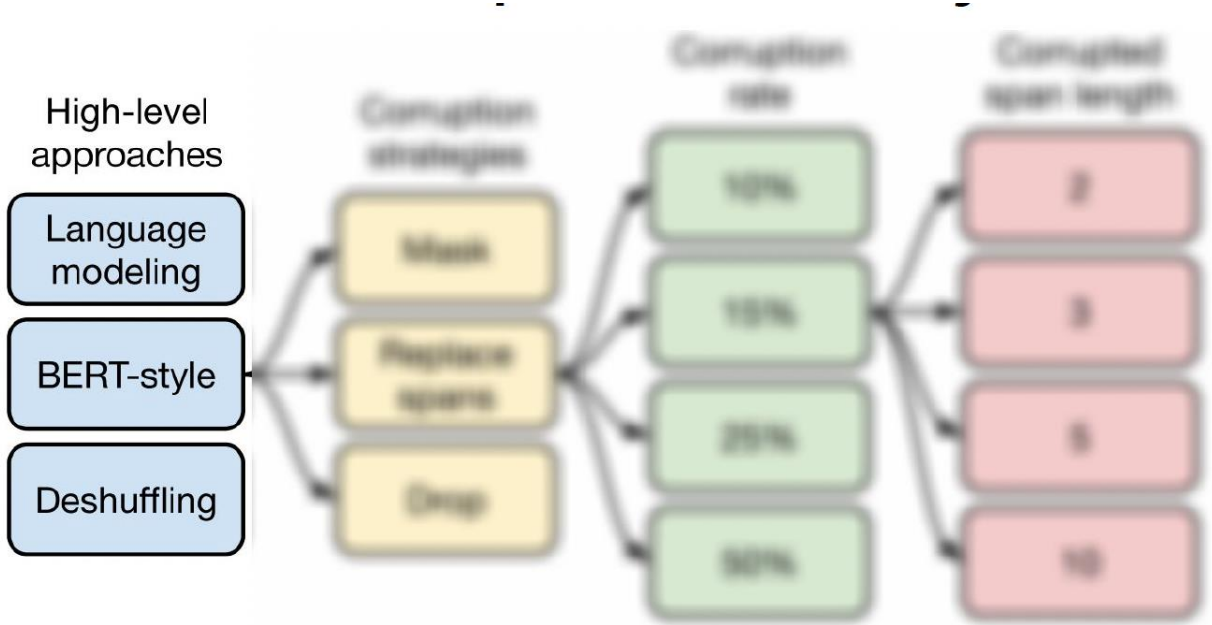
Different Unsupervised Objectives



Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	*(original text)*
Deshuffling	party me for your to . last fun you inviting week Thank	*(original text)*

- Thank you <M> <M> me to your party apple week . Thank you for inviting me to your party last week.

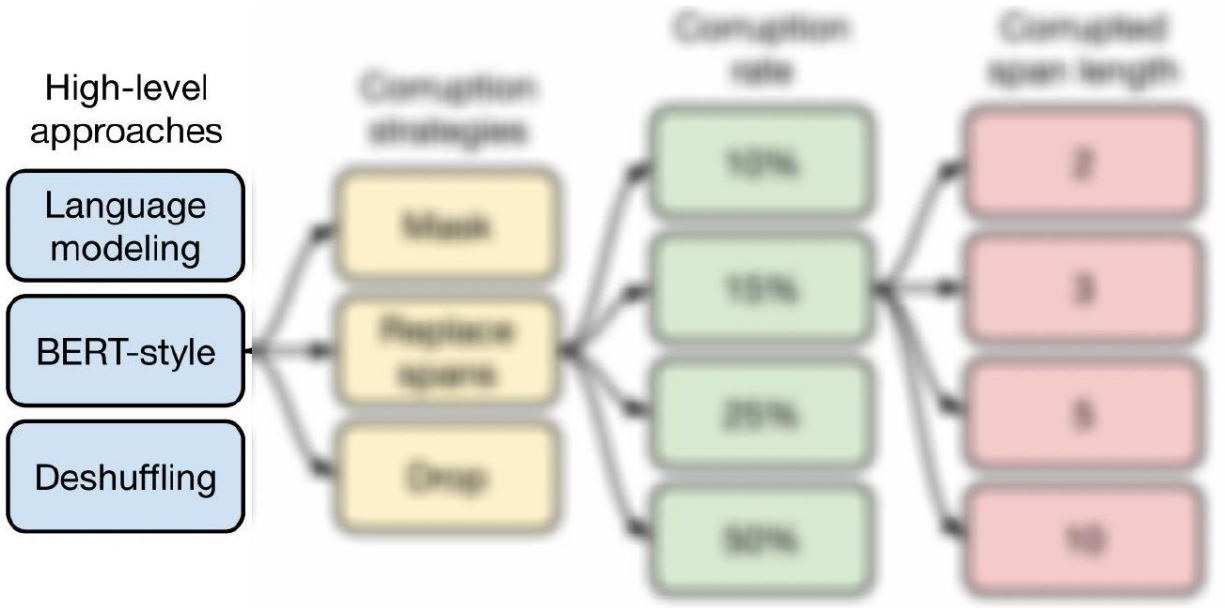
Different Unsupervised Objectives



Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	*(original text)*
Deshuffling	party me for your to . last fun you inviting week Thank	*(original text)*

- party me for your to . last fun you inviting week Thank . Thank you for inviting me to your party last week.

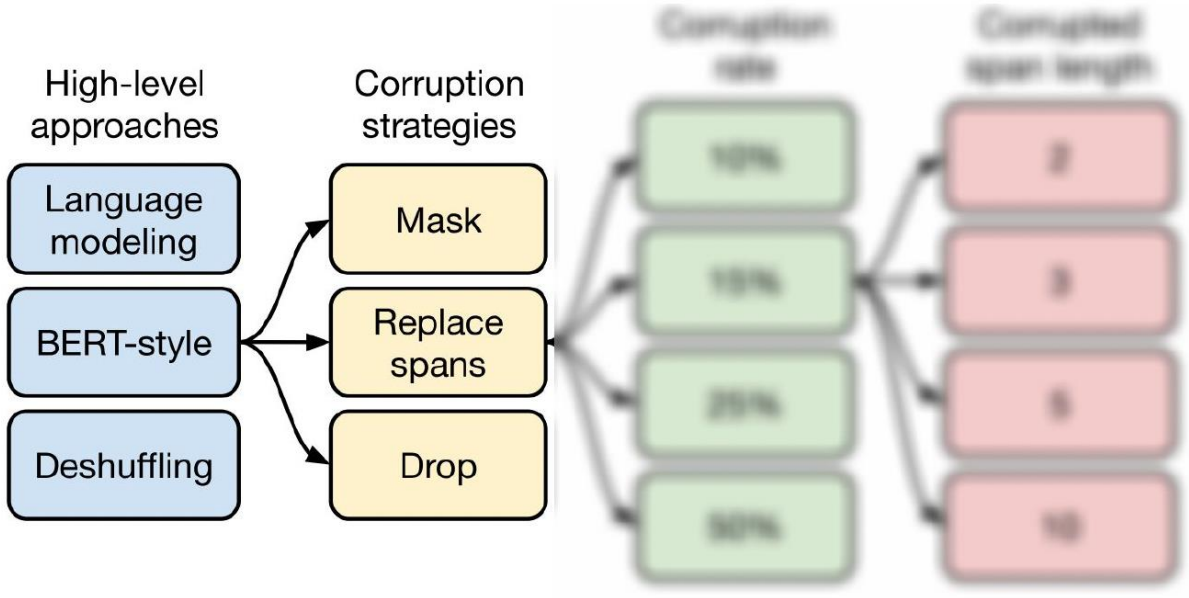
Different Unsupervised Objectives



1. BERT-style objective performs best.
2. Prefix LM works well on translation tasks.
3. Deshuffling objective is significantly worse.

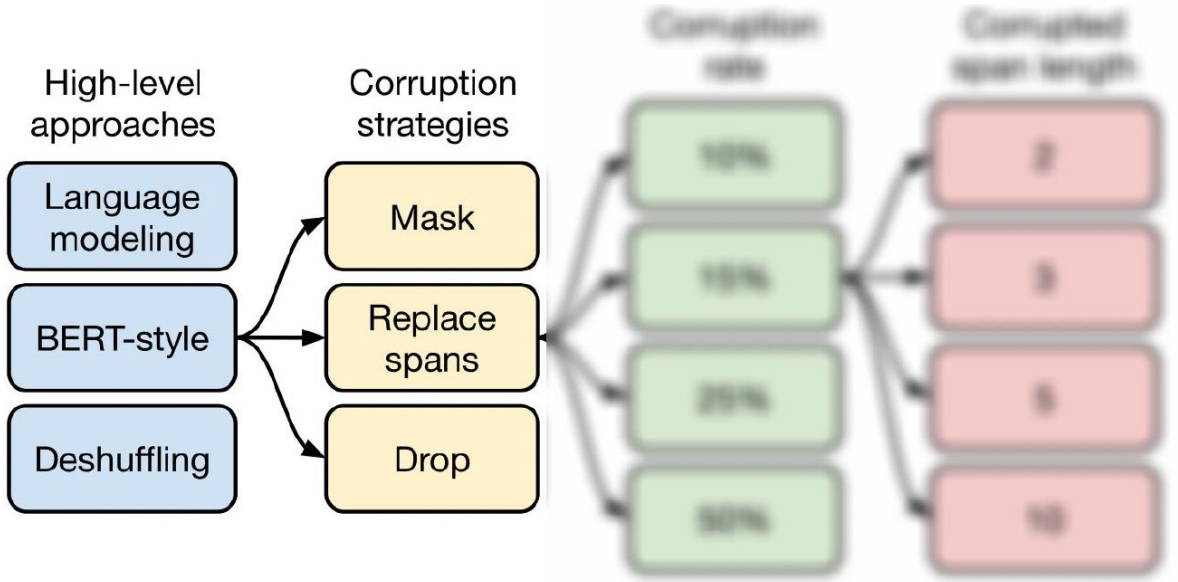
Objective	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
Prefix language modeling	80.69	18.94	77.99	65.27	**26.86**	39.73	**27.49**
BERT-style (Devlin et al., 2018)	**82.96**	**19.17**	**80.65**	**69.85**	**26.78**	**40.03**	**27.41**
Deshuffling	73.17	18.59	67.61	58.47	26.11	39.30	25.62

Different Unsupervised Objectives



Objective	Inputs	Targets
MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	*(original text)*
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last

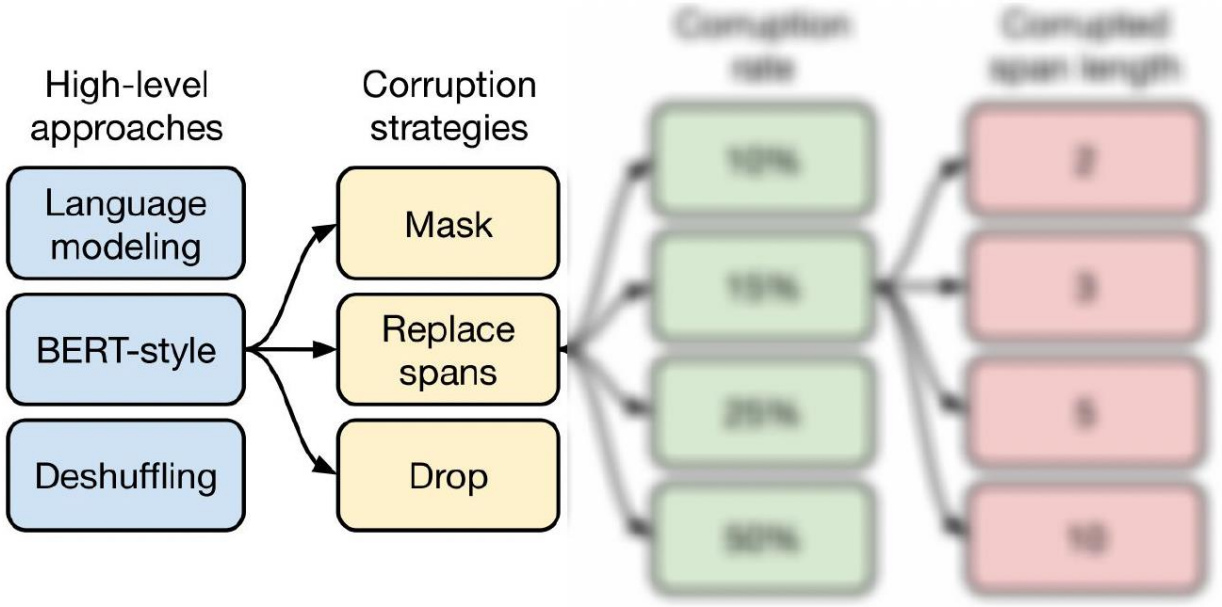
Different Unsupervised Objectives



- Thank you <M> <M> me to your party <M> week . Thank you for inviting me to your party last week

MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	*(original text)*
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last

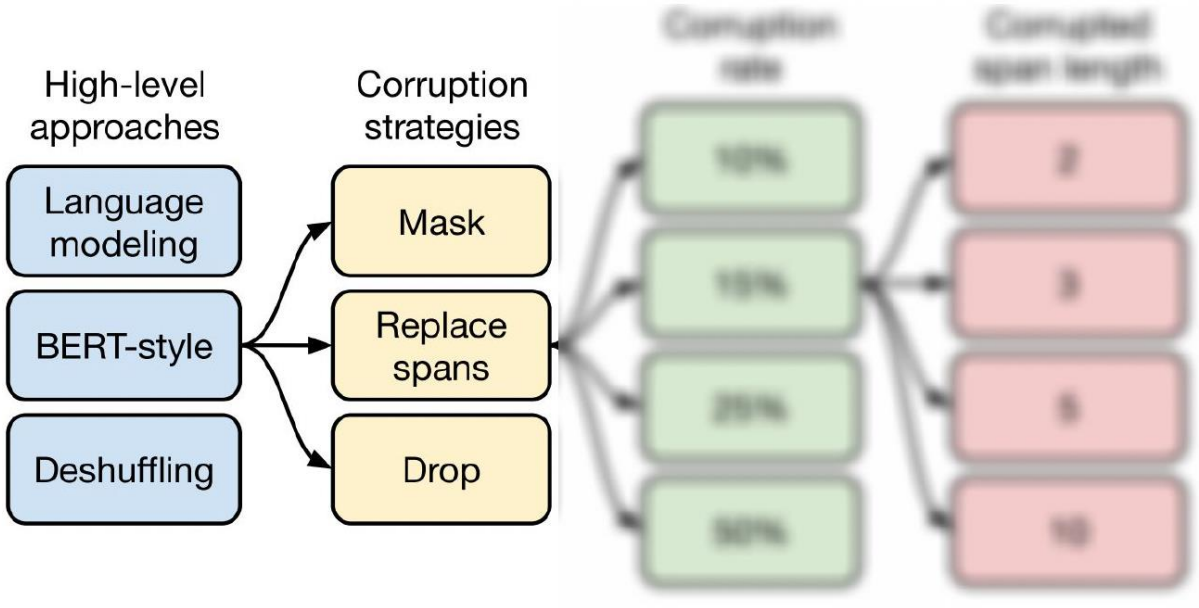
Different Unsupervised Objectives



- Thank you <X> me to your party <Y> week . <X> for inviting <Y> last <Z>

MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	*(original text)*
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last

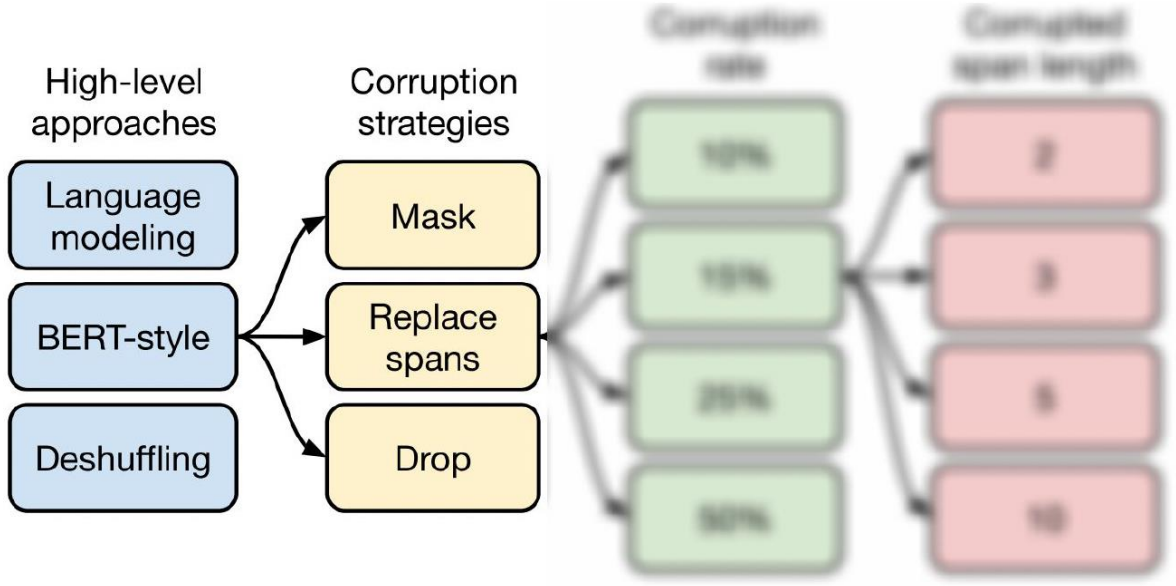
Different Unsupervised Objectives



- Thank you me to your party week . for inviting last

MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	*(original text)*
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last

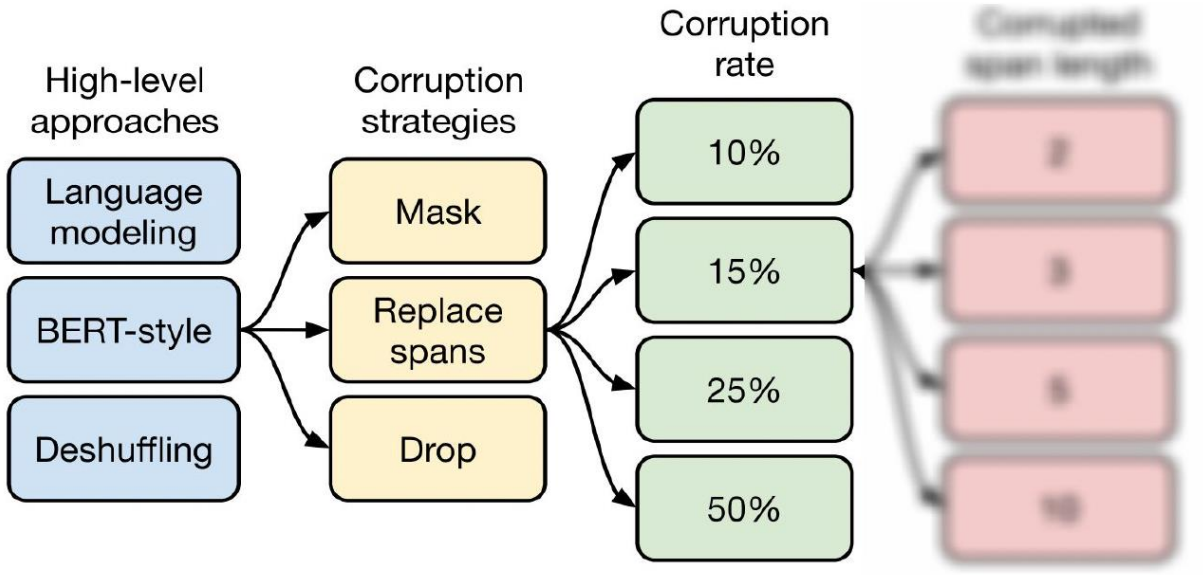
Different Unsupervised Objectives



1. All the variants perform similarly.
2. "Replace corrupted spans" and "Drop corrupted tokens" are more appealing because target sequences are shorter, speeding up training.

Objective	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
BERT-style (Devlin et al., 2018)	82.96	19.17	**80.65**	69.85	26.78	**40.03**	27.41
MASS-style (Song et al., 2019)	82.32	19.16	80.10	69.28	26.79	**39.89**	27.55
★ Replace corrupted spans	83.28	**19.24**	**80.88**	**71.36**	**26.98**	39.82	**27.65**
Drop corrupted tokens	**84.44**	**19.31**	**80.52**	68.67	**27.07**	39.76	**27.82**

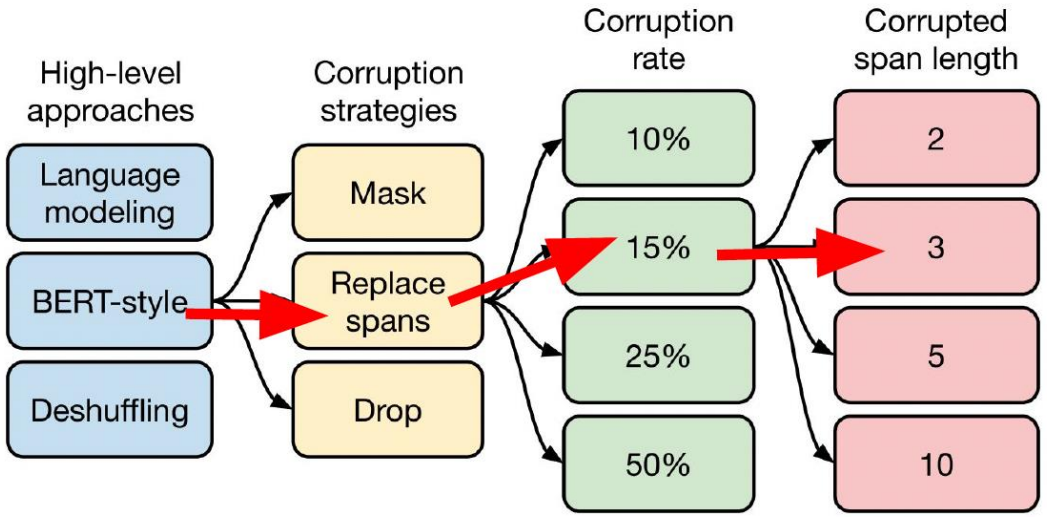
Different Unsupervised Objectives



1. Larger corruption rate leads to downstream performance degradation.
2. Larger corruption rate also leads to longer targets, slowing down training.

Corruption rate	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
10%	**82.82**	19.00	**80.38**	69.55	**26.87**	39.28	**27.44**
★ 15%	**83.28**	19.24	**80.88**	**71.36**	**26.98**	**39.82**	**27.65**
25%	**83.00**	**19.54**	**80.96**	70.48	**27.04**	**39.83**	**27.47**
50%	81.27	19.32	79.80	70.33	**27.01**	**39.90**	**27.49**

Different Unsupervised Objectives



1. Average span length of 3 works well on most non-translation tasks.
2. Span corruption produces shorter target sequences and leads to speedup in training.

Span length	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Baseline (i.i.d.)	**83.28**	19.24	80.88	71.36	**26.98**	**39.82**	**27.65**
2	**83.54**	19.39	**82.09**	**72.20**	**26.76**	**39.99**	**27.63**
3	**83.49**	**19.62**	**81.84**	**72.53**	**26.86**	39.65	**27.62**
5	**83.40**	19.24	**82.05**	**72.23**	**26.88**	39.40	**27.53**
10	82.85	19.33	**81.84**	70.44	**26.79**	39.49	**27.69**

References

1. CS224N: Natural Language Processing with Deep Learning: https://web.stanford.edu/class/cs224n/
2. COS 597G (Fall 2022): Understanding Large Language Models https://www.cs.princeton.edu/courses/archive/fall22/cos597G/

HUST

hust.edu.vn fb.com/dhbkhn

Xin trân trọng cảm ơn!